

Programação PHP

WBA0183_V1.0

Programação PHP

Autor: Prof. Cristiano Marçal Toniolo

Como citar este documento: TONIOLO, Cristiano Marçal. **Programação PHP.** Valinhos, 2016.

Sumário

Apresentação da Disciplina	04
Unidade 1: Instalando e Configurando as Ferramentas	05
Assista a suas aulas	41
Unidade 2: Conceitos Básicos de PHP	49
Assista a suas aulas	67
Unidade 3: Relacionamentos com Ambiente WEB, Desenvolver *Scripts* do Lado do Servidor PHP	75
Assista a suas aulas	91
Unidade 4: Ajax + PHP – Relacionamentos com ambiente WEB	94
Assista a suas aulas	117

Programação PHP

Autor: Prof. Cristiano Marçal Toniolo

Como citar este documento: TONIOLO, Cristiano Marçal. **Programação PHP.** Valinhos, 2016.

Sumário

Unidade 5: Banco de Dados com PHP, Utilização do Banco com a Linguagem PHP	120
Assista a suas aulas	146
Unidade 6: Desenvolvimento de Sites em PHP, Desenvolver e Manter Aplicações Dinâmicas para Internet com Interface Web, Criando Sites Dinâmicos com a Linguagem PHP	154
Assista a suas aulas	175
Unidade 7: Sistemas Open-Source em PHP, Criação e Apresentação	183
Assista a suas aulas	204
Unidade 8: Práticas com Frameworks PHP Criação e Apresentação	212
Assista a suas aulas	227

Apresentação da Disciplina

A disciplina de Programação PHP tem como objetivo introduzir ao aluno os conceitos relacionados à programação para páginas WEB, usando as ferramentas mais comuns: o banco de dados MYSQL, o servidor Web Apache e a linguagem de programação PHP. Esta também tem como foco abordar os conceitos básicos e avançados da linguagem PHP, suas aplicações e seus recursos mais utilizados atualmente, bem como “o que” e “como” são utilizados os *frameworks* de desenvolvimento juntamente com o banco de dados. Além disso, será realizada uma revisão do que é a tecnologia cliente/servidor, muito importante para o entendimento da programação para WEB e o TCP como protocolo de transferência, e serão retomados os conceitos de programação que o aluno deverá saber para dar continuidade com o curso e com a disciplina, são eles: os comandos básicos de lógica, como a programação estruturada, as decisões e os *loops*. Esta disciplina também se pautará na linguagem de marcação, que é o HTML, seus conceitos e *tags* básicas, usadas juntamente com a linguagem PHP, pois, diferentemente do que muitos profissionais analisam, a linguagem HTML ainda dá suporte para as linguagens de programação da internet, por causa do fato de que os interpretadores devolvem somente para o navegador do usuário o código em HTML.

Unidade 1

Instalando e Configurando as Ferramentas

Objetivos

1. O objetivo desta seção é instalar as ferramentas que serão utilizadas durante toda a disciplina, além de orientar todas suas configurações e seus parâmetros.
2. As ferramentas que utilizaremos são o MySQL, o Apache e o PHP.
3. Será feita a instalação do XAMPP, uma aplicação que configura todas as ferramentas para programar em PHP de maneira automática.

Introdução

Desde o início, a linguagem PHP sempre teve como "parceiros" o servidor WEB, o Apache e o banco de dados MySQL. Nesse contexto, o objetivo desta seção é instalar todas essas ferramentas para que, assim, possamos dar continuidade com a disciplina. O intuito é trabalhar dentro das características de cliente/servidor, em que o processamento das informações acontece no lado do servidor, enquanto que o cliente fica responsável por enviar as solicitações dos usuários e receber as repostas de volta.

Todas as ferramentas citadas e que serão utilizadas na disciplina são *open source*, ou seja, tem uma licença de uso que permite o uso delas sem ter que pagar por isso. Dessa forma, trabalharemos de forma ética e com *softwares* legalizados.

Para saber mais

Para saber mais sobre licença open source e outras licenças, acesse: <https://opensource.org/licenses>. Acesso em: 2 set. 2016.

Cada uma das ferramentas tem um propósito específico quando se trata de programação WEB. Iniciemos com o PHP, um interpretador que será instalado e, no momento que um código for solicitado pelo navegador WEB, fará a interpretação desse código, mostrando o resultado na tela do navegador para o usuário. Esse código será processado no lado do servidor WEB, ou seja, a resposta trará somente o código em HTML.

O servidor WEB Apache, quando instalado, tem a função de fazer que a página WEB dinâmica (veremos mais no Tema 02) seja processada retornando a informação necessária. Para isso, utilizamos um endereço WEB que chamamos de *loopback*, ou seja, este endereço somente funcionará no ambiente de desenvolvimento local, a fim de que se possa fazer os testes necessários para o funcionamento das páginas.

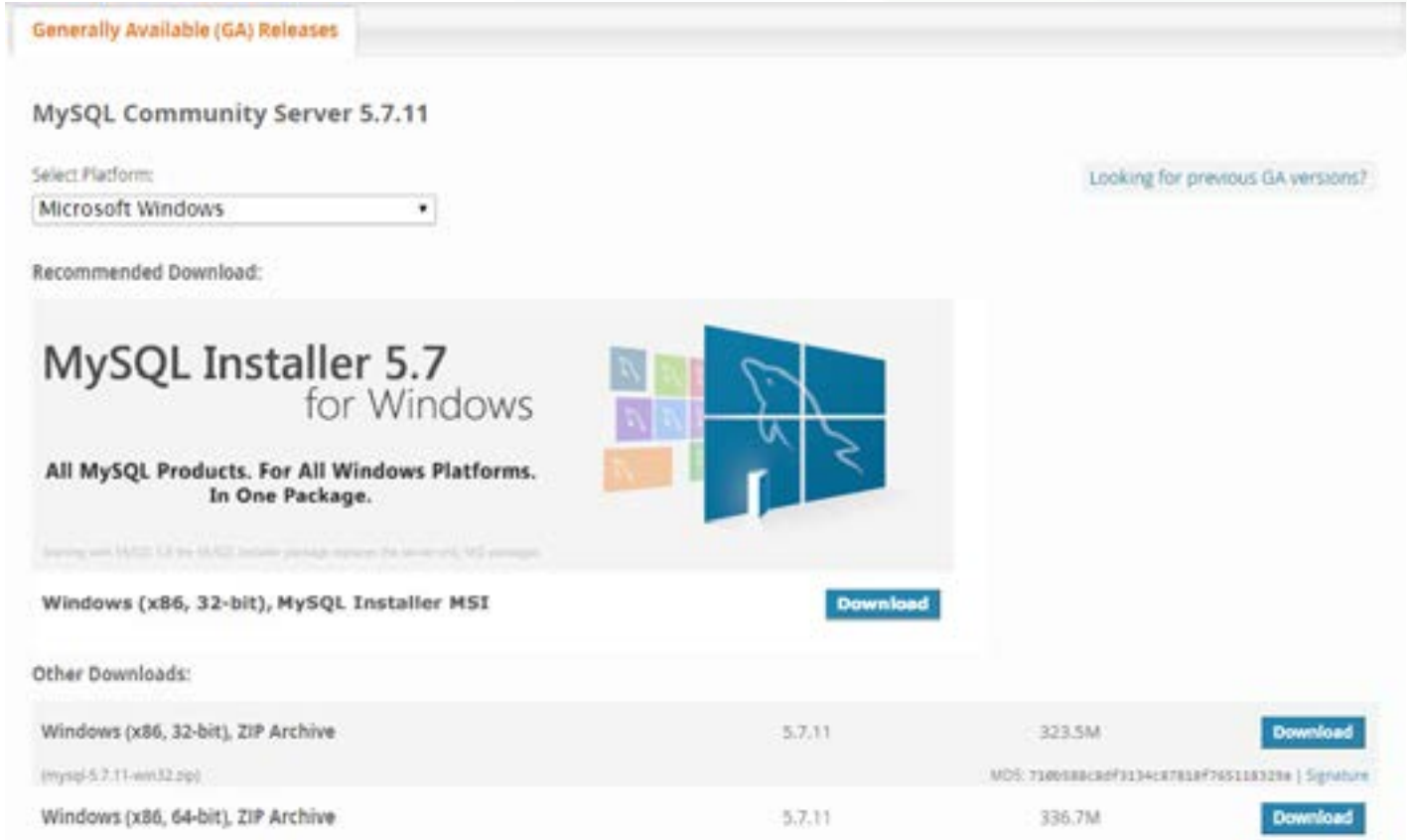
Já o banco de dados MySQL funcionará do lado do servidor e tem como funcionalidade o armazenamento de informações dentro de tabelas criadas, especialmente para o funcionamento de aplicações WEB que necessitem manter os dados seguros e que permitem, também, a recuperação desses dados. Assim, conseguimos notar que todas as três ferramentas devem trabalhar em conjunto para que o desenvolvimento de uma aplicação WEB, em PHP, tenha sucesso. Saber como tudo isso funciona é parte fundamental desse sucesso.

1. MySQL

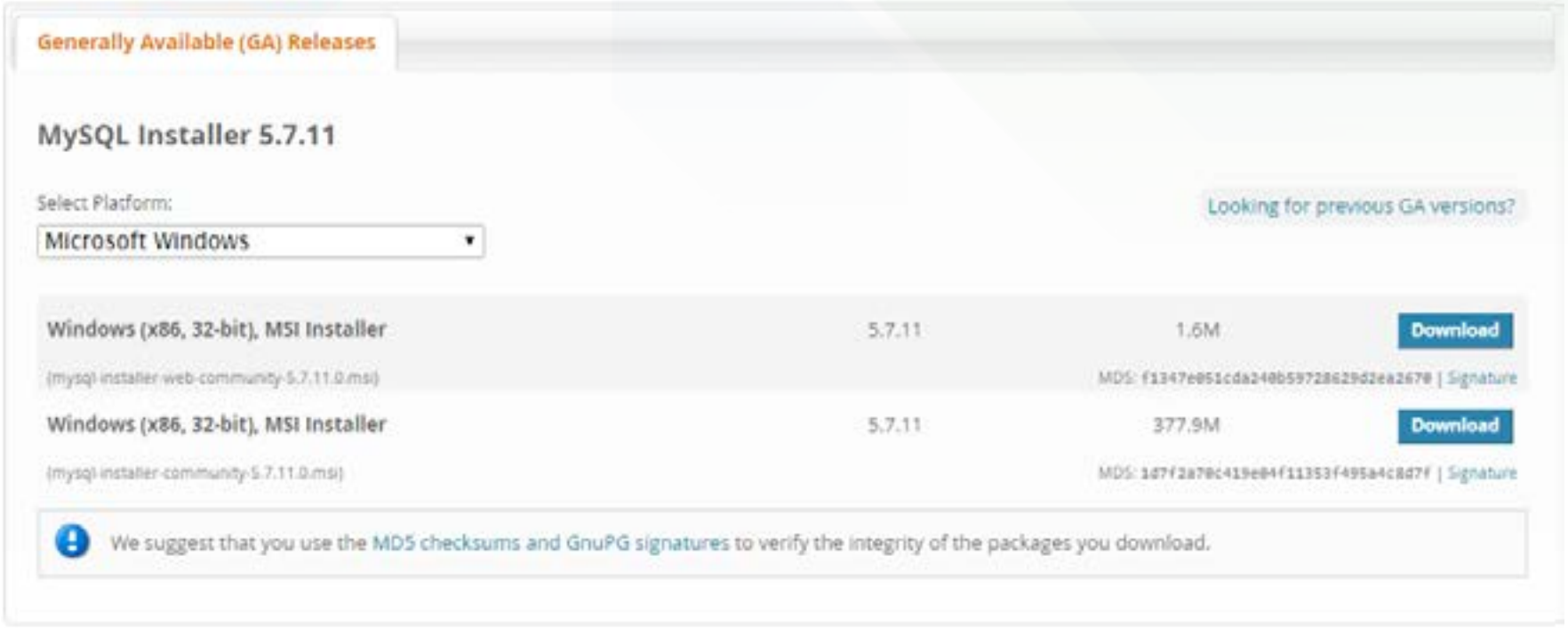
MySQL é o banco de dados *open source*, mundialmente conhecido, que permite a criação de banco de dados com suas tabelas de maneira rápida e com custo baixo, o que faz que se tenha produtos de performance relevante e que se possa ser escalável, ou seja, que permita o crescimento de suas aplicações de modo que sua atualização seja planejada para crescer.

A página para o *download* do MySQL é a seguinte: <http://dev.mysql.com/downloads/mysql/>. Acesso em: 2 set. 2016.

Ao acessar esse *link*, aparecerá a seguinte tela, com possível atualização de versão.



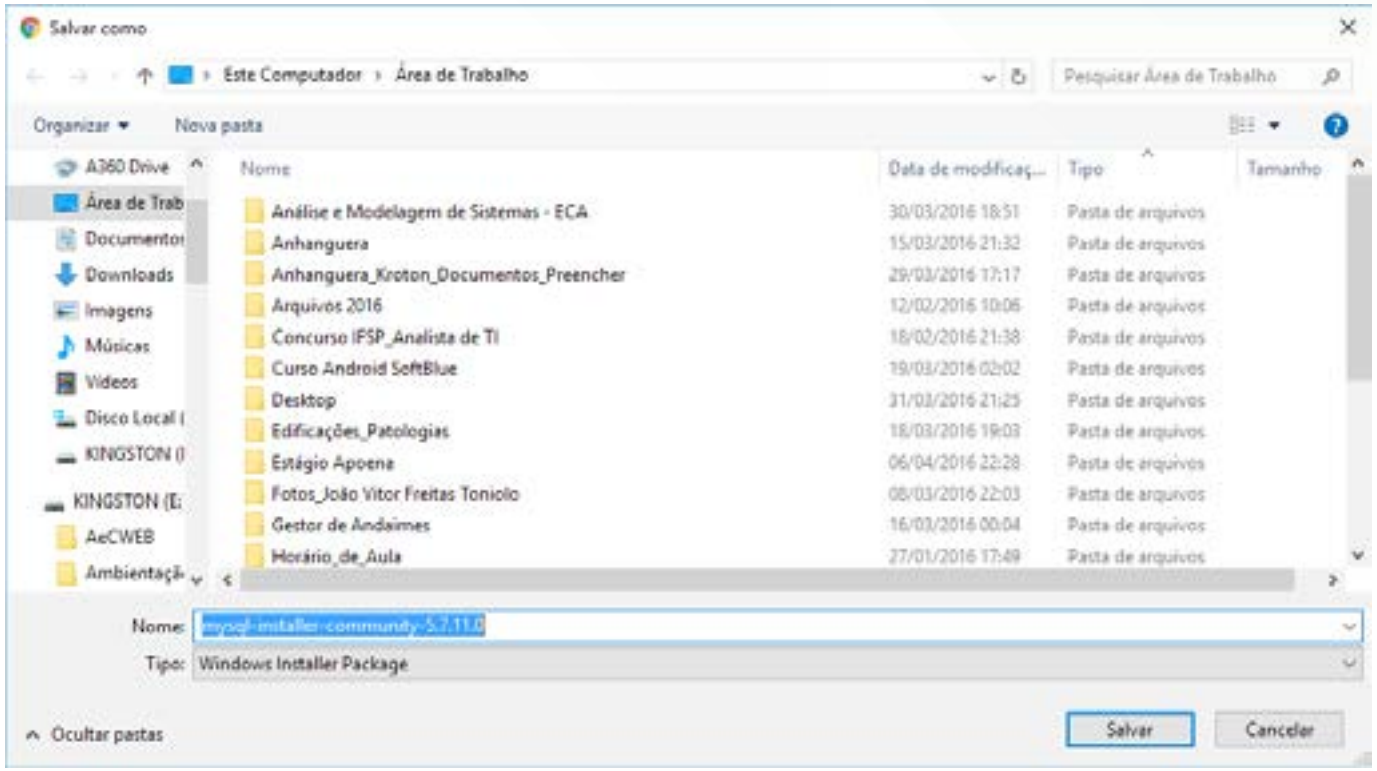
Na dropdown Select Platform, selecione a plataforma para que deseja criar seu ambiente de desenvolvimento. Em seguida, clique no botão *download*, o qual fará aparecer a seguinte tela:



Veremos os dois tipos de *download* que podemos realizar. O primeiro realiza o *download* de um instalador que necessita de acesso à internet para que possa ser realizada a instalação do MySQL. O segundo realiza o *download* do programa para o computador, realizando, assim, a

instalação *off-line*. Nesse caso, escolheremos a segunda opção clicando no botão *download*.

Na tela que aparecerá, será possível efetuar o *download* sem a necessidade de fazer um registro no site do MySQL, clicando sobre "*No thanks, just start my download*". Com base nisso, escolheremos o local em que o programa será gravado:

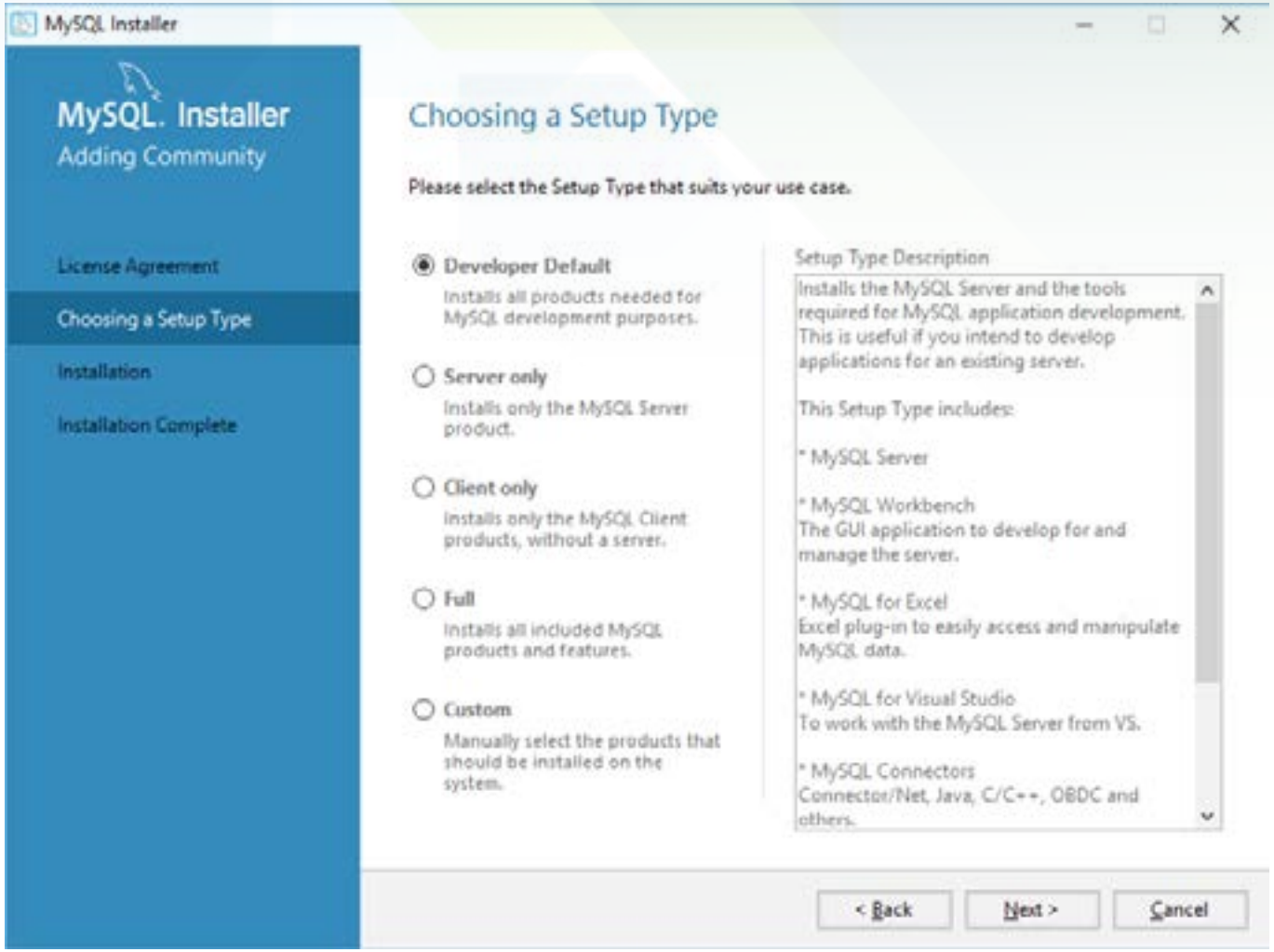


Depois que o *download* for realizado, partiremos para a instalação do MySQL. Para tanto, dê um duplo clique no executável do MySQL, seguindo os passos para a instalação.



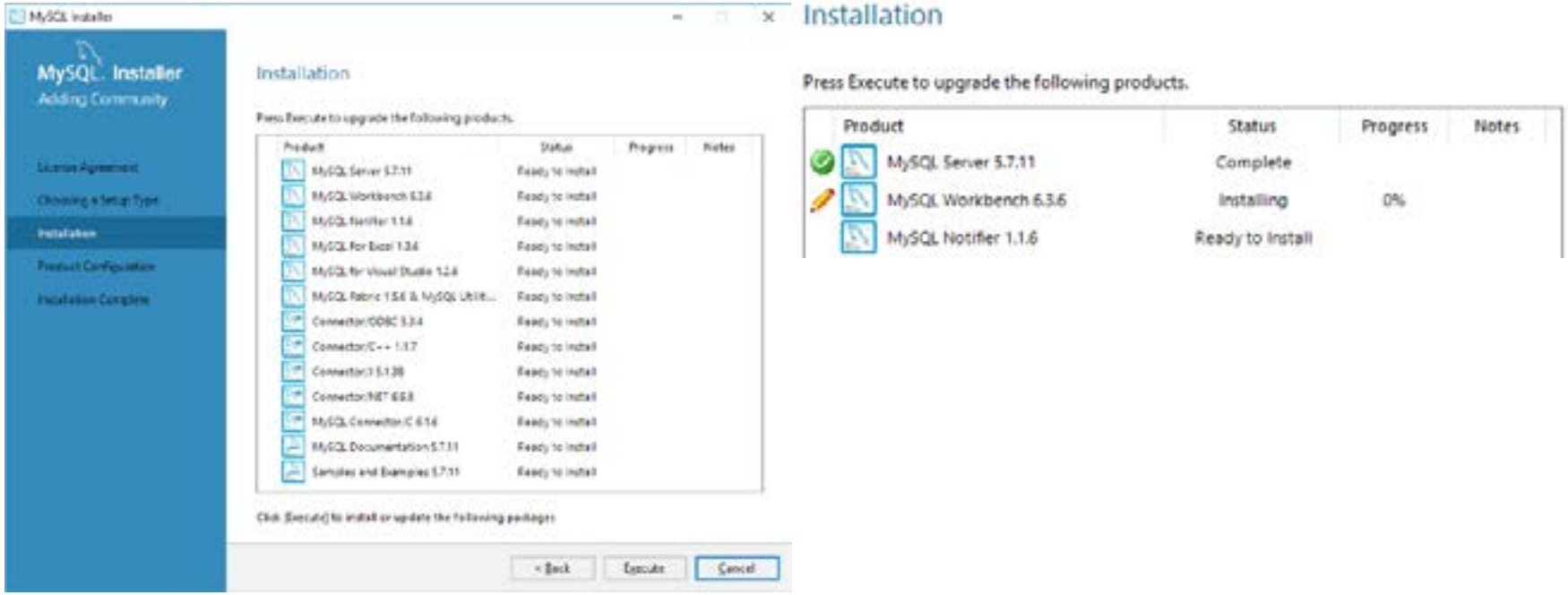
A partir desse ponto, já é possível executar a instalação do programa da mesma forma como é realizada com outros programas. Após aceitar os termos de licença, é importante configurarmos a instalação adequada do tipo de instalação do MySQL.

Existem vários tipos de instalação, cada um para uma determinada funcionalidade, como no caso da disciplina é o desenvolvimento de aplicações WEB, optamos pelo Developer Default.

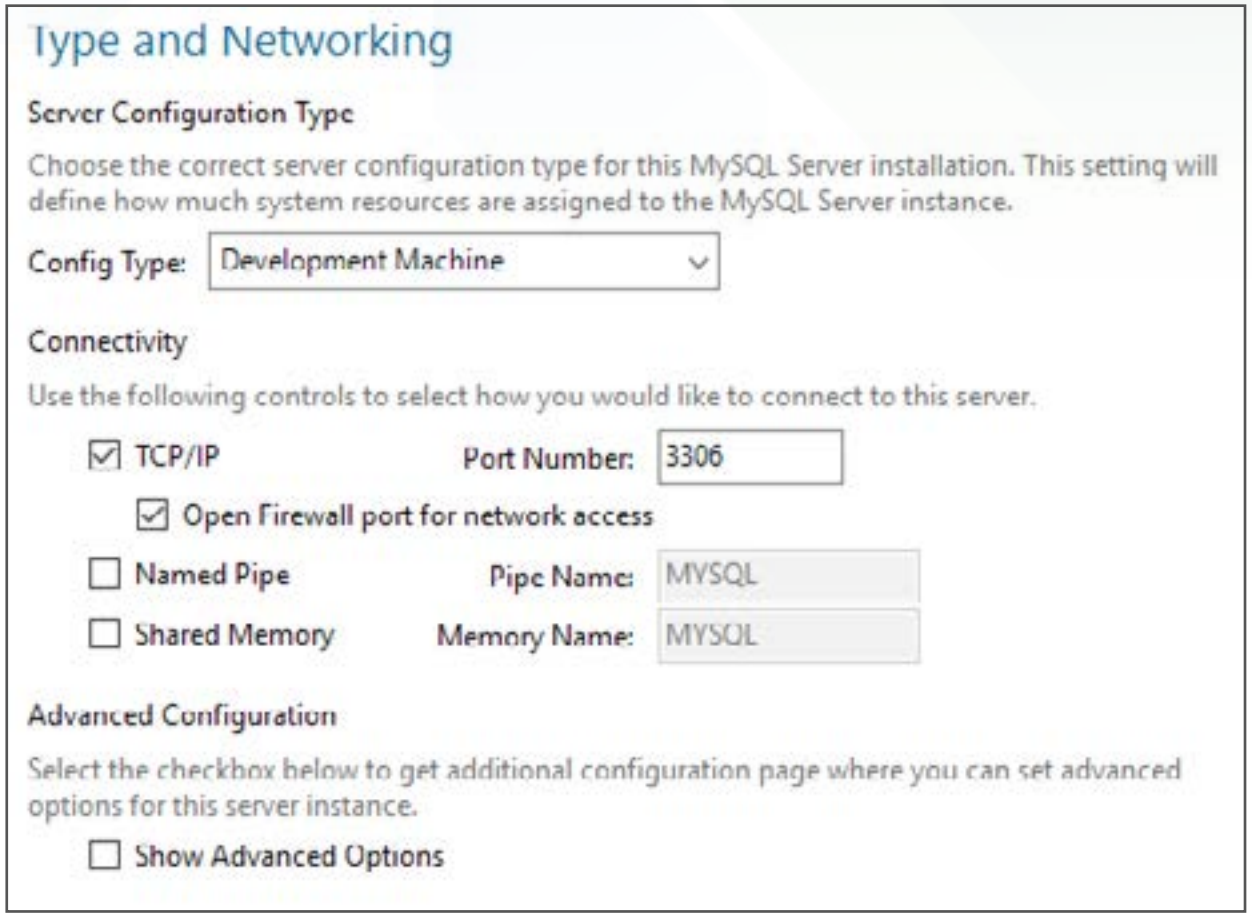

MySQL Installer
MySQL. Installer
Adding Community
License Agreement
Choosing a Setup Type
Installation
Installation Complete
Choosing a Setup Type
Please select the Setup Type that suits your use case.
Developer Default
Installs all products needed for MySQL development purposes.
Server only
Installs only the MySQL Server product.
Client only
Installs only the MySQL Client products, without a server.
Full
Installs all included MySQL products and features.
Custom
Manually select the products that should be installed on the system.
Setup Type Description
Installs the MySQL Server and the tools required for MySQL application development. This is useful if you intend to develop applications for an existing server.
This Setup Type includes:
* MySQL Server
* MySQL Workbench
The GUI application to develop for and manage the server.
* MySQL for Excel
Excel plug-in to easily access and manipulate MySQL data.
* MySQL for Visual Studio
To work with the MySQL Server from VS.
* MySQL Connectors
Connector/Net, Java, C/C++, ODBC and others.
< Back
Next >
Cancel

A nova versão do instalador permite a instalação do conector para linguagem de programação Phyton. Como não é o foco da disciplina, deixamos em branco e clicamos em *Next*.

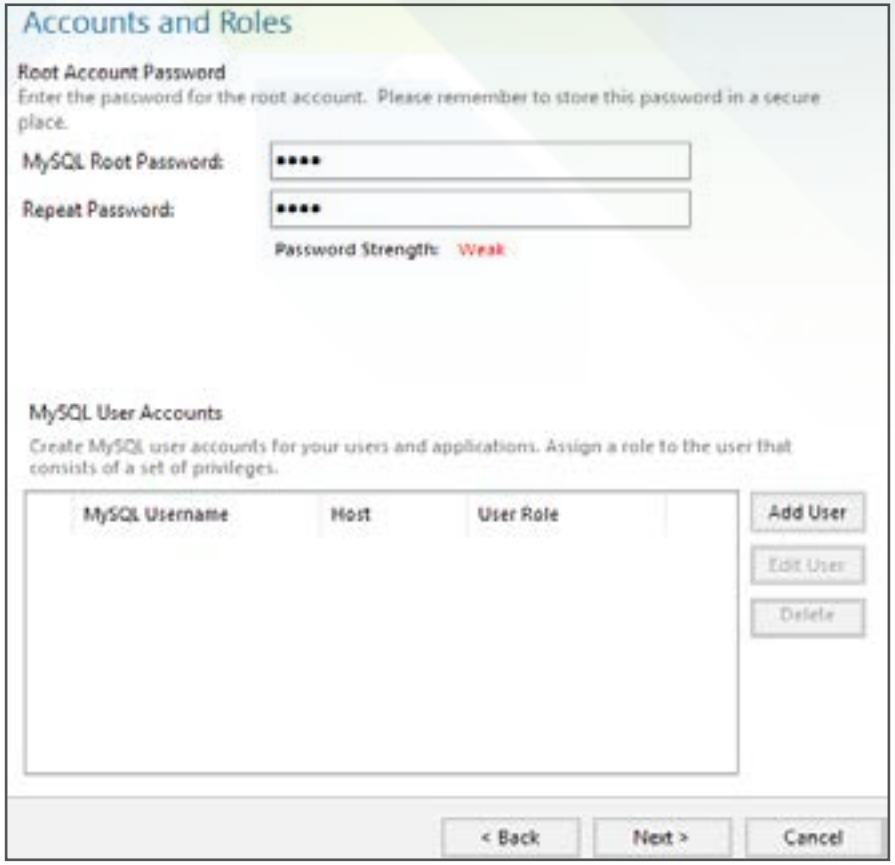
Agora, devemos atualizar os componentes do MySQL, clicando no botão *Execute*. Assim, todos os componentes listados serão instalados e atualizados para as versões mais recentes disponíveis no site do MySQL. Ao lado, a tela mostra um componente instalado e outro que está instalando.



Após a instalação de todos os componentes do MySQL, o próximo passo é a configuração do Servidor MySQL.

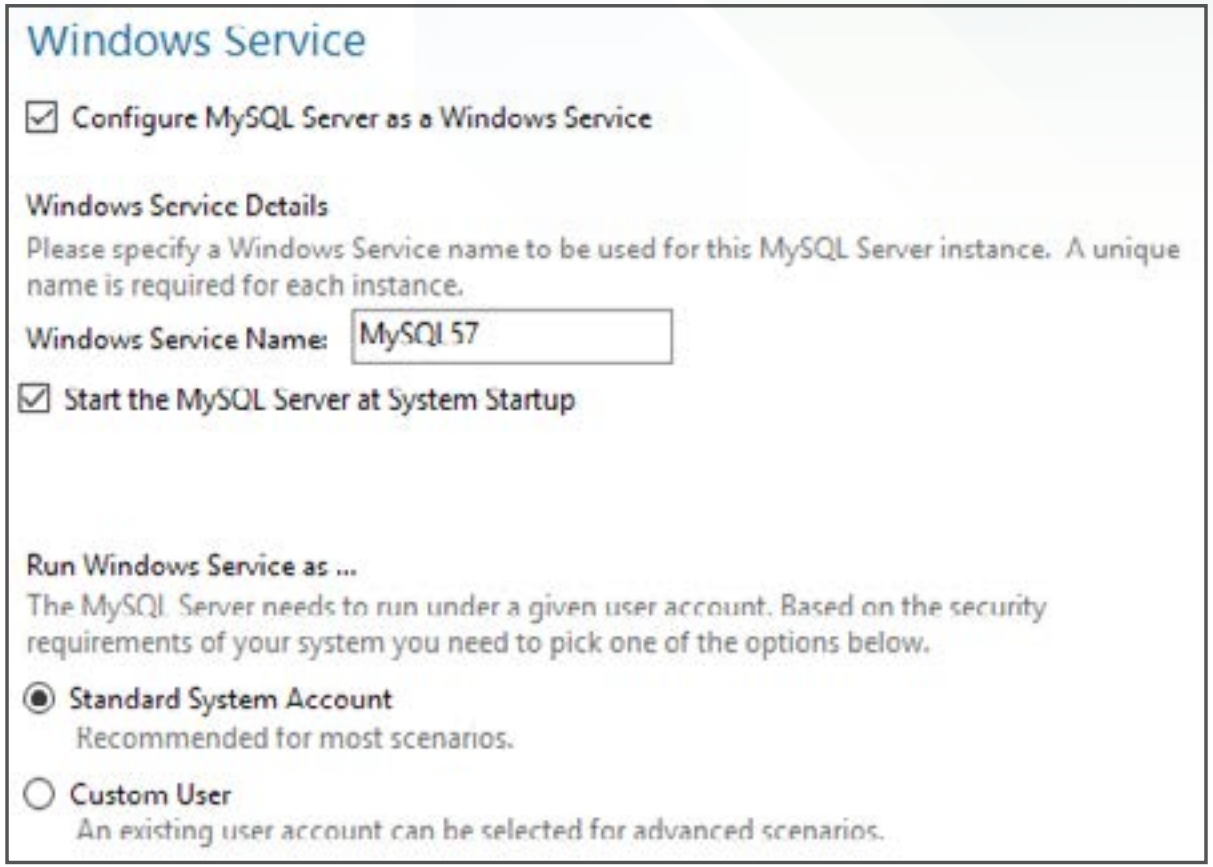


Nessa tela devemos deixar o *Config Type* como *Development Machine* (Máquina de Desenvolvimento), a porta deverá ser a 3306. Na próxima tela, devemos criar uma senha para o usuário *root*, a qual permite que façamos qualquer tipo de operação no MySQL e, se houver a necessidade, criamos outro usuário para uso no banco de dados. Em nosso caso, como estamos em ambiente de testes, deixamos somente o usuário Admin que já foi criado.



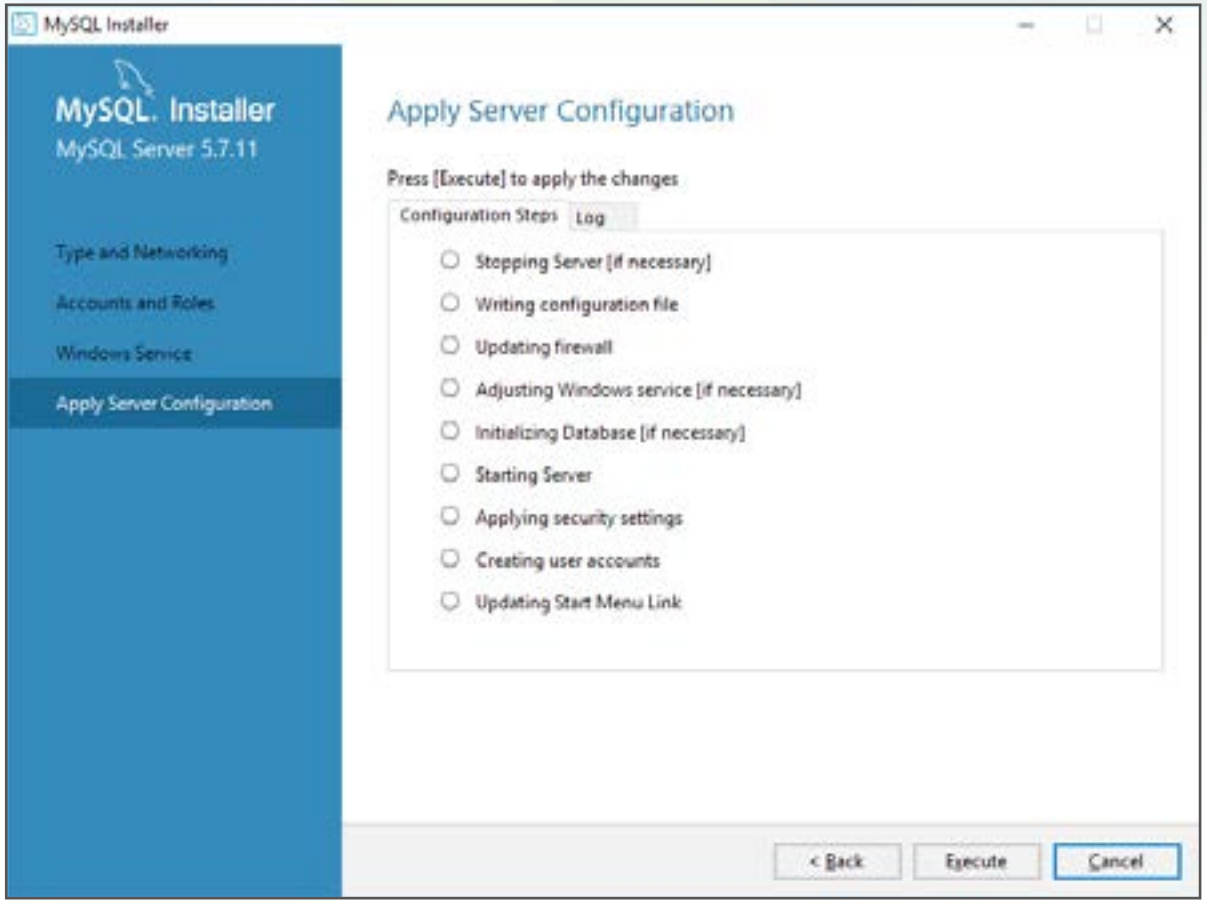
Como estamos utilizando um sistema operacional Windows, a próxima tela permite a configuração do MySQL como um serviço do Windows, ou seja, toda vez que o Windows

for inicializado, o servidor MySQL também será inicializado. Para isso, temos as seguintes configurações:

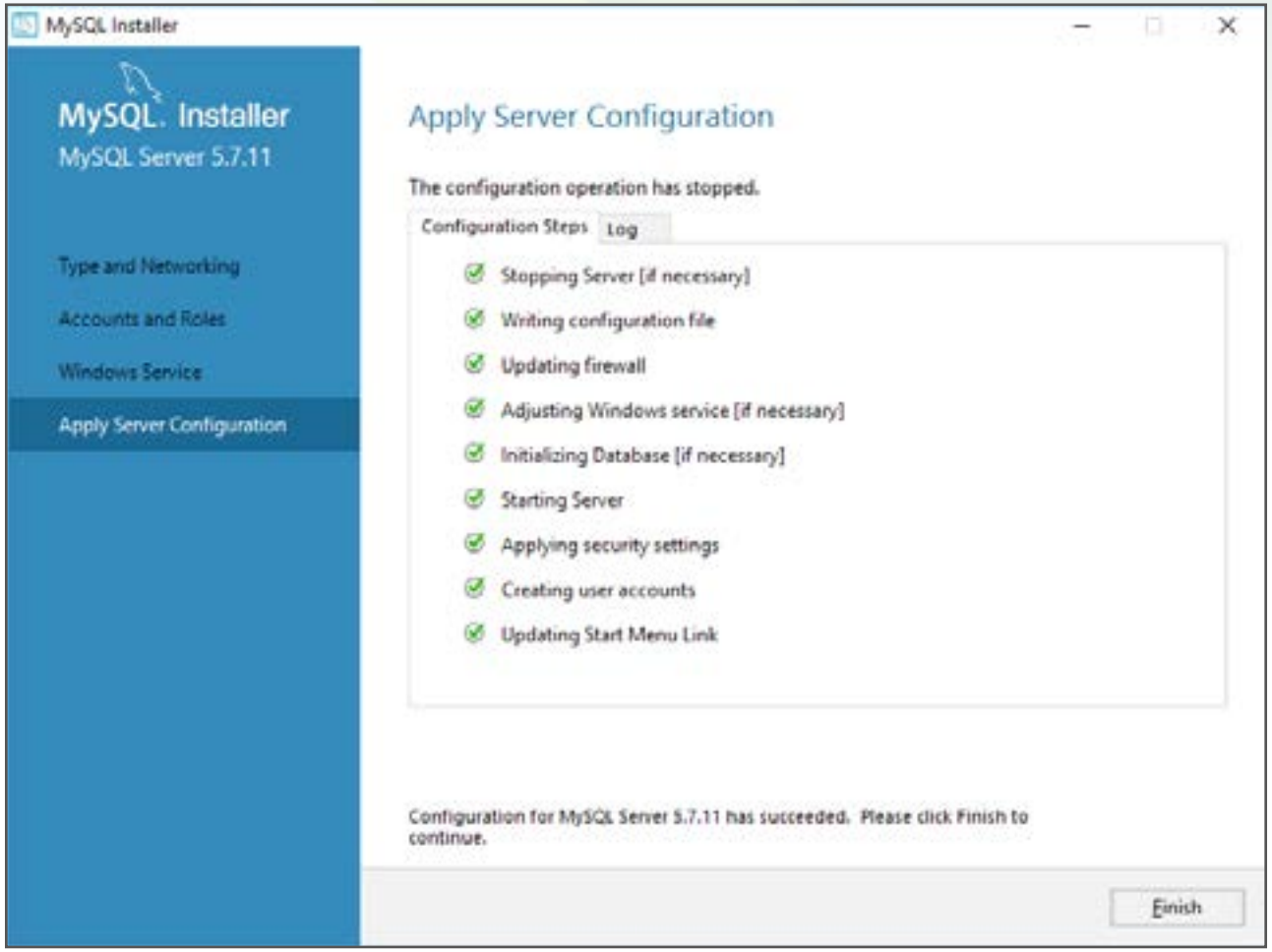


Primeiramente, deixamos que o MySQL seja um serviço do Windows e que ele seja executado quando o sistema operacional for inicializado, ficando atrelado à conta de usuário padrão.

A próxima tela permite a aplicação das configurações executadas no MySQL. Para tanto, clique em *Execute*.

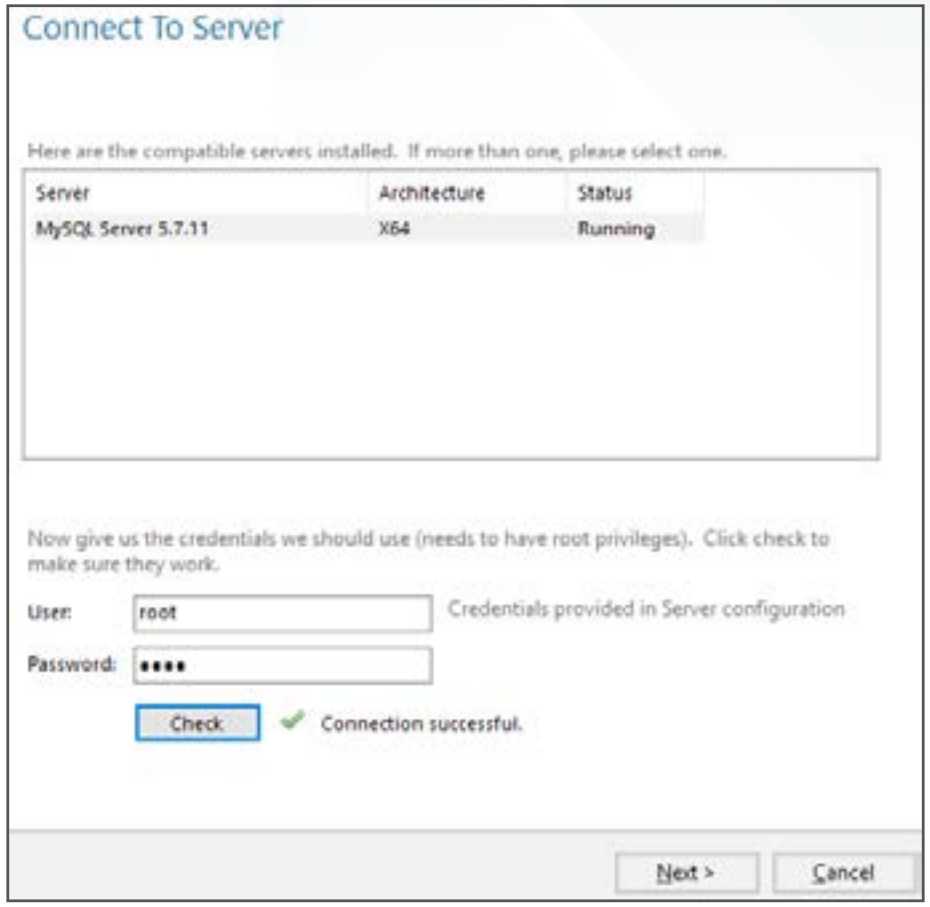


Após a aplicação das configurações, veremos a tela de conclusão da instalação a seguir, na qual falta apenas finalizar a operação clicando em *Finish*:

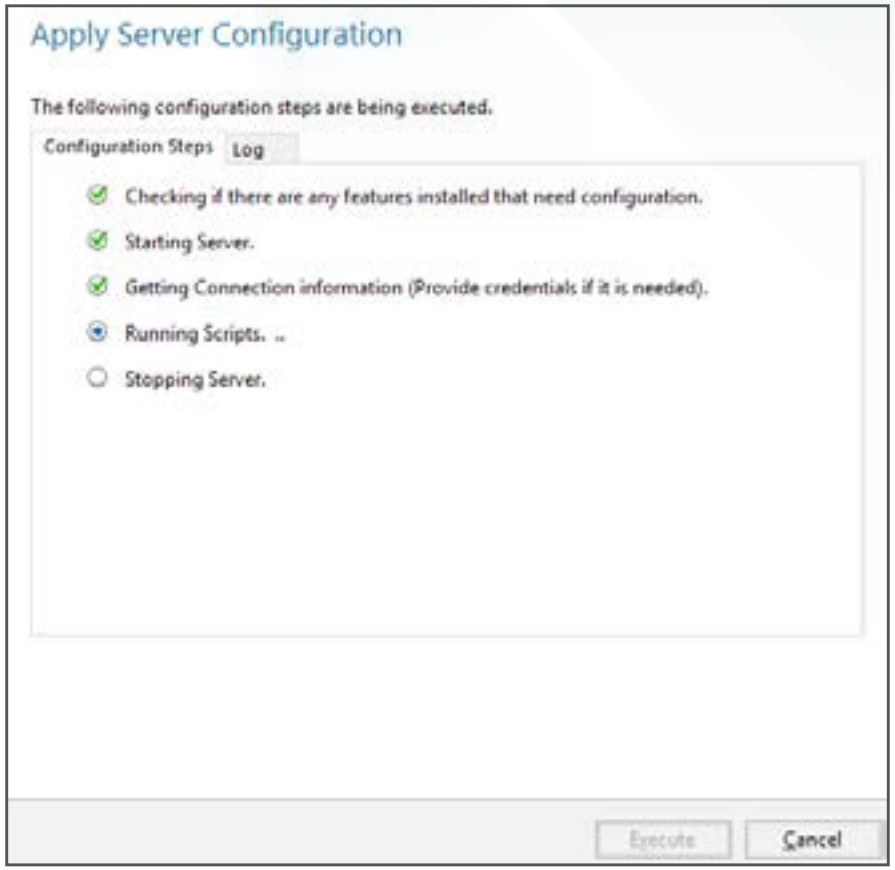


Após aplicar as configurações, devemos testar a conexão com o banco de dados. Para isso, a

próxima tela mostra que o servidor MySQL está funcionando e, em seguida, clicamos no botão *Check*, o qual terá que trazer a resposta *Connection Successful*. Pronto, é só clicar em *Next*.



A tela seguinte necessita de que apliquemos mais algumas configurações relacionadas para saber se todos os recursos estão disponíveis.



Dessa forma, finalizamos a instalação e configuração do MySQL Server ou Servidor MySQL.

2. Apache

O Apache é um dos mais famosos servidores WEB *Open Source* do mundo. Sua página WEB oficial é <http://www.apache.org/> (acesso em: 2 set. 2016), em que, desde 1999, é desenvolvido e atualizado pela empresa Apache Software Foundation. Atualmente, mantém mais de 350 projetos *Open Source*.

No caso do sistema operacional Windows, o site do Apache orienta a instalação de uma aplicação de outras empresas que permitem a instalação de todas as ferramentas de forma automática. Sendo assim, optaremos pela instalação da aplicação XAMPP, mantida também pela Apache Software Foundation e que se encontra no seguinte site: <https://www.apachefriends.org/index.html>. Acesso em: 2 set. 2016.

Para saber mais

Para saber mais sobre o Servidor WEB Apache, acesse: <www.apache.org>. Acesso em: 2 set. 2016)



Clicamos no botão XAMPP do Windows e iniciamos o *download*. Após o *download*, instalaremos o XAMMP, mas, como ele instala o Servidor Apache e o PHP, veremos a instalação dos dois na Seção 3, que trata do PHP, e faremos o teste para verificar o funcionamento das duas ferramentas de desenvolvimento na Seção 4, testando tudo.

3. PHP

Esta citação, retirada do site oficial do PHP, define o que é: "*Fast, flexible and pragmatic, PHP powers everything from your blog to the most popular websites in the world*" (PHP, 2016), ou seja, uma linguagem de programação para desenvolvimento WEB que permite grande flexibilidade e rapidez na programação de sistemas, como blogs, websites, portais, entre outros tantos que só com a prática e muito estudo que teremos uma boa noção do que pode ser feito e desenvolvido em PHP.

Podemos fazer o *download* do PHP direto do *link* a seguir, ou, instalando o XAMPP que também permite a configuração da ferramenta de interpretação de códigos PHP. O *link* oficial é: <http://php.net/downloads.php> (acesso em: 2 set. 2016), nele o desenvolvedor poderá escolher a plataforma de desenvolvimento de sua escolha e baixar o instalador.

Mas, no nosso caso, como dito na seção anterior e já realizado o *download*, instalaremos o XAMPP da *Apache Friends*, que também permite a instalação do PHP.

Após o *download* do instalador, dê um duplo clique no ícone para iniciar o processo:

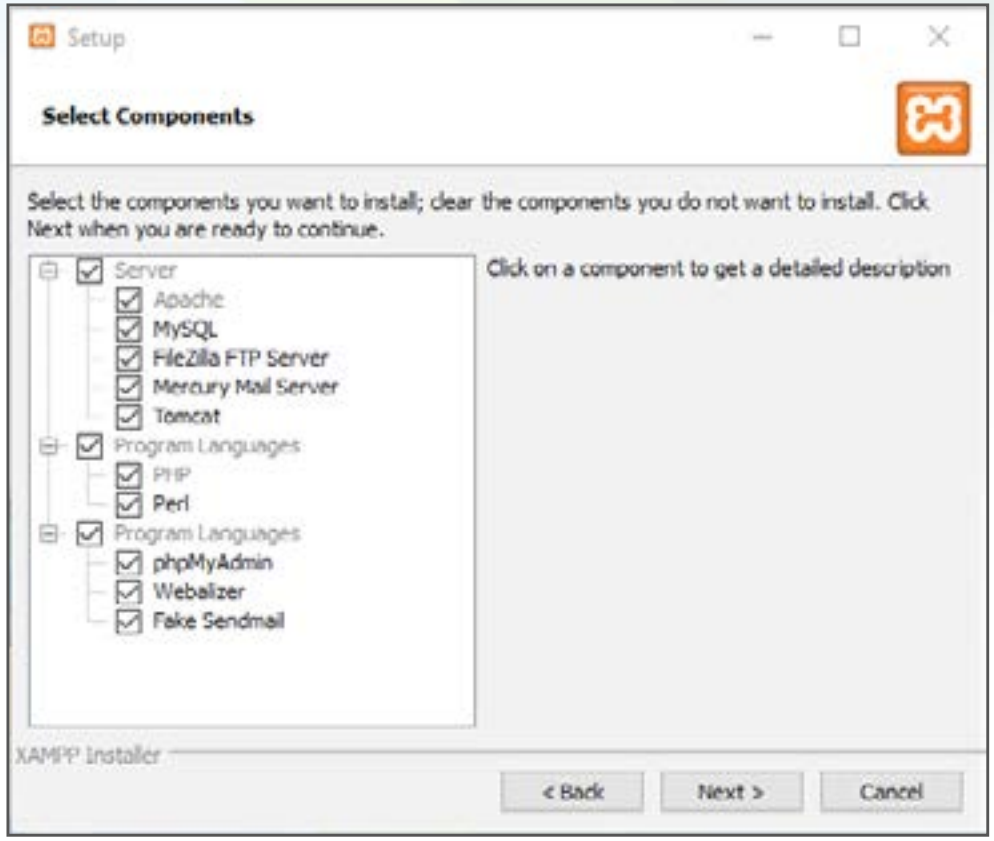


A tela de *Setup* aparece com as boas-vindas à instalação. Vendo isso, clicamos

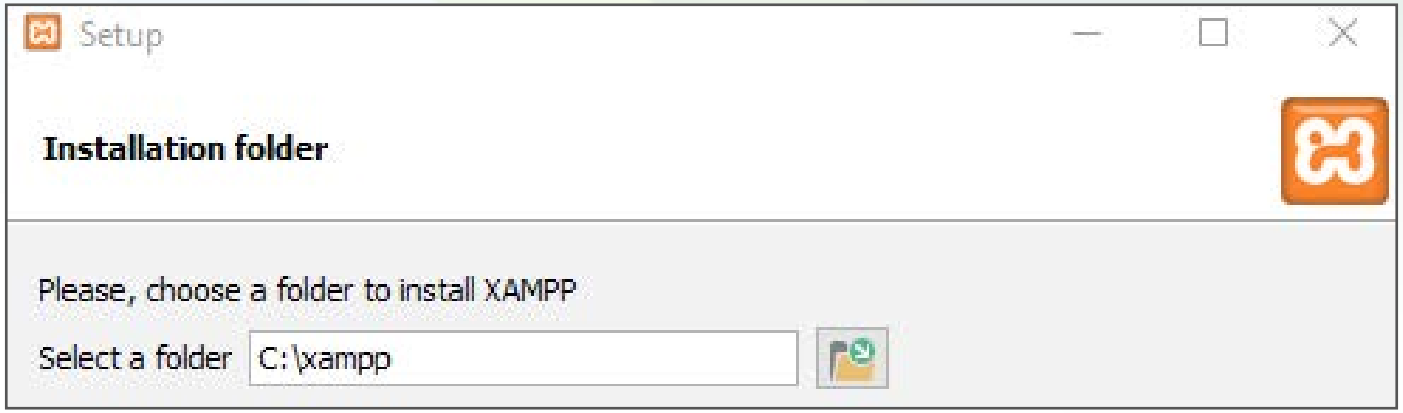
no botão *Next*.



Depois, devemos selecionar os componentes que necessitamos para o desenvolvimento. Em *Server*, deixamos todos os componentes selecionados. Em *Program Languages*, deixamos o PHP selecionado e, no segundo *Program Languages*, selecionamos todos os componentes:

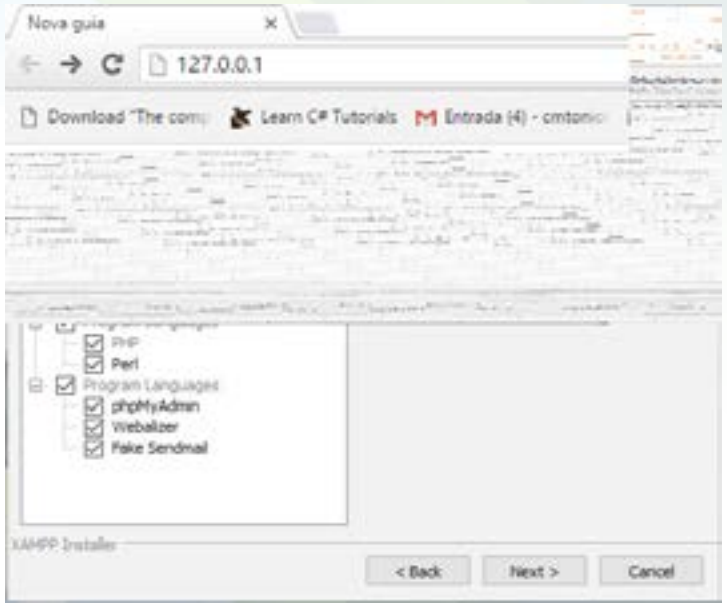


Clique no botão *Next* para o próximo passo. Selecione a pasta padrão, normalmente deixamos a configuração prévia. Clique em *Next*.



Na tela seguinte, é solicitada a visualização de outras aplicações desenvolvidas para uso com o XAMPP, assim, desmarque a caixa de checagem e clique em *Next*.

Próxima etapa, começar a instalação, para isso, clique no botão *Next* e a tela a seguir aparecerá:



Após finalizada a instalação, o ambiente estará pronto para ser testado.

Na próxima seção, ao testarmos tudo, veremos como fazer o teste para verificar o funcionamento das ferramentas instaladas.

4. Testando tudo

Até o momento, configuramos e instalamos todas as ferramentas necessárias para a criação do

nosso ambiente de desenvolvimento PHP, o qual será usado durante esta disciplina. Agora, é o momento de testar todas as ferramentas para saber se tudo foi configurado corretamente e se está em pleno funcionamento.

Para isso, usaremos alguns conceitos necessários ao entendimento. Primeiramente, devemos abrir o Painel de Controle do XAMPP e inicializar algumas ferramentas. Para facilitar o processo, sugiro criar um ícone de atalho na área de trabalho para que, ao iniciar novamente o Painel de Controle, faça-o de maneira rápida.

No Windows Explorer, acesse Disco Local, depois, a pasta XAMPP, encontrando o ícone xammp-control, conforme a seguir:

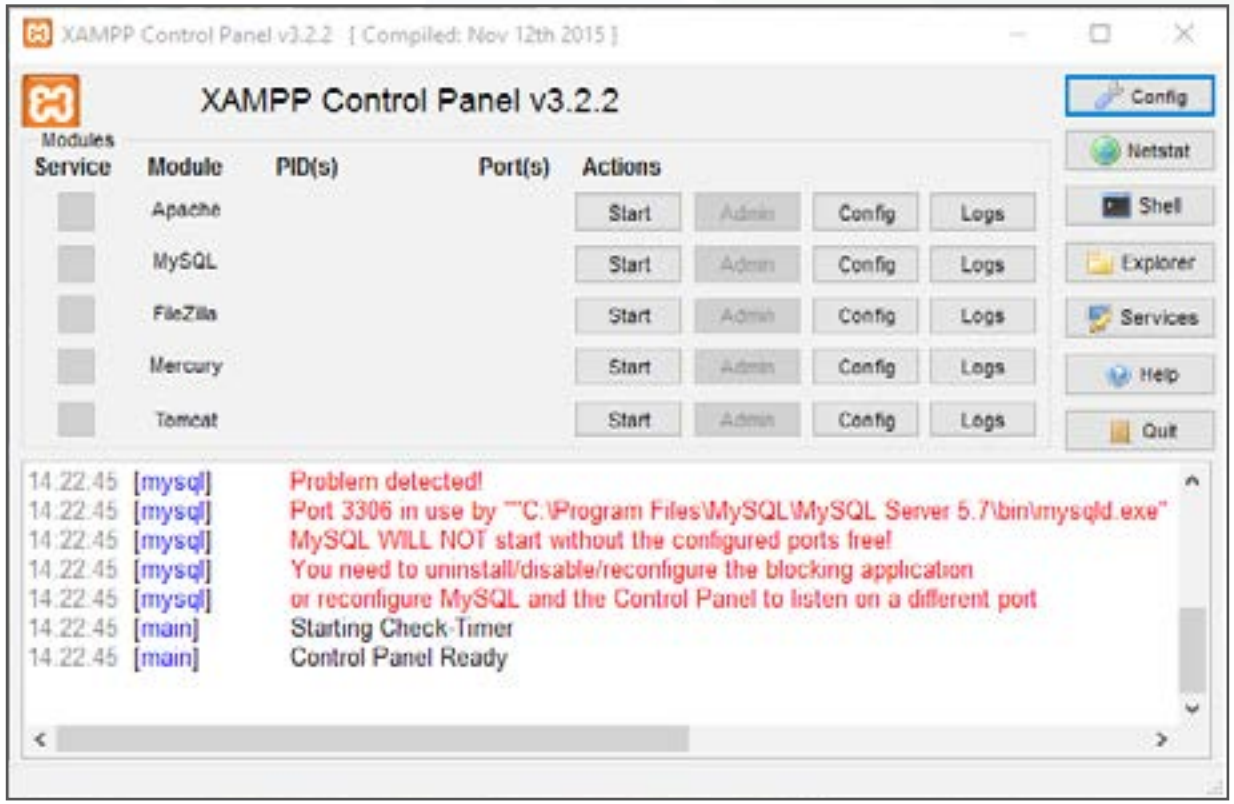


Clique com o botão direito do mouse, escolha a opção Enviar e, em seguida, a opção Área de Trabalho (criar atalho).

Criaremos o seguinte atalho na Área de Trabalho:



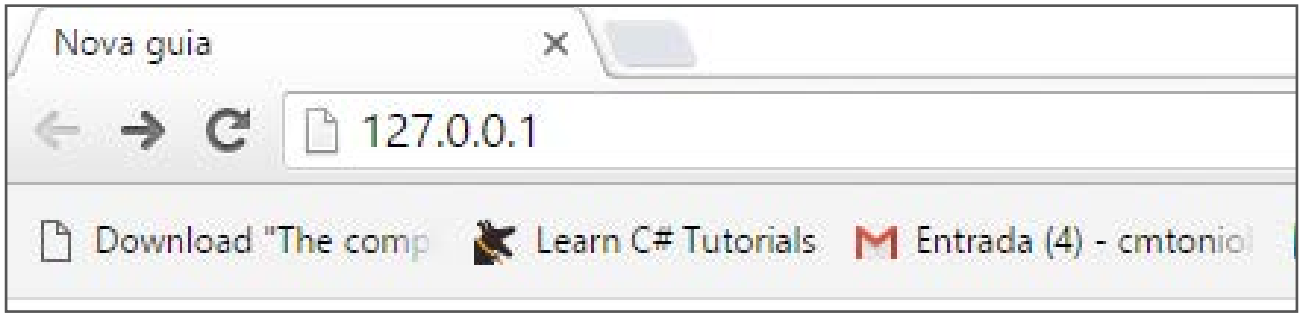
Dê um duplo clique neste ícone para abrir o Painel de Controle do XAMPP. Ao abrir a tela, clique no botão *Start* para o Apache e para o MySQL.



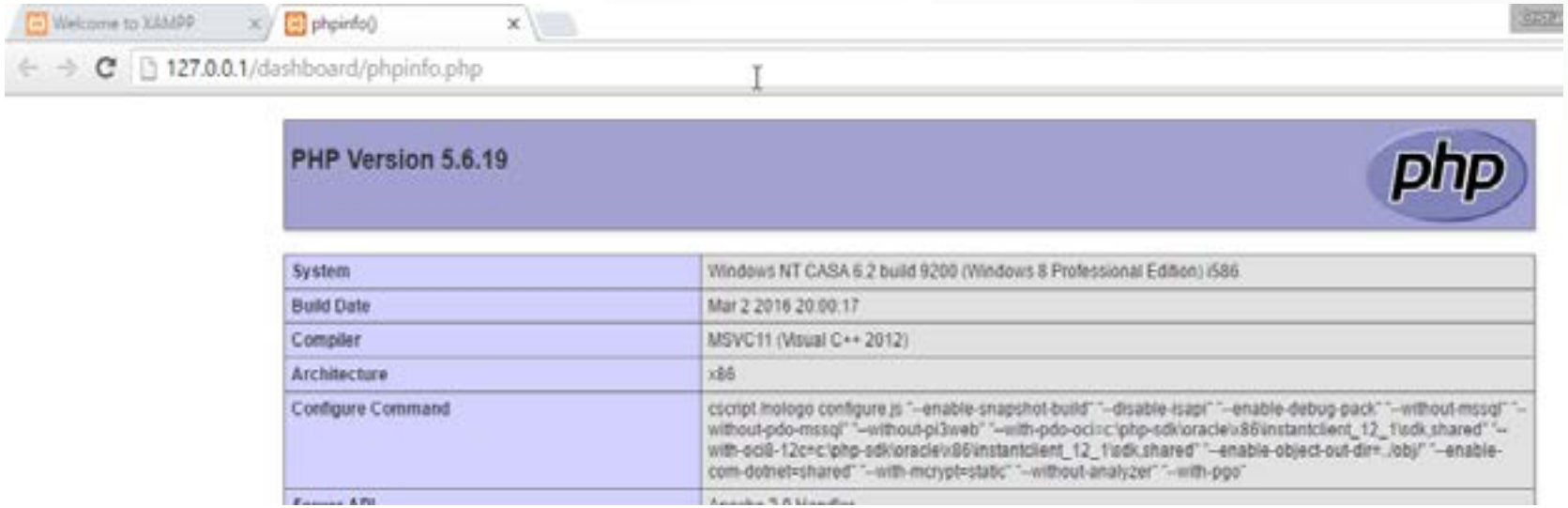
Após isso, deverá ter ficado dessa forma, informando que o Apache e o MySQL estão rodando:



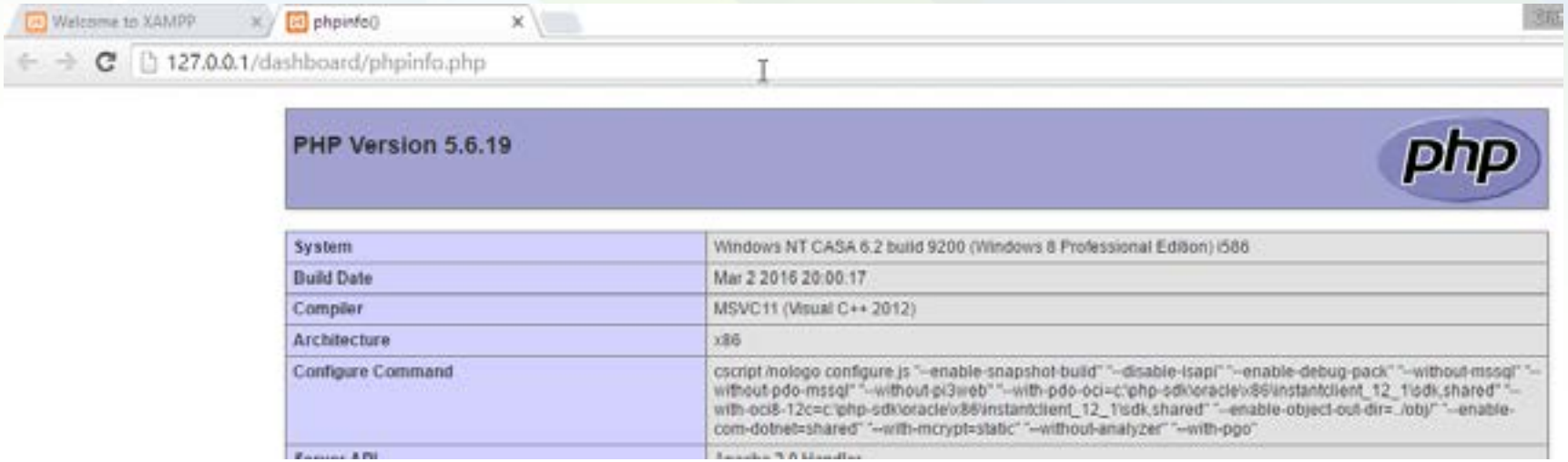
O próximo passo é o teste, para isso, usando qualquer navegador WEB, normalmente o Google Chrome ou o Mozila Firefox (funciona também no Explorer), digitaremos nosso endereço de *loopback* para fazer o teste. Este endereço é o 127.0.0.1 ou localhost. Ele fará que o navegador envie uma requisição para a internet, no formato de um pacote HTTP. Mas, como o servidor está instalado localmente, será como se estivéssemos acessando uma página na internet, mas localmente. Acompanhe nas telas a seguir:



Depois de digitar o endereço de *loopback*, tecle *Enter*. Se a tela a seguir aparecer, tudo estará correto. Preste atenção no endereço que aparece na barra 127.0.0.1/dashboard, esta é a tela de configuração do Apache retornada pelo servidor WEB.



Agora, no menu anterior, em que aparece PHPinfo e phpMyAdmin, faremos os testes de ambiente, verificando se o MySQL está com todas suas tabelas criadas. Ao clicar em PHPInfo, aparecerá a seguinte tela:

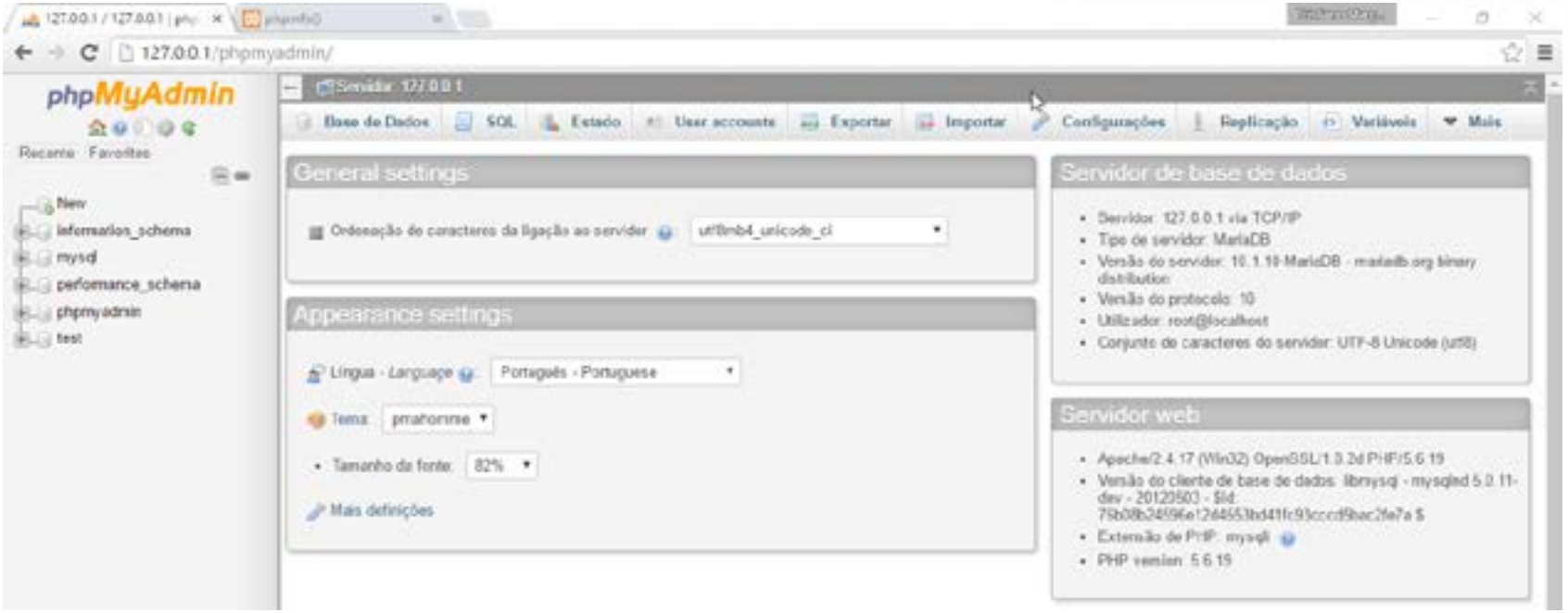


Para saber mais

Para saber mais sobre as variáveis de ambiente do PHP, acesse: <www.php.net> (acesso em: 2 set. 2016), em que você pode fazer uma busca sobre a variável que quiser.

Esta mostra as configurações das variáveis de ambiente do PHP que podem ser acessadas na página oficial, caso o aluno queira se aprofundar mais. Verifique o Saiba Mais desta página.

Clicando em phpMyAdmin, veremos a tela de administração do banco de dados MySQL, com as ferramentas para criação de bancos e tabelas, edição e exclusão de tabelas e muitos outros recursos que veremos no decorrer desta disciplina e na de Banco de Dados.



O mais interessante é que nos outros *links* do menu podemos ter uma visão geral de Guias

para aprendizagem do PHP e outras ferramentas que podem ser baixadas para que possamos usar com o XAMPP e o PHP, já prontas para cada tipo de aplicação específica que o usuário necessitar.

Assim, conseguimos criar nosso ambiente de desenvolvimento para nossos estudos. No próximo tema, veremos como usar o PHP partindo do básico e revisando os conceitos de programação.

Glossário

Open Source: licença de *software* que está dentro dos padrões de definição de *Open Source*, que significa, em termos gerais, que os *softwares* podem ser usados, modificados e compartilhados de modo gratuito.

Servidor WEB: máquina ou programa de computador responsável por receber e dar respostas HTTP para a comunicação cliente/servidor.

Loopback: endereço HTTP que simula o envio de um pacote para a rede, mas que é usado para teste de redes e para uso com servidores WEB, antes de enviarmos a página WEB para o servidor de produção.

Questão para reflexão



Para entender mais sobre o ambiente de cliente/servidor, pesquise na internet, em livros e na biblioteca da sua unidade de ensino como funciona esse conceito, quais são os requisitos que necessitamos para que uma aplicação funcione bem nesse ambiente. Pesquise também outras linguagens que o utilizam e se elas necessitam de alguma configuração diferente do que foi mostrado até o momento.

Considerações Finais

Nesta aula vimos os principais conceitos e ferramentas para que se possam desenvolver aplicações em PHP, que foram:

- Instalação e configuração do Servidor de Banco de Dados MySQL.
- Instalação e configuração do Servidor WEB Apache.
- Instalação e configuração da linguagem de programação PHP.
- Teste do ambiente usando navegador WEB com o endereço de *loopback* para o ambiente de desenvolvimento.

Referências

Open Source. Open Source Initiative. Disponível em: <https://opensource.org/licenses>. Acesso em: 2 set. 2016.

APACHE. The Apache Software Foundation. Disponível em <http://www.apache.org>. Acesso em: 2 set. 2016.

PHP. PHP 7.1.0 Release Candidate 1 Released. Disponível em: <http://php.net>. Acesso em: 2 set. 2016.

MYSQL. Download MySQL Community Server. Disponível em: <http://dev.mysql.com/downloads/mysql/>. Acesso em: 2 set. 2016.

Assista a suas aulas



Aula 1 - Tema: Instalando e Configurando as Ferramentas - Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/62d9e8bd93406e6e918f06d7b70ad299>.



Aula 1 - Tema: Instalando e Configurando as Ferramentas - Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/0a86209f10c37b01fa60f0a8c93a05b3>.

Questão 1

1. As ferramentas MYSQL, Apache e PHP permitem seu uso gratuito:

a) Não, porque devem ter suas licenças pagas.

b) Somente o Apache e o MySQL podem ser usados gratuitamente.

c) Somente o PHP pode ser usado gratuitamente.

d) Todas podem ser usadas gratuitamente, porque respeitam as regras de *Open Source.*

e) Todas podem ser usadas gratuitamente, mas não respeitam as regras de *Open Source.*

Questão 2

2. Sobre Servidor WEB é CORRETO afirmar:

a) Servidor WEB não permite trabalhar com requisições HTTP.

b) Servidor que não necessita estar na internet para funcionar.

c) Servidor WEB é a aplicação que aceita requisições HTTP de clientes e retorna respostas HTTP com a inclusão ou não de dados, páginas HTML ou imagens.

d) Servidor WEB é a aplicação que rejeita as requisições HTTP de clientes e retorna respostas HTTP com a inclusão ou não de dados, páginas HTML ou imagens.

e) Servidor WEB é a aplicação que aceita requisições HTTP de clientes e não retorna respostas HTTP com a inclusão ou não de dados, páginas HTML ou imagens.

Questão 3

3. O que é um Servidor de Banco de Dados?

a) É um programa responsável por gerenciar e criar banco de dados usados em aplicações para computador.

b) É um programa que permite a instalação de outros bancos de dados.

c) É uma máquina que permite a criação de banco de dados de outras aplicações.

d) É uma máquina com um programa de banco de dados.

e) É um processo de criação de banco de dados para aplicações.

Questão 4

4. Qual é a vantagem de se utilizar um aplicativo como o XAMPP?

a) Não conseguir configurar as ferramentas para desenvolvimento WEB.

b) Fazer que o desenvolvimento WEB se torne complicado.

c) Facilitar a instalação dos componentes para desenvolvimento WEB.

d) Ter que configurar tudo de forma manual.

e) Facilitar a instalação das ferramentas para desenvolvimento WEB.

Questão 5

5. Sobre um ambiente cliente/servidor, é CORRETO afirmar:

a) Requisições são processadas diretamente no navegador WEB.

b) As requisições feitas pelo navegador são enviadas para um Servidor WEB e processadas dentro dele, retornando respostas para o navegador.

c) Cada requisição gera uma resposta do navegador WEB, sendo assim, o servidor WEB somente gerencia isso.

d) Não tem a ver com comunicação entre servidor WEB e navegador de internet.

e) As requisições feitas pelo servidor WEB são enviadas para outro e processadas dentro deste servidor, que retorna respostas para o navegador.

Gabarito

1. Resposta: D.

Sim, todas as ferramentas são livres de licença e podem ser usadas gratuitamente por respeitarem as regras de *Open Source*, sendo usadas, alteradas e compartilhadas, sem a necessidade de se pagar por uma licença de uso como os *softwares* proprietários.

2. Resposta: C.

Servidor WEB tem como funcionalidade receber as requisições HTTP que, geralmente, são endereços de internet e deve retornar respostas HTTP, contendo imagens, páginas HTML, páginas com dados pesquisados em banco de dados, entre outros.

3. Resposta: A.

Servidor de banco de dados é uma aplicação que permite a criação de bancos para diversos tipos de aplicação, possibilitando seu gerenciamento e administração. Lembrando que existem diversas ferramentas para fazer essa administração.

4. Resposta: E.

Ferramentas como o XAMPP permitem que instale e configure as ferramentas para o desenvolvimento WEB de forma automática, ou seja, sem que haja a necessidade de instalar cada uma delas e configurá-las individualmente.

5. Resposta: B.

Cliente/servidor permite a comunicação de um cliente WEB (normalmente um navegador) com o servidor WEB no formato HTTP e retorna repostas depois de processadas dentro do servidor, podendo ou não acessar dados armazenados em banco de dados.

Unidade 2

Conceitos Básicos de PHP

Objetivos

1. O objetivo desta seção é apresentar os comandos e os conceitos básicos da linguagem PHP. Para isso, serão utilizados vários exemplos de códigos mostrando o funcionamento de cada um deles.

Introdução

Em 1994, a linguagem PHP foi desenvolvida por Rasmus Lerdorf, que originalmente significa *Personal Home Page Tools*. O objetivo de Lerdorf era saber quais usuários visualizavam seu currículo *on-line*. Mas as funcionalidades da linguagem cresceram e, após certo tempo, passou a se chamar *Hypertext Preprocessor*, como resultado de uma votação da comunidade PHP que cresce a cada ano.

Sua versão é a 5.6.20, que utiliza a programação em *scripts* e pode ou não ser embutida no HTML. Nessa versão, é possível programar praticamente qualquer tipo de problema em PHP, como coleta de dados de um formulário, geração de páginas dinâmicas ou ainda enviar e receber *cookies*.

1. Sintaxe básica

A sintaxe básica de um código PHP, que como dito anteriormente pode estar ou não embutida dentro de um código HTML, é a seguinte:

```
<?php

    Comandos

?>

<script language="php">

    Comandos

</script>

<?

    Comandos

?>
```

```
<%
    Comandos
%>
```

As *tags* mais utilizadas na programação PHP são o terceiro conjunto, sendo uma abreviação do primeiro. Dependendo do tipo de ferramentas que forem usadas no desenvolvimento, pode ser que seja necessário usar o primeiro conjunto de *tags*.

O último conjunto está relacionado aos programadores que usavam o ASP e não conseguiam se acostumar com o PHP.

2. Separador de instruções e comentários

Utilizado para que o interpretador identifique quando uma instrução termina. O separador de instruções é o ponto e vírgula (;) e é aplicado da seguinte forma:

```
<?
    echo 'Texto de instrução';
?>
```

Note que o texto que está entre aspas simples será impresso na tela do navegador e o comando que informa isso ao interpretador é o *echo*. Além disso, o ponto e vírgula no final da instrução indica que o código foi finalizado.

Quanto aos comentários, eles são usados para o desenvolvedor conseguir identificar o que é comando PHP e o que são comentários incluídos por ele, a fim de lembrar de algum detalhe que será usado no programa. Podem ser de três tipos: comentário de uma linha, de várias linhas e no estilo shell do linux.

Exemplo:

```
<?
    echo "Curso de PHP";          //
comentário de uma linha
    echo "Pós-graduação Kroton";
    /*Este comentário é feito em
    mais de uma linha e finaliza
dessa forma */
    echo "Mais uma Pós no
Curriculo";      #comentário estilo
shell
?>
```

3. Tipos de dados

Quando criamos uma variável em um programa de computador, devemos indicar o tipo de dados a que ela pertence, ou seja, que valor que essa variável pode receber para que ela possa ser utilizada na lógica de nosso programa. O PHP trabalha com alguns tipos de dados, são eles: Inteiro, ponto flutuante, string, array e objeto.

O PHP se utilizará da checagem de tipos dinâmica, ou seja, o interpretador PHP decidirá qual tipo de dados usar,

dependendo do valor que será atribuído para uma variável. Portanto, pode ser que uma variável tenha mais de um tipo de dados em um mesmo programa.

3.1. Tipo inteiro

Exemplos de atribuições para uma variável do tipo inteiro:

```
$php = 1916;     //inteiro positivo

$php = -120;     //inteiro negativo
```

Podemos questionar: qual é a diferença entre um inteiro simples e um *long*? A resposta é o tamanho da variável, mas como em PHP a conversão é feita pelo interpretador e de forma transparente ao usuário, podemos dizer que os tipos de dados são iguais.

3.2. Número em ponto flutuante (*double* ou *float*)

Uma variável com o tipo de dados em ponto flutuante é utilizada quando desejamos armazenar valores decimais, podendo ser por meio das seguintes sintaxes:

```
$var1 = 1.916;

$var2 = 23e4;    //equivale a
230.000

$a = 1.234;
$b = 1.2e3;
$c = 7E-10;
```

Dessa forma, podem-se definir variáveis do tipo decimal em PHP.

3.3. *Strings*

Quando desejamos trabalhar com textos, devemos criar as variáveis do tipo *string*. Essas variáveis permitem o armazenamento das chamadas cadeias de caracteres, que podem ser textos explicando um conceito ou que fazem uso de variáveis para compor uma resposta a um determinado problema. As *strings* são atribuídas às variáveis de duas maneiras:

1. Com o uso de aspas simples (‘) – o valor da variável será exatamente o texto que está entre as aspas.
2. Com o uso de aspas duplas (“) – o valor da variável ou caractere de *escape* será expandido antes de ser atribuído, ou seja, não será o valor literalmente que está dentro das aspas.

Exemplos:

```
<?
    $pais = “Brasil”;
    $saida = ‘-------$pais-------
\n’;
    echo “$saida”;
?>
```

- A saída do código anterior será:

```
“-------$pais-------\n”
```

```
<?
    $pais = “Brasil”;
```

```
    $saida = "-------$pais-------\n";
    echo "$saida";
?>
```

- A saída do código anterior será: "-------Brasil-------"

Nota-se que, no segundo código, o valor da variável $pais foi expandido e o valor que estava atribuído dentro dessa variável foi impresso, ou seja, Brasil. O \n é um caractere de escape que é utilizado quando desejamos que o interpretador execute alguma funcionalidade no final da impressão, no caso, pular uma linha. A tabela a seguir mostra os caracteres de escape mais utilizados em PHP:

Sintaxe	Significado
\n	Nova linha
\r	Retorno de carro (semelhante a \n)
\t	Tabulação horizontal
\\	A própria barra (\)
\$	O símbolo de $

Sintaxe	Significado
\’	Aspas simples
\”	Aspas duplas

3.4. Arrays (Vetores)

Arrays são utilizados quando há necessidade de trabalhar com mais de um valor dentro de uma variável. Eles são compostos por um dicionário de índices, que são as chaves de acesso. Como o PHP faz a checagem dinâmica, pode-se ter diferentes tipos como índices de um Array, assim, os valores mapeados também podem ser diversos.

Exemplo: criaremos um Array de cores para mostrar seu uso.

```
<?
    $cor[1] = “vermelho”;
    $cor[2] = “púrpura”;
    $cor[3] = “preto”;
    $cor[4] = “laranja”;
?>
```

Note que o índice do array está dentro dos colchetes e que foi criada uma variável da mesma forma que uma normal. Atribuíram-se os valores (*strings*) com os nomes das cores.

3.5. Listas

Usamos as listas quando há a necessidade de fazer atribuições múltiplas, ou seja, quando necessitamos atribuir valores que estão em um Array para variáveis.

Exemplo:

```
list($a, $b, $c) = array("m", "n", "o");
```

Esse comando faz a atribuição de três variáveis ao mesmo tempo, isto é, as atribuições ocorreram por causa de que o array de elementos está inicializado com índices inteiros e configurados desta forma: $a tem o índice 0, $b tem o índice 1 e $c tem o índice 2.

Exemplo 2:

```
$arr = array(1 => "um", 2 => "dois", 3 => "três", 4 => "quatro");

list($e, $f, $g, $g) = $arr;
```

Neste exemplo, inicializamos um array com quatro índices, sendo cada um com um valor *string* relacionado a números. Em seguida, foram criadas quatro variáveis dentro de um *list* que recebe como valores os índices e valores do array inicial.

3.6. Objetos

Objetos são usados em PHP desde a versão 4, na qual foram incluídos os conceitos de orientação a objetos. Um objeto em PHP usa os mesmos preceitos de orientação a objetos para ser criado, ou seja, o operador *new*. O exemplo a seguir mostra como criar um objeto em PHP e como usá-lo.

Exemplo:

```
class criaObjeto {
    function imprime() {
        echo "mensagem";
    }
}
$var = new criarObjeto();
$var -> imprime();
```

Na primeira parte do programa, criamos uma classe chamada **criaObjeto**, em seguida, dentro dessa classe, adiciona-se uma função chamada **imprime()**, que tem como objetivo imprimir uma mensagem na tela do navegador. Depois, criamos uma variável chamada **$var** que recebe como atribuição o operador *new* com o nome da classe. Isso é o que chamamos de instanciar um objeto, ou seja, criarmos um objeto chamado **$var**. Na última linha, este está usando a função **imprime()** por meio do operador **->**.

3.7. Booleanos

Os tipos booleanos são usados para quando necessitamos fazer comparações

e operações lógicas com expressões e que retornam como resultado os valores *true* (verdadeiro) ou *false* (falso).

3.8. Constantes

A linguagem PHP utiliza-se de dois tipos de constantes que podem ser: pré-definidas e constantes definidas.

Pré-definidas são as constantes já definidas no PHP, que podem ser usadas quando é necessário saber a versão do PHP, os dados do servidor, entre outras informações. Uma forma de sabermos quais são todas as constantes pré-definidas em PHP é usando a função phpinfo(), que tem como resposta uma tabela com todas as constantes do PHP, utilizadas ou não no ambiente de desenvolvimento criado em sua máquina.

Para usá-la, cria-se o seguinte código em PHP:

```
<?
    phpinfo();
?>
```

Somente esse trecho de código retornará a tabela com as constantes da linguagem PHP.

Quando necessitamos criar nossas constantes, usamos a função define antes da variável desta forma:

int define(nome_variavel, valores_da_constante);

Exemplo:

```
define("pi", 3.1415);
$circ = 2 * pi * $raio;
```

Nesse trecho de código, definimos a constante pi e seu valor, e na linha seguinte, usamos a variável $circ para receber o resultado da expressão.

4. Operadores

Ainda trabalhando conceitos vamos conhecer os operadores, que têm como função serem usados em operações matemáticas, ou seja, quando existe a necessidade de fazer a adição de dois valores, a comparação entre duas expressões aritméticas, ou ainda para uso em expressões bem mais complexas. A grande maioria das linguagens de programação usa operadores em sua lógica, portanto, pode ser que alguns dos estudantes já tenham familiaridade com os operadores que serão apresentados a partir de agora.

4.1. Aritméticos

Usados para expressões aritméticas:

+	Adição
-	Subtração
*	Multiplicação
/	Divisão
%	Módulo

4.2. De *strings*

É usado quando há a necessidade de se concatenar duas ou mais *strings*. O operador é o ponto (.).

4.3. Atribuição

Usado quando queremos atribuir valores em expressões e em comandos decisão, repetição, entre outros.

=	Atribuição simples
+=	Atribuição com adição
-=	Atribuição com subtração
*=	Atribuição com multiplicação
/=	Atribuição com divisão
%=	Atribuição com módulo
.=	Atribuição com concatenação

Exemplo:

```
$num = 7;

$num += 2; //$num passa valer 9
```

Nos exemplos, a variável do valor 7 é atribuído à variável $num. No segundo exemplo, o operador de atribuição com adição permite que o valor 2 seja atribuído à variável $num, somando-se com o valor anterior que era o número 7.

4.4. Lógico e Comparação

Usados quando há a necessidade de comparar duas expressões e quando necessitamos saber o resultado lógico de duas comparações.

And	“e” lógico
Or	“ou” lógico
Xor	“ou” exclusivo
!	Não (inversão)
&&	“e” lógico
||	“ou” lógico

Quanto aos operadores de comparação:

==	Igual a
!=	Diferente de
<	Menor que
>	Maior que
<=	Menor ou igual a
=>	Maior ou igual a

Glossário

Operadores: em linguagem PHP são usados para definir quais ações podemos executar nos códigos criados na linguagem.

Tipos de dados: são usados para definir que tipo de informação cada variável pode receber em um código criado em PHP. Não tem a necessidade de ser declarado, porque o interpretador PHP faz a verificação automaticamente.

Sintaxe básica: toda linguagem de programação tem uma sintaxe básica para cada comando seu, que nada mais é do que como os comandos devem ser estruturados e os parâmetros podem ser usados.

Questão para reflexão



Como o foco deste tema da disciplina de PHP é o conhecimento dos conceitos básicos da linguagem, é claro que não temos como abordar todos os conceitos que foram acrescentados com a evolução da linguagem PHP, pois isso resultaria num livro. Assim, uma excelente proposta de autoestudo é a leitura dos conceitos básicos disponíveis no site oficial da linguagem no seguinte *link:* <https://secure.php.net/manual/pt_BR/index.php>. Acesso em: 5 set. 2016.

Neste poderá ser encontrado mais conceitos e exemplos de uso dos conceitos básicos em trechos de código do próprio site ou em perguntas enviadas pela comunidade PHP.

Considerações Finais

- Criação e uso de variáveis e constantes.
- Uso de tipos de dados na linguagem PHP.
- Uso de conceitos, como arrays, objetos e outros.
- Uso de operadores, como os aritméticos, as atribuições, os lógicos e as comparações.

Referências

PHPBUILDER. Disponível em: <http://www.phpbuilder.com/>. Acesso em: 12 set. 2016.

SECURE. Manual do PHP. Disponível em: <https://secure.php.net/manual/pt_BR/index.php>. Acesso em: 12 set. 2016.

Assista a suas aulas



Aula 2 - Tema: Conceitos Básicos de Php - Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pA-piv2/embed/dbd3957c747affd3be431606233e0f1d/1f-826cdac0a9b0766405fc05c7ea473a>.



Aula 2 - Tema: Conceitos Básicos de Php - Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pA-piv2/embed/dbd3957c747affd3be431606233e0f1d/3cf-d485469995bda77dc7d15bb4c83ee>.

Questão 1

1. O que é PHP?

a) Linguagem de script que roda e faz o processamento do lado do cliente.

b) Linguagem de programação específica para cliente/servidor.

c) Linguagem de script que roda do lado do servidor e faz o processamento no lado do cliente.

d) Linguagem de script que executa um executável no lado do servidor.

e) Linguagem de script que roda e faz todo o processamento do lado do servidor.

Questão 2

2. Como declaramos uma variável em PHP?

a) Usamos o sinal de $ e temos que dar um espaço entre o nome da variável.

b) Devemos colocar o tipo de dados antes da declaração da variável.

c) Usamos o sinal de $ antes do nome da variável.

d) Colocamos o tipo de dados antes e, em seguida, o sinal de $ mais o nome da variável.

e) Definimos uma variável da forma que desejarmos.

Questão 3

3. Por que não necessitamos colocar o tipo de dados na declaração de uma variável em PHP?

a) Porque o PHP não tem tipos de dados.

b) Porque não há a necessidade de tipo de dados por causa do compilador.

c) Porque o interpretador PHP não resolve o tipo de dados dinamicamente.

d) Porque o interpretador PHP resolve o tipo de dados dinamicamente.

e) Porque devemos saber que a linguagem PHP é diferente das outras linguagens.

Questão 4

4. Quais são os operadores de atribuição corretos?

a) ==, +=, /=, .=.

b) ==, <=, -=, !=.

c) <>, >=, *=, <.

d) &&, ||, -=, ==.

e) -=, +=, %, !=.

Questão 5

5. Para que servem os operadores de comparação?

a) Servem para comparar dois números e letras.

b) Servem para dar nomes às variáveis.

c) Servem para nomear os próprios operadores.

d) Servem para fazer qualquer tipo de operação.

e) Servem para retornar valores lógicos na comparação entre duas expressões.

Gabarito

1. Resposta: E.

Rodar do lado do servidor significa que na arquitetura cliente/servidor, o script faz seu processamento no servidor e retorna somente código HTML para o cliente, o que garante segurança no processamento.

2. Resposta: C.

Por padrão da linguagem PHP usamos o sinal de $ antes do nome para indicar que é uma variável dentro do programa em PHP.

3. Resposta: D.

O interpretador PHP faz a atribuição dos tipos de dados dinamicamente, ou seja, dependendo do valor que a variável recebe, é o tipo de dados que o interpretador colocará nela.

4. Resposta: A.

Os operadores de atribuição são usados para atribuir valores ou resultados de uma expressão do lado direito do sinal de atribuição para uma variável que fica do lado esquerdo. Estes estão corretos, pois todos pertencem à tabela de operadores de atribuição (mostrada em aula).

5. Resposta: E.

Os operadores de comparação servem para comparar duas expressões e retornar um valor lógico como resposta, por exemplo, comparar duas idades e saber se a primeira

é maior do que a segunda, retornando um valor verdadeiro como resposta e outras infinitas aplicações.

Unidade 3

Relacionamentos com Ambiente WEB, Desenvolver *Scripts* do Lado do Servidor PHP

Objetivos

1. Neste tema, aprenderemos como os *scripts* PHP se relacionam com ambiente WEB, que pode ser real ou simplesmente um ambiente de teste, como o que criamos no Tema 01. Mostraremos como o ambiente WEB e PHP relacionam-se e como é feita a comunicação entre o navegador e servidor WEB. Enfim, criaremos telas para representar esse processo.

Introdução

Como já mencionado, a linguagem PHP é uma linguagem de *script* que tem seu processamento executado no lado do servidor Apache e que retorna para o cliente; normalmente um navegador que solicita uma requisição, uma página HTML podendo ou não conter dados extraídos de um banco de dados.

Para ilustrarmos como isso funciona na prática, vamos conhecer esse lado dos *scripts* do servidor PHP e verificar como ele executa tudo isso.

1. Relacionamento com a WEB

O que acontece quando um cliente (navegador) solicita uma página PHP para um servidor WEB Apache? Qual sequência essa solicitação percorre para retornar a página de resposta, ou não, ao cliente que a solicitou?

A partir de agora aprenderemos esse processo e como ele retorna as páginas para que possamos verificar suas respostas. Ainda continuaremos com conceitos básicos que não foram possíveis de se esgotar no tema anterior. Quando falamos em comunicação cliente/servidor na WEB, podemos perceber isso na imagem a seguir:

Figura 1 – Comunicação Cliente/Servidor PHP



Fonte: Correia, 2010

Essa imagem mostra que no centro existe um computador de um cliente que solicita uma página ao servidor. O servidor, no momento em que recebe a solicitação, verificará se a página em questão é feita em PHP. Caso ela seja, ele a executará, convertendo seus dados em código HTML e retornando para o computador cliente. Isso acontece se tudo der certo. Casos em que a página não existe, ou em que seja uma consulta a um banco de dados, faz-se o mesmo processo e, depois de convertido tudo em HTML, retorna-se a resposta ao cliente.

2. Desenvolvendo *scripts* do lado do servidor

Quando falamos em *scripts* do lado do servidor, estamos falando em comunicação por meio de formulários. Formulários

são escritos em HTML e, para que façam sentido ao desenvolvimento, terão de trabalhar com alguma tecnologia, como ASP, CGI, PHP, entre outras, a fim de que eles sejam úteis. Quando trabalhamos com formulários, devemos ter em mente duas *tags*, <Form> ... </Form>. (OLIVIERO, 2010)

Neste ponto é que está o segredo, dentro dessas *tags* temos alguns atributos, que devem ser usados para executar ações junto ao PHP. Os mais importantes são:

- Action: atributo que permite informarmos ao servidor qual página será carregada assim que o usuário clicar no botão enviar.
- Method: atributo importante que controla como as informações contidas no formulário devem ser enviadas para o servidor. Pode ser de duas maneiras, com o método GET ou POST.
- Get: acrescenta um ponto de interrogação à URL, seguido de todos os campos e seus respectivos valores.
- Post: transmite as informações de modo invisível, ou seja, não mostra as informações na URL, permitindo que uma quantidade maior de informações possa ser transmitida de uma página para outra.

Dentro de uma *tag* Form, podemos usar uma série de controles para a coleta de dados. Alguns desses controles podem ser:

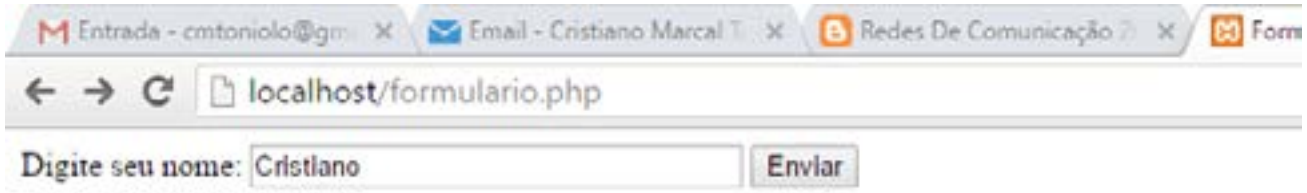
- Caixa de texto.
- Caixa de texto de rolagem.
- Caixa de seleção.
- Botão de opção.
- Menu suspenso.
- Botão de envio.

Para exemplificar, vamos a um exemplo de uma caixa de texto e um botão de envio de dados. Note que precisaremos de dois códigos para executar o programa, um será o formulário e o outro será a página .php, que receberá e exibirá as informações.

O código do formulário é mostrado a seguir e faz o seguinte: na linha 6, começamos a criar o formulário, ou seja, *tag* form e seus atributos que são método POST e nome do arquivo .php, que será procurado no servidor, no caso resposta.php. Na *tag* input, temos o atributo name=”txtNome”, o qual cria uma variável para o campo nome do formulário e será passado para o arquivo .php referenciado anteriormente. A *tag* input com o atributo submit é o comando para referenciar o botão e colocamos o valor Enviar no atributo *value* para que fique personalizado. (OLIVIERO, 2010)

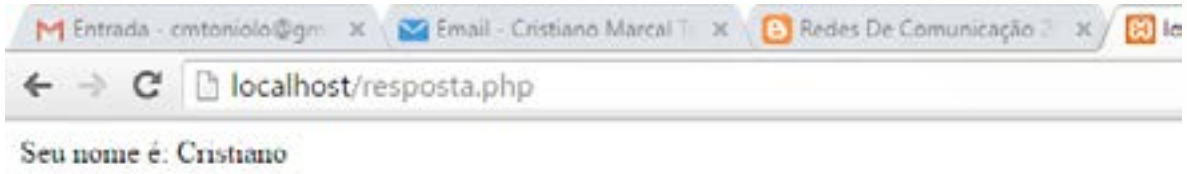
```
1  <html>
2  <head>
3      <title>Formulário em PHP</title>
4  </head>
5  <body>
6  <form method = "POST" action = "resposta.php">
7  Digite seu nome: <input type="Text" size="30" name="txtNome">
8  <input type=submit value="Enviar">
9  </form>
10 </body>
11 </html>
```

Ao rodar o formulário no navegador, veremos o seguinte formulário, simples, mas para mostrar que conseguimos passar os dados para o script .php.



Quando clicar no botão Enviar, os dados são enviados para o script resposta.php que recebe o valor escrito na caixa de texto, atribui para a variável $txtNome e, em seguida, é exibido na tela com o comando *echo*. O resultado é mostrado na seguinte tela do script:

```
1  <html>
2  <body>
3  <?php
4      $txtNome = $_POST["txtNome"];
5      echo "Seu nome é: $txtNome";
6  ?>
7  </body>
8  </html>
```

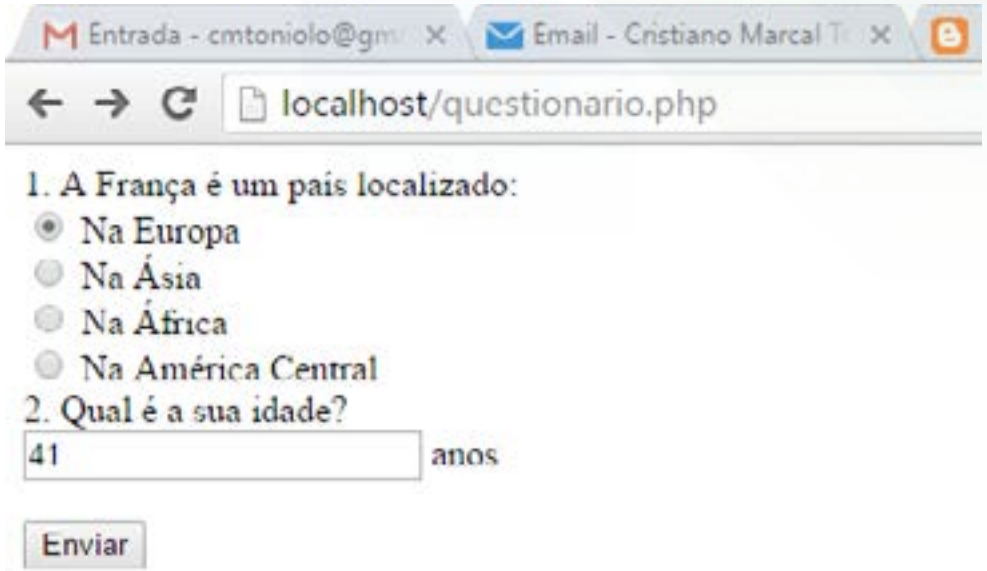


2.1. Usando comandos condicionais

O PHP permite que possamos testar mais de uma opção por meio de comandos de condição, como a instrução *if*. A sintaxe é a seguinte:

```
if(teste de condição) {
    Verdadeiro;
} else {
    Falso;
```

Exemplo de um formulário para um questionário usando os controles de botão de rádio e caixa de texto.



Este é o formulário criado e a seguir está o código HTML para ele, notem as *tags* relacionadas com o acesso ao outro arquivo.

```
<html>
<head>
    <title>Questionário</title>
</head>
<body>
<form method = "POST" action = "respquest.php">
1. A França é um país localizado: <br>
<input type="radio" name="q1" value="Europa"> Na Europa <br>
<input type="radio" name="q1" value="Ásia"> Na Ásia <br>
<input type="radio" name="q1" value="África"> Na África <br>
<input type="radio" name="q1" value="América Central"> Na América Central <br>
2. Qual é a sua idade?<br>
<input type="text" name="txtIdade">  anos<br><br>
<input type=submit value="Enviar">
</form>
</body>
</html>
```

O arquivo anterior cria o formulário que, ao clicar no botão Enviar, chama o arquivo php, chamado de respquest.php, mostrado a seguir:

```
1  <html>
2  <body>
3  <?php
4      $q1 = $_POST["q1"];
5      $idade = $_POST["txtIdade"];
6      if($q1 == "Europa")
7      {
8          echo "Sua resposta está correta <br>";
9      }
10     if($q1 <> "Europa")
11     {
12         echo "Sua resposta está errada. A resposta certa é: Na Europa <br>";
13     }
14     if($idade < "40")
15     {
16         echo "Você tem menos que 40 anos de idade";
17     }
18     else {
19         echo "Você tem 40 anos de idade ou mais";
20     }
21 ?>
22 </body>
23 </html>
```

Aqui podemos ver o uso do comando de condição *if* em duas condições diferentes. A primeira condição começa na linha 6 e serve para testar quando o usuário escolhe uma resposta para os países, por exemplo, se escolher a opção Europa, recebe a resposta correta, caso contrário, recebe uma mensagem de erro com a resposta correta em seguida.

A segunda condição começa na linha 14, quando será testado a idade que o usuário digitou, se for abaixo de 40 anos, recebe a resposta que a idade é menor que 40 anos, caso contrário, recebe a resposta que a idade é igual ou maior que 40 anos.

2.2. Usando os comandos de *loop*

O PHP trabalha com comandos de *loop*, ou seja, comandos que executam um determinado trecho de código, a quantidade de vezes que o usuário necessitar, ou até que uma condição de parada seja satisfeita. (OLIVIERO, 2010)

Dessa forma, esse tópico mostrará o comando de *loop* mais utilizado na linguagem PHP. Esse comando é chamado de *for*, que tem como premissa executar a repetição de um bloco de código em um número de vezes que pode ser conhecido pelo usuário ou não, dependendo do problema ou da aplicação. **Por exemplo, para calcular a tabuada do número 3, o programa executará 10 vezes a multiplicação, agora, se quisermos calcular a média das notas de uma turma, podemos solicitar ao usuário a quantidade de alunos dessa sala de aula. A sintaxe do comando é:** este comando se chama *for* e é executado em um determinado número de vezes que já é conhecido pelo usuário. Sua sintaxe é a seguinte:

```
for(inicialização; condição;
operador)
{
    //trecho de código a ser
repetido;
}
Exemplo:
for($i = 0; $i <= 10; $i++)
{
    echo “Pós-graduação Kroton”;
}
```

Este comando tem sua inicialização com a variável valendo zero (isso ocorre somente uma vez no *loop*), em seguida, essa variável é testada até uma determinada condição, no caso 10. Se esse teste for verdadeiro (menor ou igual a 10) e imprimir a mensagem na tela, incrementa-se a variável em um ($i++), ou seja, depois de incrementado, a variável $i é adicionado 1 no teste para verificar se a condição se tornou falsa, encerrando-se o *loop*.

Glossário

Cliente/Servidor: tecnologia que permite a programação e o acesso a documentos de um computador conectado à internet e obter respostas de um servidor também conectado à internet.

LOOP: conceito em linguagem de programação que tem a finalidade de permitir a repetição de um determinado trecho de código. Os comandos de *loop* conhecidos são: *for, while, do..while*, e, no php, temos o *foreach*, que é muito usado quando trabalhamos com consultas em banco de dados.

ASP: acrônimo de *Active Server Pages*, linguagem de programação cliente/servidor da Microsoft, com o objetivo de criação de WEB sites dinâmicos, tendo o lado cliente responsável pela parte visual das aplicações e o processamento da lógica das aplicações rodando no Servidor WEB.

POST: comando usado em programação para WEB, em que os dados de um formulário podem ser enviados para outra página sem que sejam vistos.

GET: comando usado em programação para WEB, em que os dados de um formulário são enviados para outra página pela sua URL com suas variáveis e valores sendo mostrados.

Questão para reflexão



Até aqui neste tema, vimos como funciona a programação cliente/servidor e seus exemplos, além dos comandos de formulário, condicionais e de *loop*. Deixo aqui uma questão para reflexão: pesquise no site oficial do PHP ou em livros, como o sugerido do autor Oliviero (2010), se existem mais comandos de *loop* e condicionais que possam ser executados em PHP, ou seja, para os alunos que desenvolvem em outras linguagens, isso é verdadeiro, pesquise sobre a linguagem PHP para verificarmos se isso pode ser aplicado também.

Considerações Finais

Neste tema, realmente, começamos a programar para cliente/servidor. Assim, podemos ter noções de como:

- Criar os *scripts* de formulário e os *scripts* de resposta em php.
- Saber como funcionam e para que servem os comandos de decisão.
- Saber como funcionam e para que servem os comandos de *loop*.
- Saber como funcionam os formulários e seus atributos para rodar em cliente/servidor.

Referências

OLIVIERO, C. A. J. **PHP 5.2 com MySQL 5.0**: comércio eletrônico orientado por projeto. 1. ed. São Paulo: Érica, 2010.

CORREIA, J. Redes de comunicação. Disponível em: <http://rcjoel.blogspot.com.br/p/php-hypertext-prepocessor.html>. Acesso em: 12 set. 2016.

Assista a suas aulas



Aula 3 - Tema: Relacionamentos com Ambiente Web, Desenvolver Scripts do Lado do Servidor PHP - Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/ce92e34e242e55e3bb1ff171ff961896>.



Aula 3 - Tema: Relacionamentos com Ambiente Web, Desenvolver Scripts do Lado do Servidor PHP - Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/81a1a283d5aa4188f6e4b329d5aec6a1>.

Questão 1

1. O que é acesso cliente/servidor?

a) Quando um computador cliente processa os dados de um computador servidor que está na internet.

b) Quando o acesso ocorre apenas do lado do servidor, sem o acesso do cliente.

c) Quando é necessário finalizar um comando que não temos no lado do cliente, assim que acessou o servidor.

d) O acesso cliente/servidor ocorre quando um computador cliente solicita uma informação de um computador servidor que está na internet.

e) O acesso cliente/servidor ocorre quando um computador cliente solicita uma resposta de um computador servidor que está na internet.

Questão 2

2. Qual é a função do formulário em um sistema WEB?

a) O formulário tem como função coletar e enviar os dados do cliente para o processamento no lado do servidor.

b) O formulário apenas serve para coletar os dados sem a necessidade de um *script* para seu funcionamento.

c) O formulário serve para coleta de dados e para processar os dados no lado do cliente.

d) O formulário não serve para nada, somente para deixar a tela mais elegante.

e) O formulário serve para que os dados sejam criados de maneira organizada e processados no lado do cliente.

Questão 3

3. O que faz o comando de *loop for*?

a) O *loop for* tem a função de repetir determinado código em um número indeterminado de vezes.

b) O *loop for* não tem a função de repetir determinado código em um número de vezes conhecido.

c) O *loop for* tem a função de repetir determinado código em um número de vezes conhecido.

d) O *loop for* tem a função de executar determinado código em um número de vezes conhecido.

e) O *loop for* tem a função de saber se determinado código tem um número de vezes conhecido.

Questão 4

4. Para que serve o atributo action da *tag form*?

a) O atributo action serve para indicar ao servidor a página php que não será executada com os dados do formulário.

b) O atributo action serve para indicar ao servidor a página php que tem os dados do formulário.

c) O atributo action serve para indicar ao servidor a página html que será executada com os dados do formulário.

d) O atributo action não serve para indicar ao servidor a página php que será executada com os dados do formulário.

e) O atributo action serve para indicar ao servidor a página php que será executada com os dados do formulário.

Questão 5

5. Por que temos que ter um código.php para processar os dados do formulário?

a) Porque os dados devem ser jogados fora e atribuídos em variáveis para processamento no lado do servidor.

b) Porque os dados devem ser tratados e atribuídos em variáveis para processamento no lado do servidor.

c) Porque os dados devem ser tratados e não atribuídos em variáveis para processamento no lado do servidor.

d) Porque os dados devem ser tratados e atribuídos em variáveis.

e) Porque os dados não devem ser tratados e atribuídos diretamente em variáveis para processamento no lado do servidor.

Gabarito

1. Resposta: D.

Isso se explica pelo fato de que o servidor WEB, por exemplo Apache, permite que o código seja processado em segurança que retorne somente uma página com código HTML.

2. Resposta: A.

É por meio de um formulário que ocorre a interação entre usuário e sistema e entre programas WEB e servidor. Os dados inseridos devem ser processados por um *script* que deverá estar rodando do lado do servidor, assim, com maior segurança.

3. Resposta: C.

Comando *for* executa um *loop*, ou seja, repete um determinado número de vezes o trecho de código que necessitamos, por exemplo, executar uma busca em um banco de dados por todos os alunos que têm entre 18 e 20 anos varrerá o banco até que essa condição seja falsa, não tendo mais alunos nessa faixa de idade.

4. Resposta: E.

É o atributo que o servidor WEB verifica para saber qual script será executado. Assim, o processamento será correto.

5. Resposta: B.

Código que tem como responsabilidade fazer o processamento dos dados que são recebidos de um determinado formulário, ou seja, se existe a necessidade de calcular desconto em imposto de renda, os dados sobre impostos de renda devem ser recebidos por esse formulário.

Unidade 4

Ajax + PHP – Relacionamentos com ambiente WEB

Objetivos

1. Mostrar ao aluno como a tecnologia AJAX ajuda na dinâmica de um sistema WEB e como ela resolve muitos problemas na hora de atualizar uma página, mesmo que isso seja considerado por alguns um problema pequeno em relação ao código, mas muito importante no que se diz respeito à experiência do usuário em relação à navegação e atualização das páginas na WEB.

Introdução

O Ajax é um conceito criado por Jesse James Garret, que significa *Asynchronous JavaScript and XML*, ou seja, um conceito que vai muito além da junção de JavaScript com XML. É um método de navegação e atualização de páginas WEB. Com AJAX, podem-se criar aplicações que, com apenas um clique em um botão, atualizam o conteúdo de uma seção da página em questão e não a página toda, fazendo que se tenha maior velocidade nos itens que são necessários. Exemplos de aplicações são páginas em que apenas o conteúdo principal deve ser atualizado, como os portais de notícias e o Google Maps, o mais conhecido, no qual somente a área do mapa é atualizada, enquanto que os outros componentes da página continuam da mesma forma como foram carregados.

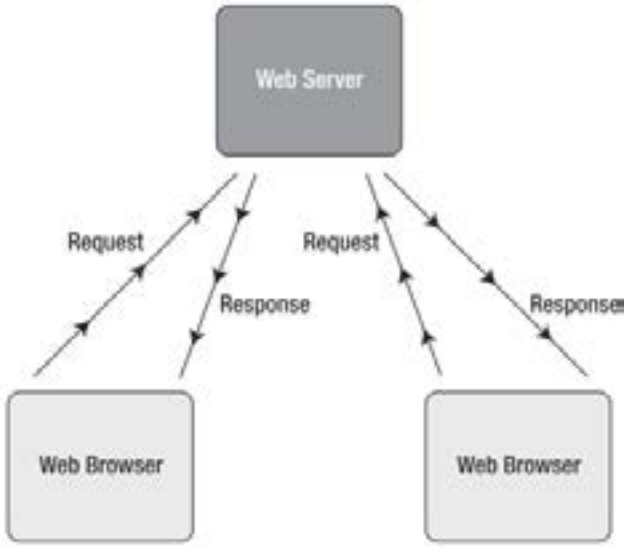
1. Explicando o Ajax

Na WEB de antigamente, as páginas eram totalmente estáticas e com uma grande quantidade de texto. Após o surgimento dos navegadores com recursos gráficos, as pessoas que desenvolviam para WEB começaram a usar mais recursos que os navegadores suportavam e, com o avanço da internet, ficou mais fácil a inclusão de tabelas, vários tipos de fontes e recursos de atualização dinâmica, que demanda linguagens de programação voltadas para WEB, como o PHP, ASP, entre outras (SOARES, 2006).

Assim, surge também mais um problema, que é a atualização dependente do envio de dados ao servidor e a espera por uma

página de resposta, conforme Soares (2006), “criando o efeito de ‘*refresh*’ da página”. Para evitar isso, alguns recursos foram acrescentados às páginas, tais como o pré-carregamento de dados, o uso de frames e iframes.

Figura 2 – Método de Requisição/Resposta na maioria dos sites Web na Internet



Fonte: Babin (2007, p. 2)

Isso faz que o usuário tenha uma ilusão de que os dados estão sendo apresentados na medida em que ele necessita deles, mas, na verdade, os dados já foram carregados desde o momento em que a página foi carregada, o que continua com a impressão de “refresh” dela, causando baixa performance na comunicação dos dados.

Podemos citar dois problemas no uso de aplicações sem Ajax:

1. Os valores são estáticos, ou seja, são carregados no momento do acesso à página WEB, gerando uma nova consulta ao servidor WEB toda vez que um novo dado é necessário.
2. Quanto maior for a quantidade de dados que necessitamos, maior será

> o tamanho do arquivo HTML gerado para o navegador. Assim, o uso de atualização sem AJAX comporta-se bem com uma quantidade de dados de até 1000 itens, como o caso de produtos; acima disso, fica inviável (SOARES, 2006).

Quando existe a necessidade de se programar usando Ajax para o desenvolvimento de páginas dinâmicas para WEB, devemos pensar na atualização de dados de somente uma variável, por exemplo, o preço de um determinado produto. Isso é o que o Ajax resolve, isto é, uma forma de poder separar o dado da parte gráfica da aplicação.

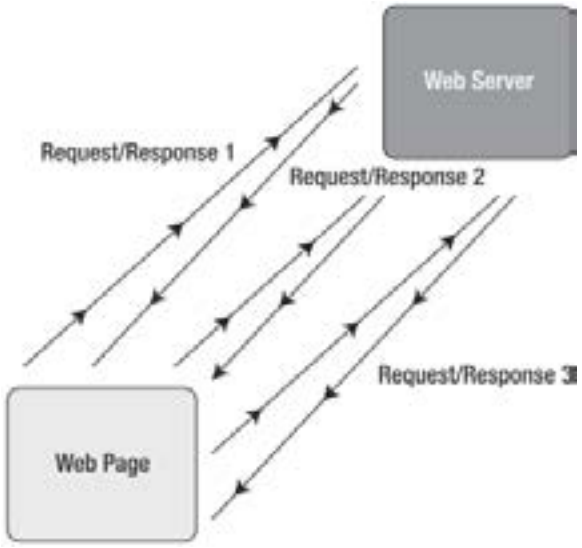
Mas para que possamos utilizar toda a funcionalidade do Ajax em desenvolvimento WEB, necessitamos saber de todos os preceitos do método estudado, são eles: o uso de JavaScript, XML, DOM e CSS.

JavaScript é a linguagem de *script* que usamos para dar mais dinâmica às páginas WEB do lado do cliente e fazer validações de dados. XML, é a linguagem de marcação que denota a criação de *tags* específicas de acordo com cada aplicação, atualmente, muito utilizado para transferência de dados entre aplicações. DOM é chamado de Modelo de Objeto de Documentos, item de grande importância na navegação e manipulação dos elementos (nós) de um documento XML.

A seguir vemos um modelo de como se comporta uma página WEB usando o

conceito de Ajax no seu desenvolvimento. Podemos notar que as requisições podem ser feitas para o servidor sem que haja a necessidade de que a página seja recarregada a cada nova consulta, ou seja, sem o “refresh”.

Figura 3 – Modelo de requisição/resposta usando a metodologia assíncrona Ajax



Fonte: Babin (2007, p. 7)

Antes de sabermos mais sobre o Ajax e o PHP, devemos entender como uma página WEB processa uma requisição e recebe a resposta de um Servidor WEB. Todos os navegadores utilizam-se do protocolo HTTP para enviar e receber informações, o que significa que o navegador envia uma requisição por meio de um site WEB e recebe uma resposta de um servidor WEB, o qual está preparado para retornar aquela resposta. Para que essa comunicação seja executada, existem várias formas de métodos de requisição disponíveis. Alguns mais conhecidos, como GET e POST, são bastante utilizados, mas existem outros métodos.

O objeto XMLHttpRequest é um conceito usado para descrever a interação do

lado do cliente com os *scripts* do lado do servidor. Este objeto contempla uma grande quantidade de métodos e propriedades, que serão estudados e aplicados em outro tema, o qual trata de criação de aplicações dinâmicas para WEB. Um exemplo de envio de requisição para o servidor pode ser visto no código a seguir:

```
function makerequest(serverPage, objID) {

  var obj = document.getElementById(objID);
  xmlhttp.open("GET", serverPage);
  xmlhttp.onreadystatechange = function() {
    if (xmlhttp.readyState == 4 && xmlhttp.status == 200) {
      obj.innerHTML = xmlhttp.responseText;
    }
  }
  xmlhttp.send(null);
}
```

Basicamente a funcionalidade aqui ocorre na requisição do ID do elemento HTML e a página do servidor, além de usar a função open() para que possa ser feita a conexão com a página do servidor WEB, mostrando-nos que a conexão pode ser feita de maneira correta e funcional, sem se esquecer de que este código foi escrito em JavaScript.

2. Ajax + PHP

Ajax e PHP são duas tecnologias que podem ser usadas em conjunto por trabalharem usando o conceito de Cliente/Servidor, ou seja, quando falamos no lado do cliente, usamos o Ajax, enquanto que a linguagem JavaScript é utilizada para

tratar informações do lado do navegador do usuário. Quando falamos do lado do servidor, usamos a linguagem PHP, a qual permite que as páginas WEB fiquem mais dinâmicas sem a necessidade de se fazer várias requisições de dados ao servidor, ou seja, os dados são carregados e, quando há a necessidade de sua atualização, somente os que necessitam ser atualizados serão enviados pelo servidor.

Parece uma coisa simples, mas ao tratarmos sobre comunicação entre cliente e servidor, quanto menos tráfego pudermos gerar na rede, melhor será a performance dos sistemas, como no nosso caso, sistemas voltados para WEB.

O Ajax, conforme dito anteriormente, é uma tecnologia que une várias ferramentas para que o conceito seja colocado em funcionamento. Duas tecnologias usadas no Ajax são: o XML (*Extensible Markup Language*) e a API DOM (*Document Object Model*). Elas realizam com o PHP a integração de sistemas, construção de aplicações multicamadas, entre outras formas de resolução de problemas.

Outra ferramenta importante para que o conceito seja viável é a classe simpleXML, sobre a qual falaremos a seguir junto ao API DOM.

2.1. SimpleXML

Classe disponível no PHP desde a versão 5, que tem como finalidade a manipulação de arquivos e *strings* XML, mas sem a

criação deste documento (XML). Sua funcionalidade se resume em três passos: fazer a carga de um arquivo XML, ou de uma *string* XML, ou pela importação de um documento criado pela API DOM.

Para que esse processo funcione, devemos usar duas funções: *simplexml_load_file* e *simplexml_load_string*.

2.1.1. Simple_load_file

Tem como finalidade fazer a carga de um arquivo XML existente. Assim, se essa carga for executada de forma correta, a variável que receber o resultado da função se tornará um objeto da classe *simpleXML*. Sua sintaxe é a seguinte:

```
Objeto simplexml_load_file(nome_
arquivo);
```

Explicando, objeto é a variável que receberá o resultado da carga do arquivo XML, nome_arquivo é o arquivo XML existente que, se corretamente formatado, será usado para fazer a carga pela função.

2.1.2. simple_load_string

Sua finalidade é a de fazer a carga de uma *string* contendo um documento XML que deve ser bem estruturado. Seu resultado, como a primeira função, é a criação de um objeto com os dados contidos na *string* carregada. A seguir temos a sintaxe dessa função:

```
Objeto simplexml_load_string(string_xml);
```

em que o objeto é a variável que recebe o resultado da carga contida na *string*, passada como parâmetro para a função. String_xml é a *string* contendo o documento XML existente e bem estruturado para que se possa usar os métodos disponíveis na classe *simpleXML*.

Para que possamos entender esse funcionamento, a seguir será criado um exemplo simples que poderá ser executado no ambiente de testes criado no início deste material. Primeiramente, cria-se o documento XML estruturado que deverá ser salvo com o nome de **exemplo.xml**.

```xml
<?xml version='1.0' encoding='utf-8' ?>
<livros version='1'>
    <item>
        <titulo>Ajax, Rich Internet Aplication e desenvolvimento WEB</titulo>
        <autor>Paul J. Deitel, Harvey M. Deitel</autor>
        <editora>Pearson</editora>
        <data>2008<data>
        <paginas>750</paginas>
    </item>
    <item>
        <titulo>Crie um site B2C com PHP4 e MySQL</titulo>
        <autor>Walace Soares</autor>
        <editora>Erica</editora>
        <data>2001<data>
        <paginas>620</paginas>
        <isbn>8571947740</isbn>
    </item>
</livros>
```

Explicando esse arquivo: na primeira linha é mostrado o cabeçalho com a versão XML usada e o encoding referente à acentuação das palavras. A segunda e a última linhas do arquivo são referentes à raiz do XML, que, conforme as regras de criação de um arquivo XML, pode estar relacionada ao objetivo que está sendo trabalhado, no caso livros. Dentro da raiz, temos dois livros que têm seus dados envolvidos pelas tags `<item>`, como se fossem registros de um banco de dados.

O próximo arquivo a ser editado é o arquivo PHP, que irá tratar do carregamento do arquivo `exemplo.xml` com a função `simplexml_load_file()`. Este deverá ser salvo com o nome de `load.php`.

```php
<?php
    $_xml = simplexml_load_file('exemplo.xml');
    echo "<pre>";
    var_dump($_xml);
    echo "</pre>";
?>
```

Explicando os itens desse arquivo, na primeira linha foi criada uma variável chamada $_xml, que será responsável por receber a carga do documento xml a ser carregado, e a função simplexml_load_file(), que tem como parâmetro o arquivo exemplo.xml, criado anteriormente e corretamente estruturado.

Os dois comandos echo fazem a "impressão" na tela dos dados do arquivo xml pré-formatados (relembrar os comandos html se necessário) para que estes dados sejam mostrados de uma

maneira estruturada. A função PHP var_dump mostrará as informações contidas no objeto $_xml.

2.2. Métodos usados na classe simpleXML

Nesta seção, será mostrado como usar os métodos da classe simpleXML, que têm como finalidade a manipulação dos elementos e *tags* existentes dentro dos arquivos XML.

2.2.1. Attributes

Método que retorna todos os atributos existentes em tags XML, retornando-os por esses métodos ou por meio de uma array ($obj -> elemento[nome_atributo]).

2.2.2. Children

Usado para encontrar os filhos, ou seja, os nós descendentes de outro nó em um documento XML, devolvendo um objeto XML com os filhos do nó informado. No exemplo do livro, o nó <livro> terá dois filhos que têm o mesmo nome <item>.

2.2.3. Xpath

Método que executa uma busca nos nós do documento XML a procura dos filhos que satisfarão o critério de busca informado no método. Exemplo de formato de parâmetro: “/no_1/no_2/.../no_n”, em que no_1 até no_n é a quantidade de nós que uma estrutura XML pode conter.

2.3. DOM

DOM ou *Document Object Model* é uma interface que está disponível no PHP desde sua versão 5 e que, atualmente, está aderente ao padrão DOM 2. Ela permite a criação de documentos XML a partir do zero, ou seja, não existe a necessidade de se ter um documento XML pronto para uso, além de poder manipular documentos que já existem, de forma mais complexa. Agora, serão mostrados os principais métodos que podemos usar nas classes, tais como DomDocument, DomNode, DomElement, DomText, DomAttr, entre outros métodos.

2.3.1. Instanciando a classe DomDocument

Para usar o DOM, primeiramente, devemos instanciar a classe DOM e, em seguida, criar um documento XML que pode ser um documento novo, a importação de um documento ou uma *string* contendo um documento XML válido. Assim, é necessário fazer a instância da classe DomDocument, que pode ser feita usando o operador ***new***. Este operador é usado por algumas linguagens de programação orientada a objetos com a linguagem Java. A seguir um exemplo de instanciação:

```
<?php
    $_dom = new DomDocument();
?>
```

Podemos observar que foi feita a declaração de uma variável chamada $_dom, que se torna um objeto pelo fato de ser instanciada pelo operador ***new***, referenciando o construtor da classe DomDocument.

2.3.2. Load e loadXML

A diferença entre esses dois métodos é que o primeiro faz a carga de um arquivo XML e o segundo comando é a carga de uma *string* XML. Para que haja entendimento do funcionamento, dois exemplos serão mostrados a seguir:

```
<?php
    $_dom = new DomDocument();
    $_dom->load('exemplo.xml');
    echo "<pre>";
    $_dom->formatOutput = TRUE;
    echo htmlentities($_dom->saveXML());
    echo "</pre>";
?>
```

Na linha três deste programa, nota-se o método load() sendo passado como parâmetro o arquivo exemplo.xml, mas, antes disso, foi feita a instanciação do objeto $_dom. A seguir será mostrado o método loadXML em outro exemplo:

```php
<?php
    $_dom = new DomDocument();
    $_xml2 =
        '<estado uf="SP">
            <cidade nome="Santo Andre">
                <bairro nome="Jardim">
                    <rua>Figueiras</rua>
                    <rua>Goias</rua>
                    <rua>D.Pedroll</rua>
                </bairro>
            </cidade>
            <cidade nome="S.Bernardo do Campo">
                <bairro nome="Parque Espacial">
                    <rua>Gabriel de Souza</rua>
                    <rua>Cap.Casa</rua> </bairro>
            </cidade>
        </estado>');
    $_dom->loadXML($_xml2);
    echo "<pre>";
    $_dom->formatOutput = TRUE;
    echo htmlentities($_dom->saveXML());
    echo "</pre>";
?>
```

A linha $_dom->loadXML($_xml2) é onde foi feito o carregamento da *string* XML, a qual foi criada na variável $_xml2 após a instanciação do objeto $_dom. Assim, o parâmetro passado para o método foi a variável contendo a *string* e, depois disso, foi exibido na tela e salvo os dados com o método saveXML().

Glossário

Ajax: acrônimo de *Asynchronous Javascript and XML*, um conceito de transferência e interatividade de páginas WEB que pode ser programado com base em uma série de ferramentas, como XML. Ele faz que as páginas fiquem mais interativas para os usuários e que se possa usar as transferências assíncronas das informações.

Operador *new*: usado quando existe a necessidade de se fazer a instanciação de um objeto, ou seja, um conceito de orientação que permite a criação de objetos referentes a uma classe na qual estes passam a ter todos os atributos e métodos dela.

DOM: interface que permite a criação e manipulação de arquivos XML a partir do zero e existentes. Tem uma série de atributos e métodos para fazer a manipulação das *tags* criadas dentro dos documentos XML, que permitem listar, adicionar, entre outras funções, os componentes de um documento.

Questão para reflexão



Neste tema, foi apresentado um conceito muito importante e que, atualmente, está em destaque: o desenvolvimento de aplicações PHP usando o Ajax. No tema foi descrito os conceitos iniciais de Ajax com PHP, mas não foi esgotado todo o conteúdo sobre este tema, pois ele é muito amplo. Então, o aluno deverá refletir sobre o que realmente pode ser feito com o Ajax e a linguagem PHP e quais são os recursos que podem ser usados por essas duas tecnologias?

Considerações Finais

Ajax é um conceito que pode ser aplicado usando várias ferramentas, como XML, JavaScript, entre outras, e pode ser facilmente integrado com o PHP no desenvolvimento de aplicações para WEB. Assim, foram abordados:

- Qual é o funcionamento de uma aplicação Ajax e seus conceitos em relação a aplicações cliente/servidor.
- Como carregar um arquivo, uma *string* ou uma importação de um documento XML por meio da classe *simpleXML*.
- Como carregar um arquivo, uma *string* ou criar um documento XML com DomDocument da API DOM incorporada no PHP.
- Foi feito o uso da classe simpleXML, da API DOM e dos métodos load e loadXML.

Referências

BABIN, Lee. **Beginning Ajax with PHP, from novice to professional.** New York: Editora Apress, 2007.

SOARES, Wallace. **Ajax (Asynchronous Javascript and XML), guia prático para Windows**. 1. ed. São Paulo: Érica, 2006.

Assista a suas aulas



Aula 4 - Tema: AJAX + PHP – Relacionamentos com Ambiente Web - Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/1ac948477bdc640199d540ec3c87fc5a>.



Aula 4 - Tema: AJAX + PHP – Relacionamentos com Ambiente Web - Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/a5f-4fc4ea2f9a831d295f9612b4ded3d>.

Questão 1

1. Qual é a definição de Ajax?

a) Conjunto de linguagens de programação e desenvolvimento WEB, que utiliza tecnologias para carga de informações de forma assíncrona.

b) Conjunto de técnicas de programação e desenvolvimento WEB, que utiliza tecnologias para deixar a página estática.

c) Conjunto de técnicas de programação e desenvolvimento WEB, que deixa de utilizar tecnologias para carga de informações de forma assíncrona.

d) Quantidade de técnicas que utiliza programação assíncrona para a criação de programas de computador.

e) Conjunto de técnicas de programação e desenvolvimento WEB, que utiliza tecnologias para carga de informações de forma assíncrona.

Questão 2

2. Para que usamos a API DOM?

a) Usada para representar uma *string* XML de forma estruturada.

b) Usada para representar um documento XML de forma estruturada.

c) Usada para representar uma página em XML de forma estruturada.

d) Usada para representar um documento XML de forma não estruturada.

e) Usada para ver um documento XML de forma estruturada sem conectar na internet.

Questão 3

3. Porque usamos a classe *simpleXML*?

a) Para criar e fazer o carregamento de documentos XML.

b) Para definir como os documentos XML serão apresentados na tela.

c) Para fazer somente o carregamento de documentos XML.

d) Para fazer o carregamento de documentos XML, *string* XML e importação de documentos XML já prontos.

e) Para fazer o carregamento de documentos XML, *string* XML e importação de documentos XML desde o zero.

Questão 4

4. A instrução: $_dom = new DomDocument(); é usada para quê?

a) Para deixar de fazer instância de um objeto da classe DomDocument.

b) Para fazer a instância de um objeto da classe DomDocument.

c) Usa-se o comando *new* para criar uma variável do tipo DomDocument.

d) Para fazer a instância de um objeto do método DomDocument.

e) Para fazer a criação de um objeto da classe DomDocument.

Questão 5

5. O que faz o método load e loadXML?

a) O load faz a carga de um documento XML, enquanto que o loadXML faz a carga de uma *string* XML.

b) O loadXML faz a carga de um documento XML, enquanto que o loadXML faz a carga de uma *string* XML.

c) O load faz a carga de um documento XML, enquanto que o load faz a carga de uma *string* XML.

d) O loadXML faz a carga de um documento XML, enquanto que o load faz a carga de uma *string* XML.

e) O load faz a carga de uma *string* XML, enquanto que o loadXML faz a carga de um documento XML.

Gabarito

1. Resposta: E.

O Ajax, por meio de várias tecnologias, permite que os desenvolvedores possam oferecer mais interatividade em suas páginas WEB, além de permitir a transferência assíncrona de dados, o que faz que o tráfego da rede diminua bastante.

2. Resposta: B.

A API DOM tem uma variedade de métodos e atributos que permite a representação de documentos XML, além de manipular e criar documentos, caso seja necessário.

3. Resposta: D.

Essa classe permite que os documentos XML sejam carregados para visualização ou manipulação dos dados. Diferentemente da API DOM, a *simpleXML* não permite a criação de documentos XML, só a carga de documentos existentes.

4. Resposta: B.

Comando ou instrução que faz a instanciação de um objeto, isso significa que a variável criada passa a ser um objeto que contém todos os atributos e métodos contidos na classe que foi utilizada, no caso, a DomDocument.

5. Resposta: A.

Dois métodos utilizados para manipulação de dados que fazem parte da API DOM onde o primeiro faz a carga de um documento XML e o segundo faz a carga de uma string referenciando um documento XML válido.

Unidade 5

Banco de Dados com PHP, Utilização do Banco com a Linguagem PHP

Objetivos

1. Nos dias atuais, os sistemas WEB dinâmicos têm a necessidade de se integrar com um banco de dados para todas as finalidades, seja para o armazenamento de usuários e senhas, seja para gerenciar websites complexos, como os portais de notícias e lojas virtuais. Seguindo esta orientação, abordaremos o uso do PHP com o banco chamado MySQL, para que possamos verificar o poder da integração da linguagem de programação com os recursos de manipulação deste banco de dados.

Introdução

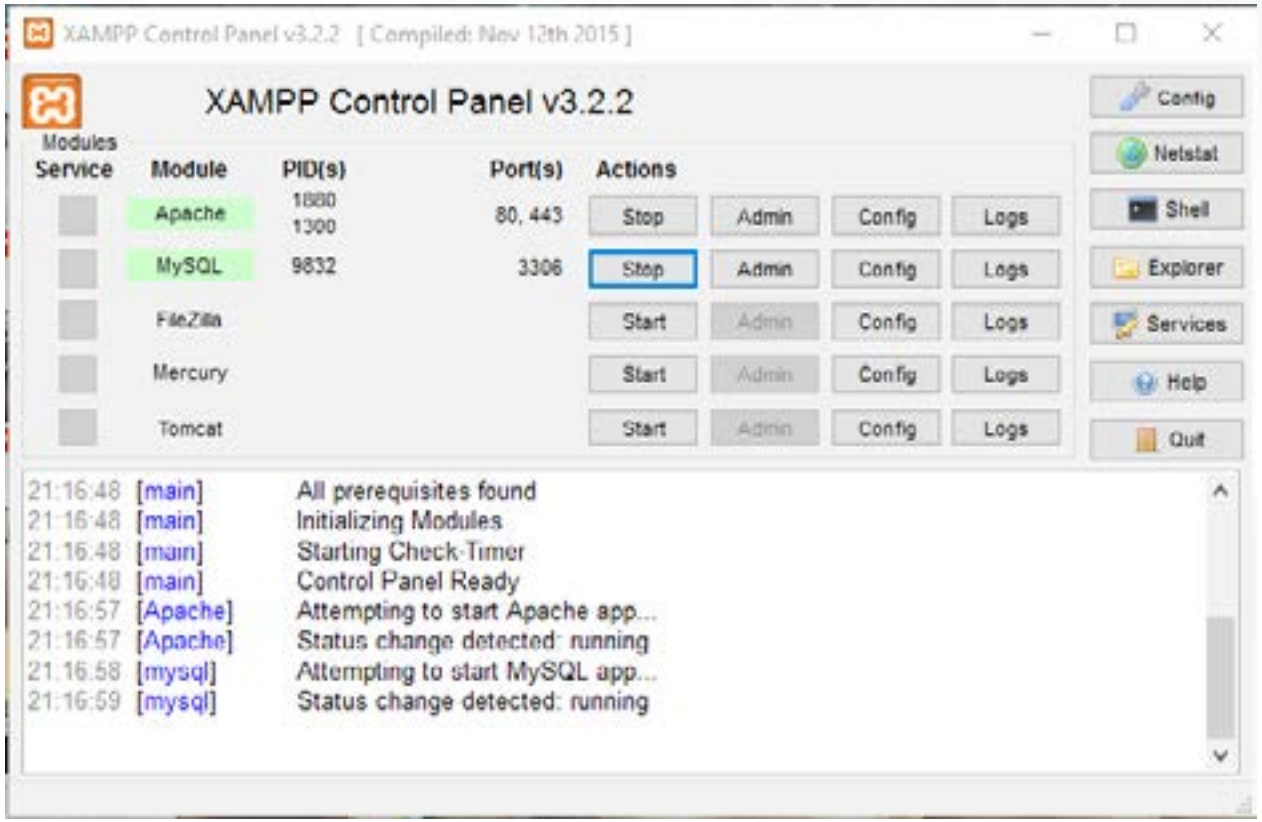
Depois de mostrar os conceitos básicos e as operações que podemos executar com o PHP, este tópico mostrará as ferramentas que a linguagem dispõe para acesso a dados que permitem uma conexão com um banco de dados, como o MySQL. Essas ferramentas, além de fazer a conexão com o banco, o armazenamento e a recuperação dos dados, tem funções de inserção, atualização, exclusão e consulta aos dados, podendo o acesso ser de dentro da empresa, fora dela ou em qualquer parte do mundo (OLIVIERO, 2010).

Conhecer como manipular os dados armazenados no banco de dados é o segredo do sucesso para o desenvolvimento de páginas dinâmicas, pois essa é uma das principais tarefas dos desenvolvedores.

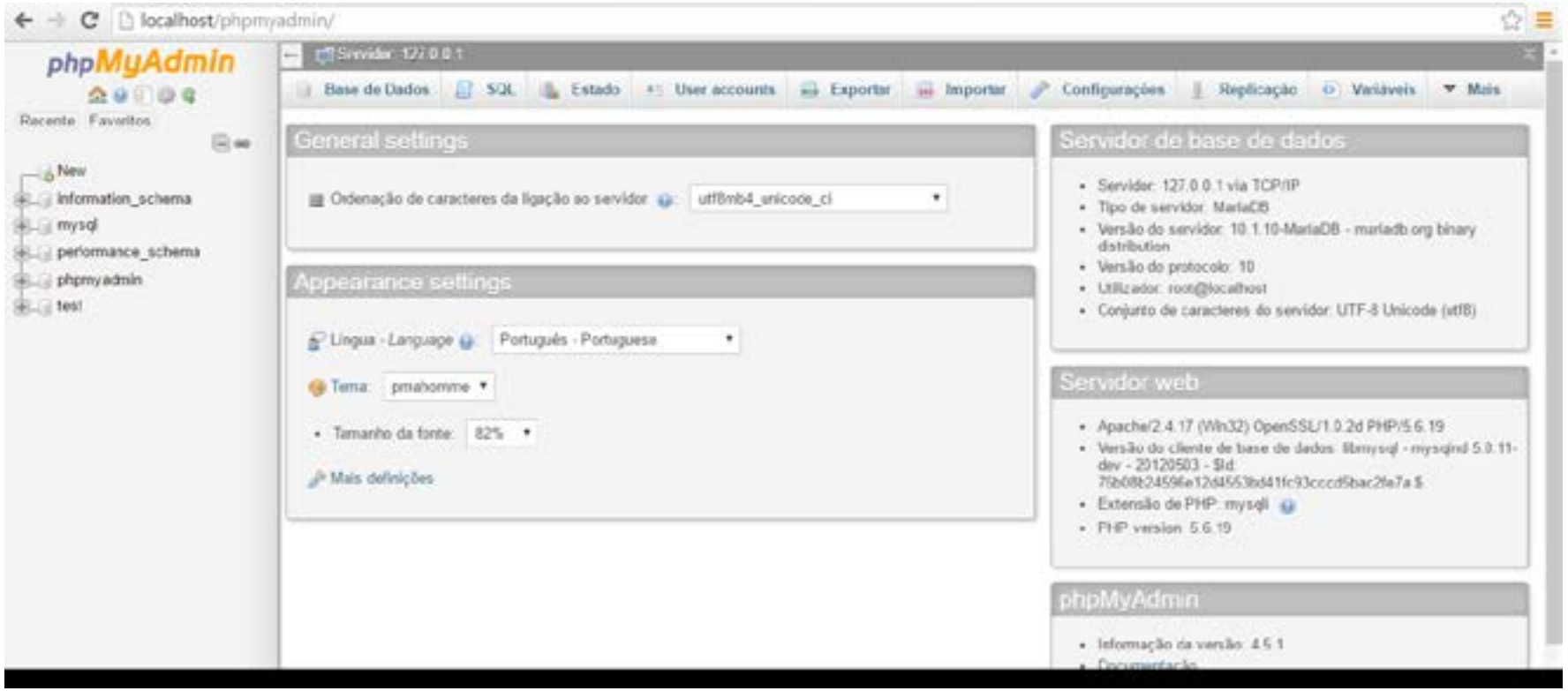
1. Criando o banco de dados

Este tópico abordará a criação do banco de dados que será utilizado para manipulação dos dados com os comandos PHP. O gerenciador de banco de dados nativo do PHP é o phpmyadmin, que será acessado pelo gerenciador do XAMPP usando o endereço de *loopback*, conforme visto anteriormente. Para criarmos o banco de

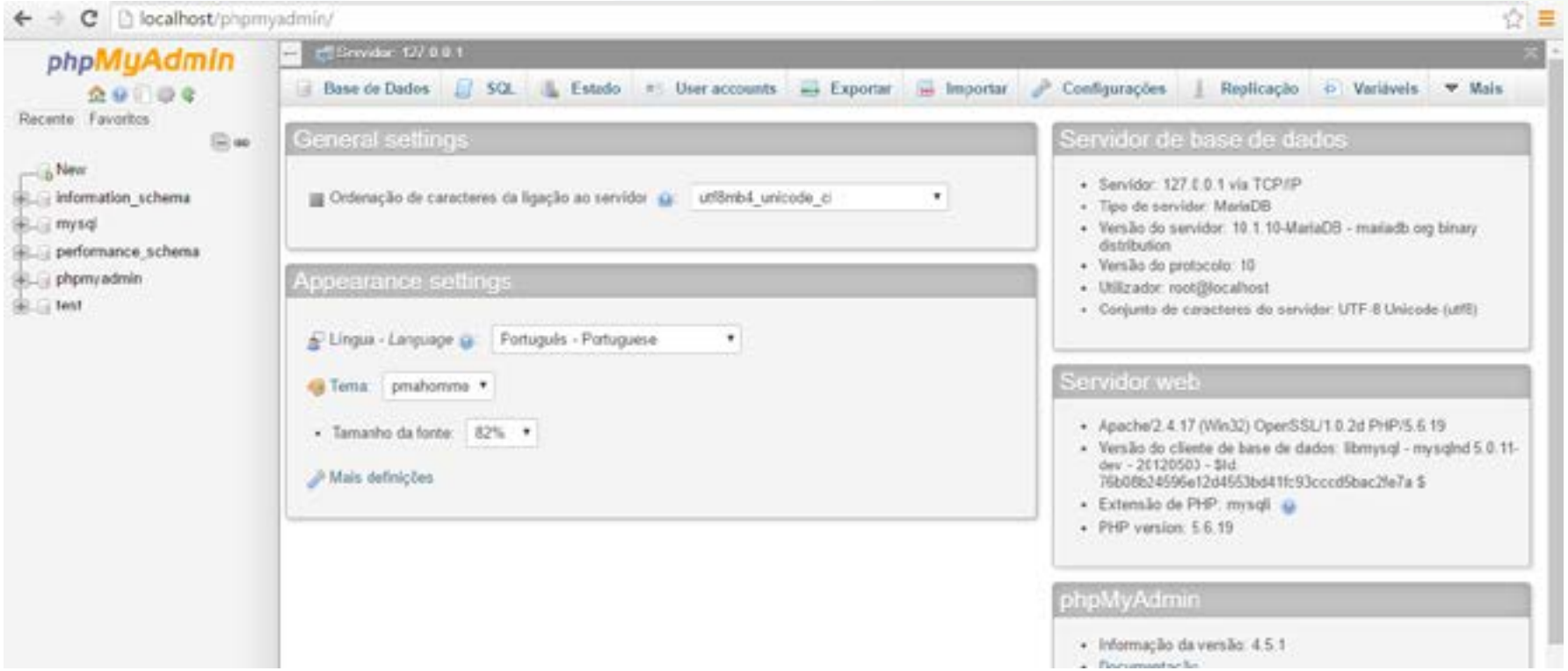
dados, primeiramente, acessaremos o painel de controle do XAMPP, inicializando o servidor WEB Apache e o banco de dados MySQL.



Perceba que os dois programas foram acionados pelo fato de que em *Actions* os botões estão com o status de *stop*, ou seja, se quisermos parar um dos dois basta clicar no botão correspondente. Depois de iniciar os programas, na barra de endereços deverá ser digitado o endereço *localhost* ou o endereço de IP *127.0.0.1*, que será redirecionado para a seguinte tela:



O aluno perceberá que no menu anterior tem uma opção para acessar o phpMyAdmin, o qual deverá ser clicado para que possa acessar a página do gerenciador de banco de dados.



Veja que nesta tela existem alguns itens que devem ser abordados, por exemplo, do lado esquerdo da tela são listados todos os bancos de dados criados pelo gerenciador phpMyAdmin.

Acima da tela, existem vários menus que têm uma característica específica para cada objetivo no gerenciador, como o menu Base de Dados, no qual existem os comandos para criar um novo banco de dados, o menu SQL, que tem como finalidade a criação de banco de dados pela digitação de *scripts* SQL, o menu Estado, que mostra informações sobre quanto tempo o servidor MySQL está funcionando, entre outros que serão abordados na disciplina de Banco de Dados com mais detalhes.

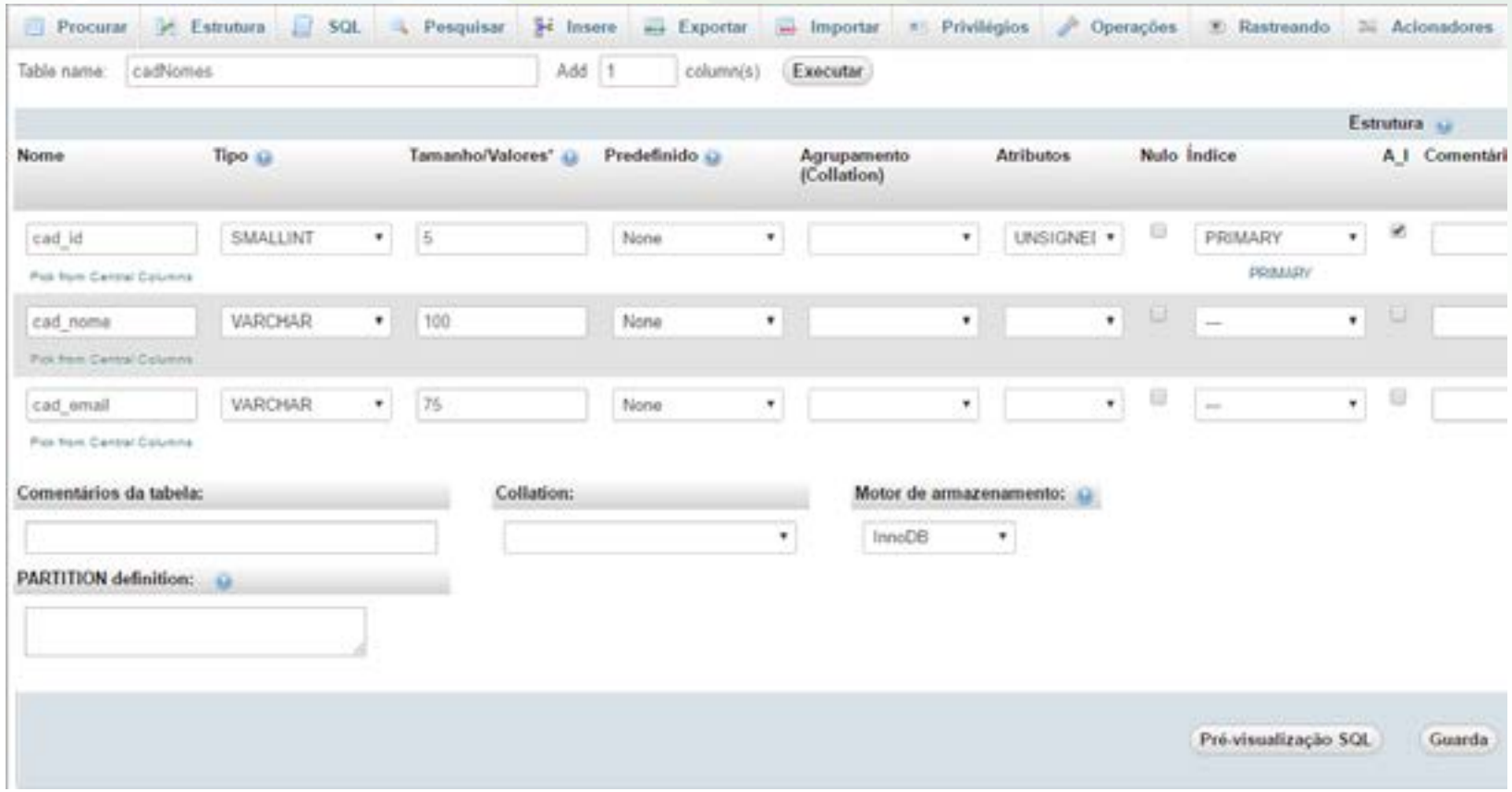
Como exemplo, será criado um banco de dados para fazer um pequeno cadastro, assim, clique no menu Base de Dados do phpMyAdmin, digite, em seguida, no cadastro no campo do nome do banco de dados e, por fim, clique no botão criar:



A tela a seguir permite a criação das tabelas do nosso banco de dados. Note que no menu do lado esquerdo está listado o banco que foi criado. Iremos nomear a tabela chamando de cadNomes, além de ter três colunas com a seguinte forma:

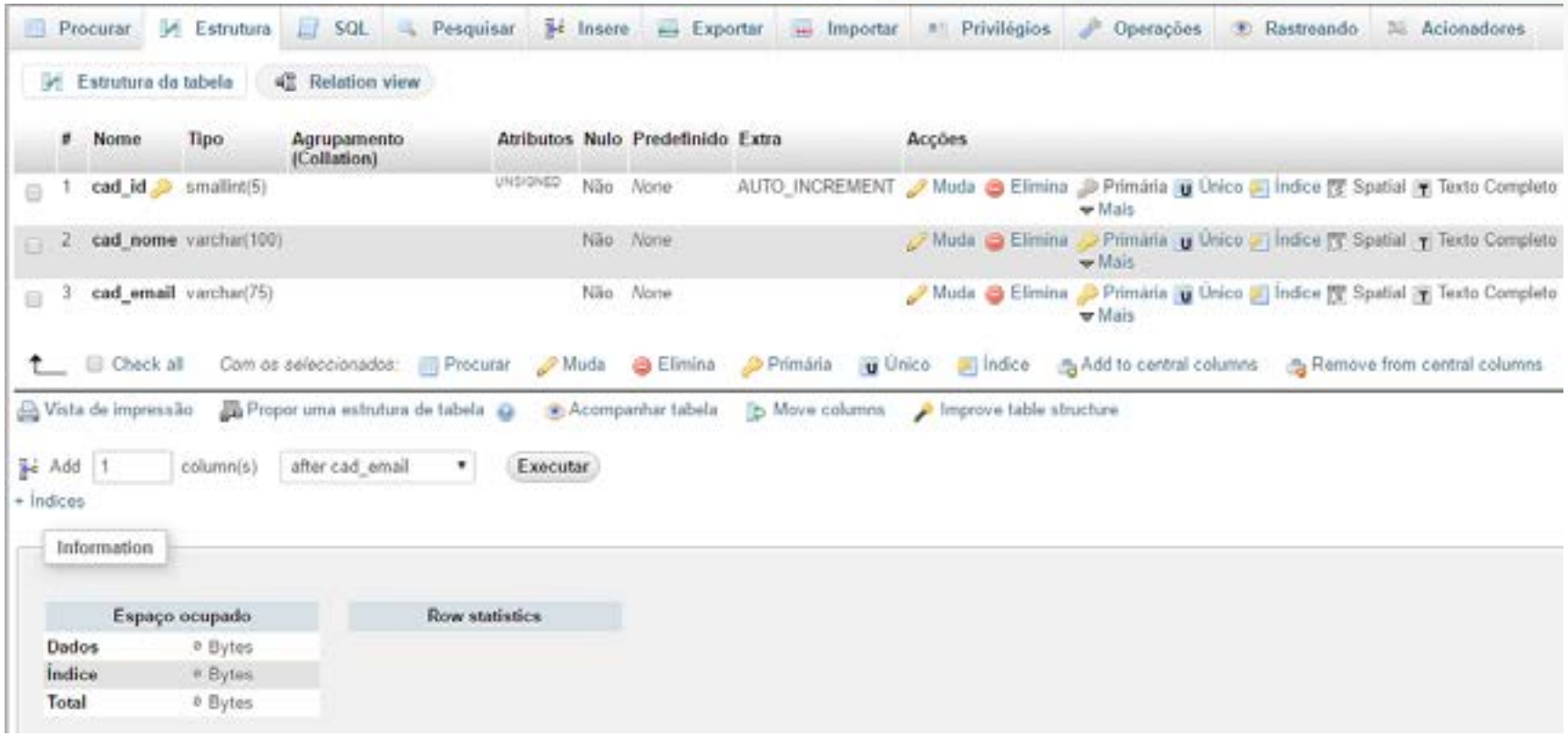


Depois de clicar no botão Executar, uma nova tela será apresentada para que possamos definir as colunas do banco de dados. Elas terão a seguinte configuração:



Os campos foram: cad_id, cad_nome e cad_email, ou seja, para um cadastro simples, somente para mostrar os comandos de manipulação de dados do PHP. Após clicar no botão Guarda, a tela final é mostrada a seguir, na qual podemos perceber que o campo cad_id tem uma

pequena chave amarela ao lado, que indica ser a chave primária da tabela, e que existem várias opções para edição e alteração dos campos dela, caso haja necessidade.



2. Conectando o PHP ao Banco de Dados

Nesta seção, será mostrado como conectar o PHP ao banco de dados criado na seção anterior, ou seja, quais comandos serão utilizados para isso. Dessa forma, devemos editar um arquivo específico para conexão com o banco, o qual daremos o nome de dbcon.php, mostrado a seguir:

```
1  <?php
2      $nome_host = 'localhost';
3      $nome_usuario = 'root';
4      $senha = '';
5      $banco = 'cadastro';
6      $db = mysql_connect($nome_host, $nome_usuario, $senha);
7      mysql_select_db($banco, $db);
8  ?>
```

Explicando esse código, podemos notar a criação de quatro variáveis a partir da linha 2 até a linha 5 do programa, que, respectivamente, são $nome_host: referente ao servidor em que está o banco de dados, no caso, o servidor local, $nome_usuario: referente ao usuário do banco de dados, no caso, o usuário é root, ou seja, usuário padrão do MySQL (lembrando que esse

ambiente é para testes). Após isso, vem $senha e $banco que são a senha do banco de dados, que, no caso, está em branco e o nome do banco de dados criado. Importante agora é a variável $db, que recebe como atribuição o comando mysql_connect, responsável por receber como parâmetros as informações referentes ao banco de dados e conectar o PHP ao banco escolhido. Os três parâmetros que esse comando recebe são o servidor, o usuário do banco de dados e a senha. Por fim, o comando mysql_select_db, responsável pela seleção do banco de dados, recebe como parâmetros a variável $banco com o nome do banco de dados que será usado e a variável $db com as informações sobre a conexão com o banco de dados.

3. Cadastrando dados

Após criarmos o arquivo de conexão, o próximo passo será a criação de uma página de cadastro, a qual fará uso desse arquivo para conexão com o banco de dados e, por consequência, usará as tabelas desse banco. O arquivo a seguir deverá ser salvo com o nome de cadastro.html e nele teremos uma tabela que organiza dois rótulos e duas caixas de texto dentro de uma *tag* form, a qual permite a criação de um formulário em html.

O mais importante é o arquivo passado dentro dos atributos *action* e *post* da *tag* form, em que o primeiro chama-se arquivo inserir.php e o segundo método de passagem das informações para o servidor WEB, que, no caso, está sendo usado o POST.

```
<html>
<head>
    <title>Cadastro de usuário</title>
</head>
<body>
    <form action="inserir.php" method="POST" name="Cadastro" id="Cadastro">
    <table width="450" border="0" cellspacing="0" cellpadding="3">
    <tr>
        <td width="111"><div align="right">Nome:</div></td>
        <td width="339"><input name="nome" type="text" id="nome" size="30"></td>
    </tr>
    <tr>
        <td width="111"><div align="right">Email:</div></td>
        <td width="339"><input name="email" type="text" id="nome" size="30"></td>
    </tr>
    <tr>
        <td height="29" colspan="2" valign="top"><div align="center">
            <input type="submit" name="enviar" value="GRAVAR">
        </div>
        </td>
    </tr>
    </table>
    </form>
</body>
</html>
```

```
<html>
<head>
    <title>Inserindo os dados</title>
</head>
<body>
<?php
    require("dbcon.php");
    $nome = $_POST["nome"];
    $email = $_POST["email"];

    $sqlinsert = "INSERT INTO cadnomes (cad_nome,cad_email) VALUES ('$nome', '$email')";
    mysql_query($sqlinsert) or die ("Não foi possivel inserir no banco de dados!");
?>
</body>
</html>
```

No arquivo anterior de nome inserir.php, podemos notar novos comandos, por exemplo, o *require*, cuja função é incluir um determinado arquivo no código que está sendo desenvolvido, como, no caso, fazer a inclusão do arquivo de conexão com o banco de dados. O $_POST que tem o objetivo de receber os dados vindos do formulário de cadastro via POST, sem passar as informações na URL do navegador. Outra função de grande importância é a mysql_query, a qual envia o resultado de uma declaração SQL para o servidor de banco de dados, no caso, essa declaração está na variável $sqlinsert (OLIVIERO, 2010).

4. Listando os dados do banco de dados

Após inserir os dados, normalmente um suposto usuário desejará recuperar as informações cadastradas e, em outros casos, poderá editar esses dados por ter tido alteração em um cadastro de um usuário ou de uma empresa. Dessa forma, podemos usar os comandos do PHP para manipular esses dados dentro do banco de dados MySQL. Isso será mostrado nos códigos a seguir. O primeiro código deverá ser gravado com o nome de exibir.php e nele podemos notar algumas informações, como o uso do comando de inserção de arquivos externos, o require, e a criação de uma variável $sql que receberá a instrução SQL para consultar os dados no banco de dados, a qual é a instrução Select.

A variável $conteudo recebe o resultado do Select executado pelo comando mysql_query. Em seguida, ela é usada no comando mysql_fetch_array dentro do loop while, no qual a intenção é varrer o banco de dados e retornar os dados, armazenando-os dentro das variáveis e exibindo no comando echo após o loop.

```php
<html>
<head>
    <title>Inserindo os dados</title>
</head>
<body>
<?php
    require("dbcon.php");

    $sql = "SELECT * FROM cadnomes";

    $conteudo = mysql_query("$sql");

    while($sql = mysql_fetch_array($conteudo))
    {
        $cad_id =    $sql["cad_id"];
        $cad_nome =  $sql["cad_nome"];
        $cad_email = $sql["cad_email"];
    }

    echo "Usuário ID = $cad_id<br>Nome: $cad_nome<br>Email: $cad_email";
?>
</body>
</html>
```

No próximo tema, será abordado um sistema completo com todas as funções de manipulação em banco de dados, como inserção, consulta, edição e exclusão de dados.

Glossário

phpMyAdmin: aplicativo de gerenciamento de banco de dados, nativo do pacote PHP para o gerenciamento de todos os bancos de dados criados dentro de um servidor MySQL.

Endereço de *loopback*: endereço de IP local para que se possa simular uma rede de computador localmente em um computador ou servidor, utilizado, geralmente, para testes de desenvolvimento de sistemas WEB.

Método POST: forma de passagem de dados em uma rede cliente/servidor, em que os dados são passados por meio das variáveis sem a necessidade de mostrá-los na URL, ou seja, em casos de sistemas com dados sigilosos, este método é o mais recomendável.

Questão para reflexão



Neste tema, vimos como criar um pequeno sistema para trabalhar com o banco de dados MySQL e a linguagem PHP. Esse exemplo foi criado de maneira simples e didática para mostrar os comandos, como o mysql_fetch_array e o mysql_query. Deixo a seguinte questão para refletir: existem apenas esses dois comandos de manipulação de dados do PHP para MySQL?

Considerações Finais

Nesta seção, aprendemos a criar um sistema que integra um banco de dados criado no MySQL. Assim, devemos ter absorvido os seguintes conhecimentos:

- Criação de um banco de dados no Servidor MySQL.
- Como desenvolver um arquivo de conexão com o banco de dados.
- Como inserir o arquivo de conexão em um código PHP.
- Como manipular os dados usando os comandos para inserir e para consultar os dados no banco de dados.
- Conhecer os comandos require, mysql_connect, mysql_fetch_array.

Referências

OLIVIERO, Carlos A. J. **Faça um site:** PHP 5.2 com MySQL 5.0: Comércio Eletrônico: orientado por projeto. 1. ed. São Paulo: Érica, 2010.

Assista a suas aulas



Aula 5 - Tema: Banco de Dados com PHP - Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/0c283e57ac7de98d406e71c0d38f8c4a>.



Aula 5 - Tema: Banco de Dados com PHP - Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/004f59da47701ded5caf4286e32a4e80>.

Questão 1

1. Qual é o objetivo da função *require*?

a) Permitir a inclusão de arquivos textos dentro de um código PHP.

b) Permitir a inclusão de arquivos externos dentro de um código PHP.

c) Permitir que os arquivos não sejam incluídos em um código PHP.

d) Permitir a inclusão de arquivos internos dentro de um código PHP.

e) Permitir a inclusão de arquivos externos fora de um código PHP.

Questão 2

2. Qual é a função do atributo *action* da *tag form* no programa HTML?

a) Este atributo armazena o endereço de uma página de internet em que a ação deve ser executada.

b) Este atributo armazena o endereço de um arquivo executável em que a ação deve ser executada.

c) Este atributo armazena o endereço de um arquivo em que uma ação não deve ser executada.

d) Este atributo armazena o endereço do arquivo em que uma ação deve ser executada.

e) Este atributo armazena o endereço de um arquivo de texto em que uma ação deve ser executada.

Questão 3

3. Qual é a função do phpMyAdmin?

a) É uma aplicação para gerenciamento do banco de dados MySQL pago para PHP.

b) É uma aplicação para gerenciamento do banco de dados Oracle feito em PHP.

c) É uma aplicação para gerenciamento do banco de dados MySQL não disponível para PHP.

d) É uma aplicação para gerenciamento de um banco de dados qualquer feita em PHP

e) É uma aplicação para gerenciamento do banco de dados MySQL feita em PHP.

Questão 4

4. Tratando-se do código dbcon.php, qual é o objetivo da função mysql_connect()?

a) Permitir a conexão com o banco de dados sem a passagem de parâmetros.

b) Permitir a conexão com o banco de dados sem se importar com quais parâmetros devem ser passados.

c) Permitir a conexão com o banco de dados recebendo o servidor, o usuário e a senha do banco como parâmetros.

d) Permitir a conexão com o banco de dados que só recebe o usuário como parâmetro.

e) Permitir a conexão com o banco de dados que só recebe a senha como parâmetro.

Questão 5

5. A função mysql_query envia:

a) Uma pesquisa para o banco de dados a partir de uma linha SQL.

b) Uma consulta para o banco de dados a partir de um comando MySQL.

c) Uma listagem para o banco de dados a partir de uma instrução SQL.

d) Uma consulta para o banco de dados a partir de uma instrução SQL.

e) Uma pesquisa para o banco de dados a partir de uma instrução SQL.

Gabarito

1. Resposta: B.

A função *require* é usada para incluir arquivos que são criados externamente para que possamos fazer uma organização dos códigos desenvolvidos em um programa, por exemplo, arquivos de conexão com banco de dados, arquivos de inclusão de imagens ou arquivos de inclusão de textos que gastariam muito tempo e programação para digitar em todas as páginas que eles seriam necessários. Outras linguagens de programação também têm funções com o mesmo objetivo.

2. Resposta: D.

Entende-se por endereço o local em que o arquivo passado no atributo está gravado, ou seja, no caso do action, para o arquivo inserir.php, o atributo tem o endereço ou a URL deste arquivo. Dessa forma, os dados são passados sem erros.

3. Resposta: E.

Aplicativo WEB desenvolvido em PHP que permite o gerenciamento de banco de dados MySQL criados em PHP. Este foi criado em 1998 e passou a ter esse nome a partir de 2001. Sua vantagem mais importante é a interface gráfica que tem as mesmas funções do ambiente de texto para gerenciar bancos de dados MySQL.

4. Resposta: C.

Função responsável por conectar, por meio dos parâmetros recebidos, uma aplicação WEB a um banco de dados PHP. Pode ser programada em cada programa PHP individualmente, ou programado em um arquivo de conexão externo.

5. Resposta: D.

Quando usamos esta função, normalmente ela envia uma operação realizada para o servidor de banco de dados, podendo ser de inclusão, consulta, alteração ou exclusão de dados.

Unidade 6

Desenvolvimento de Sites em PHP, Desenvolver e Manter Aplicações Dinâmicas para Internet com Interface Web, Criando Sites Dinâmicos com a Linguagem PHP

Objetivos

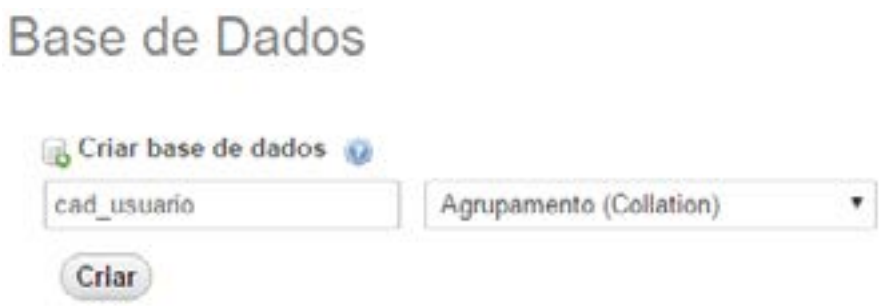
1. Este tema tem como objetivo apresentar uma aplicação WEB desenvolvida em PHP, como conexão a um banco de dados em MySQL. Pretende-se mostrar ao aluno como criar esse sistema e suas telas mais utilizadas, como cadastro, pesquisa e listagem. A cada novo comando ou função apresentada, será feita uma explicação sobre seu funcionamento e, caso seja necessário, sua sintaxe será mostrada com todos seus parâmetros.

Introdução

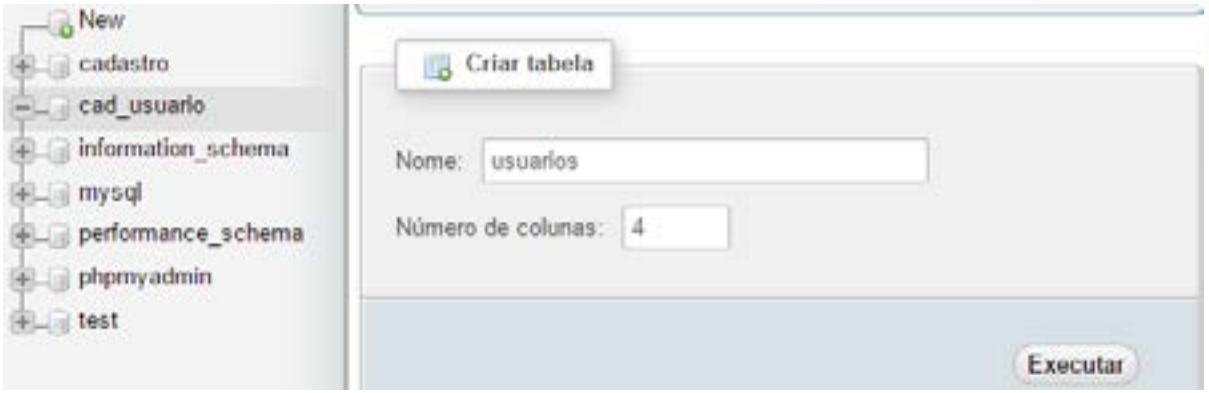
A partir de agora será apresentado o desenvolvimento de um sistema desenvolvido em PHP, para mostrar como isso pode ser feito na prática conectando esse sistema a um banco de dados desenvolvido em MySQL, em que será mostrado somente o diagrama do banco, a parte de desenvolvimento ficará por conta do usuário, como executado no tema anterior. Em desenvolvimento de sistemas, usamos o acrônimo CRUD para designar um sistema com as operações de *Create* (Criação), *Retrieve* (Consulta), *Update* (Atualização) e *Delete* (Exclusão) para os dados de um sistema, ou seja, todas as funções de manipulação de dados em uma aplicação. Assim, no exemplo a seguir, será construído um sistema com todas as quatro operações de manipulação de dados em PHP para MySQL (WAZLAWICK, 2014).

1. O banco de dados

Primeiramente, será feita a criação do banco de dados para que o sistema funcione. O banco de dados será chamado de cad_usuario e terá a seguinte configuração:



Tela de criação do banco de dados, note que foi inserido o nome do banco de dados e seu agrupamento. Após isso, deve-se clicar no botão Criar. Em seguida, aparecerá a tela que informa a criação da tabela do banco de dados, na qual devemos inserir o nome da tabela, no caso, usuários, e o número de colunas, clicando no botão executar.



Após criada a tabela, as posições para a criação dos quatro campos que nosso sistema terá aparecem na seguinte tela, na qual necessitamos configurar conforme mostrado a seguir, ou seja, criando os campos id_usuario, primeiro_nome, ultimo_nome e cidade_usuario. O sistema é bem simples, mas o objetivo aqui é mostrar para o aluno como usar os conceitos de CRUD em um sistema PHP.

#	Nome	Tipo	Agrupamento (Collation)	Atributos	Nulo	Predefinido	Extra
1	id_usuario	int(10)			Não	None	AUTO_INCREMENT
2	primeiro_nome	varchar(30)			Não	None	
3	ultimo_nome	varchar(30)			Não	None	
4	cidade_usuario	varchar(50)			Não	None	

Tudo pronto, agora, o próximo passo será a criação das telas de cadastro e consulta e os *scripts* de manipulação de dados em PHP.

2. Telas de cadastro

A primeira coisa que faremos é a criação do arquivo php para a conexão com o banco de dados, o qual terá o nome de dbconfig.php, apresentado a seguir:

```php
<?php
    $nome_host = "localhost";
    $nome_usuario = "root";
    $senha = "";
    $banco = "cad_usuario";
    $db = mysql_connect($nome_host, $nome_usuario, $senha);
    mysql_select_db($banco, $db);
?>
```

O dbconfig em suas linhas 6 e 7 mostra os comandos para conexão com o banco de dados MySQL. Na linha 6, é exibido o comando mysql_connect que recebe os parâmetros do banco de dados: nome do servidor (no caso, localhost ou 127.0.0.1), nome do usuário do banco de dados e a senha do banco de dados.

Já na linha 7, o comando mysql_select_db permite a seleção do banco de dados passando os dados de conexão e ainda o nome do banco de dados armazenado na variável $db.



```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>Sistema em PHP and MySql - Cadastro</title>
<link rel="stylesheet" href="style.css" type="text/css" />
</head>
<body>
<center>
<div id="header">
    <div id="content">
        <label>Cadastro de Usuários</label>
    </div>
</div>
<div id="body">
  <div id="content">
    <form action="insert.php" method="POST" >
    <table align="center">
    <tr> <td align="center"><a href="index.php">Voltar para a Página Principal</a></td> </tr>
    <tr> <td><input type="text" name="primeiro_nome" placeholder="Primeiro Nome" requerido /></td> </tr>
    <tr> <td><input type="text" name="ultimo_nome" placeholder="Último Nome" requerido /></td> </tr>
    <tr> <td><input type="text" name="nome_cidade" placeholder="Cidade" requerido /></td> </tr>
    <tr> <td><button type="submit" name="btn-salvar"><strong>Salvar</strong></button></td> </tr>
    </table>
    </form>
    </div>
</div>
</center>
</body>
```

O segundo arquivo a ser editado fará referência à tela inicial do sistema que tem como objetivo listar os registros já cadastrados, permitindo, assim, a inclusão dos dados, por meio de um *link* acima da tabela, e também a edição e exclusão dos dados, por meio de dois botões posicionados ao lado de cada registro do banco de dados. Esta tela tem o nome de inicio.php (aqui foi trocada a extensão para mostrar que podemos usar para as telas de cadastro tanto a extensão html quanto a extensão php) que terá seu código apresentado a seguir.

No início temos que observar a inclusão do arquivo de conexão com o banco de dados, além do comando *if* em PHP que permite a exclusão de dados do sistema, desde que seja clicado no ícone de exclusão da tabela inicial. Este comando verifica se o valor da variável *delete_id* é passado como parâmetro para a instrução SQL, a DELETE, responsável por excluir os dados. Caso seja igual ao que está cadastrado no sistema, o registro será excluído e, com o comando *header*, ele será redirecionado para a própria página de Listagem (inicio.php).

```php
<?php
include_once "dbconfig.php";
// delete condition
if(isset($_GET["delete_id"]))
{
    $sql_query="DELETE FROM usuarios WHERE id_usuario=".$_GET["delete_id"];
    mysql_query($sql_query);
    header("Location: $_SERVER[PHP_SELF]");
}
?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>Sistema em PHP and MySql - Listagem</title>
<link rel="stylesheet" href="style.css" type="text/css" />
<script type="text/javascript">
function edt_id(id)
{
    if(confirm('Tem certeza que quer editar ?'))
    {
       window.location.href='editar_dados.php?edit_id='+id;
    }
}
function delete_id(id)
{
    if(confirm('Tem certeza que quer EXCLUIR ?'))
    {
       window.location.href='inicio.php?delete_id='+id;
    }
}
</script>
</head>
<body>
```

```
<center>
<div id="body">
 <div id="content">
    <table align="center">
    <tr>
    <th colspan="5"><a href="Cadastro_usuario.html">Cadastrar Usuário</a></th>
    </tr>
    <th>Primeiro Nome</th>
    <th>Último Nome</th>
    <th>Cidade</th>
    <th colspan="2">Operações</th>
    </tr>
    <?php
 $sql_query="SELECT * FROM usuarios";
 $result_set=mysql_query($sql_query);
 while($row=mysql_fetch_row($result_set))
 {
  ?>
        <tr>
        <td><?php echo $row[1]; ?></td>
        <td><?php echo $row[2]; ?></td>
        <td><?php echo $row[3]; ?></td>
        <td align="center"><a href="javascript:edt_id('<?php echo $row[0]; ?>')">
            <img src="edit_blue.png" align="EDIT" /></a></td>
        <td align="center"><a href="javascript:delete_id('<?php echo $row[0]; ?>')">
            <img src="delete_blue.png" align="DELETE" /></a></td>
        </tr>
        <?php
 }
 ?>
    </table>
```

A partir da linha 49 é feito o processo de consulta ao banco de dados até que este tenha registros para serem exibidos. O comando responsável pela consulta é o mysql_query, o qual recebe como parâmetro uma variável que tem o comando SQL com o SELECT na tabela desejada. Após isso, o comando mysql_fetch_row buscará o resultado relacionado com a chave procurada e retornará um vetor na variável $row, com os dados dessa linha, então, dentro do comando while. Estes dados serão exibidos na tela do navegador em formato de tabela (PHP, 2016).

As colunas que têm a *tag* img carregam duas imagens para indicar as operações de edição e exclusão dos dados da tabela do banco de dados.

Caso seja clicado no *link* para cadastrar um usuário, será redirecionado para a página Cadastro_usuario.html, que exibe um formulário de cadastro e redireciona (atributo *action*) para o arquivo *insert.php*, responsável por receber os dados digitados nos campos e inseri-los no banco de dados. A seguir temos o código do arquivo Cadastro_usuario.html.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<title>Sistema em PHP and MySql - Cadastro</title>
<link rel="stylesheet" href="style.css" type="text/css" />
</head>
<body>
<center>
<div id="header">
    <div id="content">
        <label>Cadastro de Usuários</label>
    </div>
</div>
<div id="body">
 <div id="content">
    <form action="insert.php" method="POST" >
    <table align="center">
    <tr> <td align="center"><a href="inicio.php">Voltar para a Página Principal</a></td> </tr>
    <tr> <td><input type="text" name="primeiro_nome" placeholder="Primeiro Nome" requirido /></td> </tr>
    <tr> <td><input type="text" name="ultimo_nome" placeholder="Último Nome" requerido /></td> </tr>
    <tr> <td><input type="text" name="nome_cidade" placeholder="Cidade" requerido /></td> </tr>
    <tr> <td><button type="submit" name="btn-salvar"><strong>Salvar</strong></button></td> </tr>
    </table>
    </form>
    </div>
</div>
</center>
</body>
```

Quando o usuário clicar no botão Salvar, os dados serão enviados via método POST para o arquivo *insert.php*, o qual é apresentado a seguir:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
    <title>Inserindo os dados</title>
    <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
    <link rel="stylesheet" href="style.css" type="text/css" />
</head>
<body>
<?php
    require("dbconfig.php");
    if(isset($_POST['btn-salvar'])) {
        // variaveis de entrada de dados
        $primeiro_nome = $_POST["primeiro_nome"];
        $ultimo_nome = $_POST["ultimo_nome"];
        $nome_cidade = $_POST["nome_cidade"];

        // sql query usada para inserir os dados no banco de dados

        $sql_query = "INSERT INTO usuarios (primeiro_nome,ultimo_nome,cidade_usuario) VALUES
        ('$primeiro_nome','$ultimo_nome','$nome_cidade')";
        mysql_query($sql_query);
    }
?>
<table align="center">
<tr>
    <td align="center">Usuário Cadastrado com Sucesso!</td>
</tr>
<tr>
    <td align="center"><a href="Cadastro_usuario.html">Voltar para a Página de Cadastro</a></td>
</tr>
</table>
</body>
</html>
```

Como podemos notar, sempre devemos inserir o arquivo de conexão com o banco de dados. Após isso, é necessário verificar se o botão Salvar foi clicado, recebendo os valores que foram passados via POST para as variáveis. Assim que estes valores são armazenados, a variável $sql_query receberá o comando INSERT, os campos do banco de dados e os valores que serão gravados nesses campos. Em seguida, a função mysql_query faz realmente a inserção dos dados no banco. O usuário receberá uma mensagem de confirmação dos dados e um *link* para retornar à página de cadastro.

Voltando à página inicial do sistema, caso o usuário clique no ícone de edição, será passado o *id* da coluna para uma função *JavaScript* chamada *edt_id*, que recebe como parâmetro este *id* e o envia para a página *editar_dados.php* dentro da variável *edit_id*.

Então, a página com o formulário de edição, muito semelhante ao de cadastro, é carregada com os dados referentes ao registro selecionado prontos para serem editados. Depois de feita a alteração, o usuário deverá clicar no botão Atualizar para que os dados sejam atualizados no banco de dados. A seguir, temos o código da tela de edição de dados, que é um pouco extenso pelo fato de que podemos colocar o código PHP juntamente com o HTML dentro do mesmo arquivo. Assim, quando a execução é simples, fica fácil de gerenciar qualquer mudança, mas quando se

necessita de que esse código PHP apareça em vários outros arquivos, é necessária a criação de um arquivo de redirecionamento por meio do atributo *action* da *tag* form.

Neste arquivo, no início temos vários pontos a serem tratados, como a inclusão do arquivo de configuração do banco de dados novamente, o primeiro comando *if* que faz referência a uma consulta ao banco de dados com o índice do registro selecionado com a variável *edit_id*, ou seja, no código anterior existe a função de que no momento em que o usuário clicar no ícone de edição, ele alimenta a função javascript chamada *edt_id*, que passa como parâmetro o valor do *id* do registro a ser alterado e carrega a variável $sql_query passada como parâmetro para a função mysql_query, a qual gerará o vetor com os dados do registro pesquisado com a função mysql_fetch_array, colocando os dados dentro da variável $fetched_row, isto é, a linha do registro encontrada.

O segundo comando *if* está relacionado com o botão atualizar que recebe os dados do formulário (em tese com valores diferentes do que estavam no banco de dados), passando para as variáveis nas linhas 12 a 14. Eles são atualizados usando o comando SQL UPDATE que está configurado na linha 17. Depois de

atualizados, as funções *Javascript* mostrarão uma caixa de mensagem avisando se a atualização ocorreu corretamente ou não.

```php
<?php
include_once 'dbconfig.php';
if(isset($_GET['edit_id']))
{
     $sql_query="SELECT * FROM usuarios WHERE id_usuario=".$_GET['edit_id'];
     $result_set=mysql_query($sql_query);
     $fetched_row=mysql_fetch_array($result_set);
}
if(isset($_POST['btn-update']))
{
 // variaveis para entrada de dados
 $primeiro_nome = $_POST['primeiro_nome'];
 $ultimo_nome = $_POST['ultimo_nome'];
 $nome_cidade = $_POST['nome_cidade'];

 // comando sql para atualizar dados no banco de dados
 $sql_query = "UPDATE usuarios SET primeiro_nome='$primeiro_nome',ultimo_nome='$ultimo_nome',cidade_usuario='$nome_cidade'
 WHERE id_usuario=".$_GET['edit_id'];
```

```
19
20      // função para execução de uma consulta Sql
21      if(mysql_query($sql_query))
22      {
23       ?>
24       <script type="text/javascript">
25       alert('Dados atualizados com Sucesso!!!');
26       window.location.href='inicio.php';
27       </script>
28       <?php
29      }
30      else
31      {
32       ?>
33       <script type="text/javascript">
34       alert('Um erro ocorreu na atualização dos dados...');
35       </script>
36       <?php
37      }
38      // sql query execution function
39  }
40  if(isset($_POST['btn-cancel']))
41  {
42   header("Location: inicio.php");
43  }
44  ?>
45  <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
46  <html xmlns="http://www.w3.org/1999/xhtml">
47  <head>
48  <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
49  <title>Atualização de dados</title>
50  <link rel="stylesheet" href="style.css" type="text/css" />
51  </head>
52  <body>
53  <center>
```

```
<div id="header">
 <div id="content">
    <label>Atualização de dados</label>
    </div>
</div>

<div id="body">
 <div id="content">
    <form method="post">
    <table align="center">
    <tr>
    <td><input type="text" name="primeiro_nome" placeholder="Primeiro nome" value="<?php echo $fetched_row['primeiro_nome']; ?>" requerido /></td>
    </tr>
    <tr>
    <td><input type="text" name="ultimo_nome" placeholder="Último nome" value="<?php echo $fetched_row['ultimo_nome']; ?>" requerido /></td>
    </tr>
    <tr>
    <td><input type="text" name="nome_cidade" placeholder="Cidade" value="<?php echo $fetched_row['cidade_usuario']; ?>" requerido /></td>
    </tr>
    <tr>
    <td>
    <button type="submit" name="btn-update"><strong>Atualizar</strong></button>
    <button type="submit" name="btn-cancel"><strong>Cancelar</strong></button>
    </td>
    </tr>
    </table>
    </form>
    </div>
</div>

</center>
</body>
</html>
```

Assim, concluímos nosso sistema simplificado de cadastro com todas as funções de CRUD necessárias em qualquer sistema para que ele seja funcional. A necessidade de um banco de dados e as funções para manipulação dos dados sempre devem ser desenvolvidas. Outro

ponto importante é que devemos ter um código legível e com comentários para que se possa entender o que foi programado futuramente.

Os próximos temas abordarão tecnologias recentes em relação à programação PHP.

Glossário

CRUD: acrônimo de *Create, Retrieve, Update* e *Delete*, funções de manipulação em sistemas que têm integração com um banco de dados. Na prática são funções de criação, recuperação, alteração e exclusão de dados por meio de uma interface com o usuário.

SQL: acrônimo de *Structured Query Language* ou Linguagem de Pesquisa Estruturada, que permite a criação de comandos de pesquisa de dados em banco de dados relacionais, como o MySQL, SQL Server, entre outros.

Comando UPDATE: comando SQL que permite a atualização dos dados passados via formulário e armazenados em variáveis para atualizar todos ou alguns campos do banco de dados.

Comando INSERT: comando SQL que permite a inserção de dados passados via formulário para serem inseridos nos campos de uma tabela do banco de dados.

Comando DELETE: comando SQL que permite a exclusão de dados de um registro selecionado com base em uma consulta a uma tabela do banco de dados.

Comando SELECT: comando SQL que permite a consulta de dados de um registro cadastrado em uma tabela do banco de dados.

Questão para reflexão



Neste tema, foi apresentado um sistema de cadastro com integração com banco de dados. Note que, no banco de dados, criamos apenas uma tabela para facilitar o aprendizado. A proposta para reflexão aqui será com relação ao processo de criação de um sistema com mais de uma tabela, ou seja, pensando em algo mais complexo, o sistema deverá ter um banco de dados e, dentro deste banco, a implementação de mais de uma tabela e, consequentemente, mais de uma tela de cadastro. Como deveria ser um projeto desse?

Considerações Finais

- Projeto de criação de um banco de dados e sua tabela com seus atributos.
- Elaboração das telas de comunicação no sistema, como as telas de cadastro e consulta dos dados.
- Elaboração dos códigos em PHP para que essas telas sejam funcionais, ou seja, permitam a interação com o banco de dados.
- Criação de códigos PHP para serem incluídos nos arquivos que têm a lógica de inclusão, exclusão, alteração e consulta de dados com base nas telas de interação criadas.

Referências

Exemplo:

PHP. Website Oficial da linguagem PHP. Disponível em: <php.net/manual/pt_BR/function.mysql-fetch-row.php>. Acesso em: 15 maio 2016.

WAZLAWICK, Raul Sidnei. **Análise e design orientados a objetos para sistemas de informação:** modelagem com UML, OCL e IFML. 3. ed. Rio de Janeiro: Editora Elsevier, 2014. 488 p.

Assista a suas aulas



Aula 6 - Tema: Desenvolvimento de Sites em Php, desenvolver e manter aplicações dinâmicas para Internet com interface Web, Criando Sites Dinâmicos com a Linguagem PHP - Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f-1d/033b7a67ad56c35a835636bf0ac6aa20>.



Aula 6 - Tema: Desenvolvimento de Sites em Php, desenvolver e manter aplicações dinâmicas para Internet com interface Web, Criando Sites Dinâmicos com a Linguagem PHP - Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/b9302b3bbe64263a6bef4074422f728d>.

Questão 1

1. Qual é a finalidade de criar um sistema com a funcionalidade de CRUD?

a) Ter todas as funções necessárias para um sistema único.

b) Ter quase todas as funções necessárias para a manipulação de dados do sistema.

c) Não usar as funções necessárias para manipulação de dados do sistema.

d) Ter todas as funções necessárias para manipulação de dados do sistema.

e) Ter todas as funções necessárias para inserção de dados do sistema.

Questão 2

2. Qual é a função do comando *require* no código PHP?

a) O comando permite a exclusão de um arquivo externo no código.

b) O comando permite a inclusão de um arquivo externo no código.

c) O comando permite que o código seja criado no PHP.

d) O comando não permite a inclusão de um arquivo externo no código.

e) O comando permite que o código seja alterado.

Questão 3

3. Qual é o significado do acrônimo CRUD para um sistema WEB?

a) Significa *Cancel, Retrieve, Upgrade* e *Delete*.

b) Significa *Create, Release, Update* e *Drop*.

c) Significa *Cancel, Release, Upgrade* e *Drop*.

d) Significa *Consult, Release, Update* e *Delete*.

e) Significa *Create, Retrieve, Update* e *Delete*.

Questão 4

4. O que significa o comando: $primeiro_nome = $_POST['primeiro_nome'];

a) O dado é passado da internet primeiro_nome via POST para a variável $primeiro_nome.

b) O dado é passado do campo primeiro_nome via POST para a variável $primeiro_nome.

c) O dado é passado do campo primeiro_nome via GET para a variável $primeiro_nome.

d) O dado é passado do campo primeiro_nome via POST para o campo primeiro_nome.

e) O dado não é passado do campo primeiro_nome via POST para a variável $primeiro_nome.

Questão 5

5. Quais são os comandos SQL referentes ao conceito de CRUD?

a) UPDATE, SELECT, INSERT e DELETE.

b) SELECT, INSERT, DELETE e UPDATE.

c) INSERT, SELECT, UPDATE e DELETE.

d) INSERT, DELETE, UPDATE e SELECT.

e) SELECT, UPDATE, INSERT e DELETE.

Gabarito

1. Resposta: D.

Entende-se como funções básicas de manipulação a capacidade que o usuário tem em poder fazer a entrada de dados no sistema, alterar os dados, exclui-los e consultá-los no momento em que for necessário.

2. Resposta: B.

Require é uma função PHP que permite a inclusão de arquivos externos dentro de outros arquivos em PHP, normalmente utilizado para inclusão de arquivo de conexão com banco de dados e também de imagens para cabeçalhos e rodapés de páginas WEB.

3. Resposta: E.

Create para inserir dados no banco de dados, ***Retrieve*** para consulta dos dados gravados, ***Update*** para atualização dos dados gravados e ***Delete*** para exclusão de algum dado gravado.

4. Resposta: B.

A variável $primeiro_nome receberá o valor digitado em um campo texto que tem como nome $primeiro_nome e que será passado via método POST, para não aparecer na URL da página.

5. Resposta: C.

A sequência é definida na descrição deste acrônimo, ou seja, inclusão, consulta, alteração e exclusão de dados que os comandos SQL executam.

Unidade 7

Sistemas Open-Source em PHP, Criação e Apresentação

Objetivos

1. Neste tema, falaremos de uma aplicação open-source, ou seja, código aberto, que foi desenvolvida em PHP e que usa como banco de dados o MySQL. Esta aplicação se chama WordPress, um gerenciador de conteúdo que permite a criação de *blogs* profissionais para empresas e para uso pessoal.
2. O objetivo será fazer uma breve explicação dos conceitos relacionados a esse tema e, depois disso, fazer um laboratório para a instalação, configuração e criação de uma página no WordPress no servidor local. Portanto, essa é uma aplicação muito interessante, mas existem muitas outras disponíveis para *download* e instalação para qualquer tipo de uso e aplicabilidade.

Introdução

Atualmente, a internet baseia-se em sua grande parte na distribuição de conteúdo e informação a todos seus usuários que, a cada dia, ficam mais exigentes quanto ao tipo de veículo que eles acessam para obter essas informações. Por causa da grande quantidade de dispositivos, como computadores, notebooks, smartphones, tablets, não é qualquer tipo de página que atrairá a atenção dos usuários, e sim aplicações que tenham design bonito e que possam se apresentar em qualquer um dos itens anteriores. Portanto, o WordPress é uma ferramenta que permite isso, pois por ser open-source pode ser modificada e trabalhada conforme a necessidade do desenvolvedor para atender às especificações e aos desejos de seus clientes.

Segundo Hedengren (2012), o WordPress é mais que uma plataforma de publicação de blogs. Para ele, pode-se criar qualquer coisa usando o WordPress; tudo se a necessidade for sites que, além de serem flexíveis, consigam carregar o conteúdo de maneira rápida. Uma de suas vantagens, segundo o autor, é sua facilidade de desenvolvimento e sua grande comunidade ativa que poderá ajudá-lo quando tiver dificuldades.

Outra vantagem importante desta plataforma é que ela se baseia totalmente em linguagens, como o PHP, a HTML e a CSS, portanto, totalmente fiel à temática deste curso.

1. Conceitos básicos

Antes de partirmos para a instalação e configuração, nesta seção, vamos trabalhar os conceitos apresentados durante a introdução. Isso se torna necessário na medida em que nem todos os alunos e usuários de internet sabem exatamente o que são cada um dos conceitos ou apenas têm uma noção do que sejam.

O primeiro deles é o open-source que, segundo o website <opensource.org> (acesso em: 8 set. 2016), é não só o significado de acesso ao código fonte, mas também os conceitos de redistribuição gratuita, ou seja, a licença não requer *royalty* ou algum tipo de doação para sua venda.

Outro conceito é o código fonte, que prevê sua distribuição em si e também de forma compilada, ou seja, quando se deseja a distribuição por meio de formulário que permite a doação para cobrir custos de reprodução com download pela internet sem custos. (Disponível em: <https://opensource.org/>. Acesso em: 8 set. 2016)

O gerenciador de conteúdo também é um fator importante. Este é um sistema que, segundo Chagas et al. (2008), pode ser dividido em três áreas: criação de conteúdo, entrega de conteúdo e recuperação de conteúdo, as quais podem ser usadas tanto em meio eletrônico quanto impresso. Isso significa que se pode fazer um aplicativo de gerenciamento de conteúdo para atingir qualquer tipo

de segmento e não apenas para ser um blog, como meio de comunicação. Existem sistemas que com base em um banco de dados, exibem as informações para os usuários, ou seja, entregam o conteúdo, outros ainda podem fazer mais de uma função.

Quando se menciona sistema de gerenciamento de conteúdo, fala-se da sigla CMS, do inglês *Content Management System*, com surgimento na década de 90, a qual foi um grande passo na gestão dos conteúdos de websites das organizações. A partir desse momento, surgiu uma grande gama de aplicações com código livre e para uso gratuito (CHAGAS et al., 2008).

Os recursos que uma aplicação de CMS nos fornece partem da criação de conteúdo, seu gerenciamento, sua distribuição, publicação e recuperação, sendo que o gerenciamento deve ser definido sob o ponto de vista das atividades das pessoas e dos seus objetivos (CHAGAS et al., 2008).

No nosso caso, falaremos do WordPress, um gerenciador de conteúdo que, na grande maioria das vezes, está associado a um criador de blogs. Para os usuários, pode ser assim, mas para os desenvolvedores é uma ferramenta que atende a grande quantidade de recursos que os usuários têm acesso hoje em dia, já comentado anteriormente e, que em grande parte, ocorre por ser simples de configurar e desenvolver. Esta plataforma necessita apenas de conhecimento em PHP, como linguagem de programação, MySQL, que é

o banco de dados usado pelo WordPress, e HTML e CSS, que fazem referência à parte visual e de estilos das páginas a serem desenvolvidos, ou seja, temos uma ideia de acesso por qualquer dispositivo que seja necessário.

2. O ambiente WordPress

Para criar o ambiente WordPress, será necessário, primeiramente, fazer seu *download* no seguinte endereço: <https://br.wordpress.org/latest-pt_BR.zip> (acesso em: 8 set. 2016), ou na página oficial desta plataforma, pressionando o botão Baixar o WordPress 4.5.2.



Após clicar no botão, será necessário escolher uma local para baixar a aplicação. Escolha o que for melhor para seu gerenciamento. O próximo passo será a instalação que está no seguinte item.

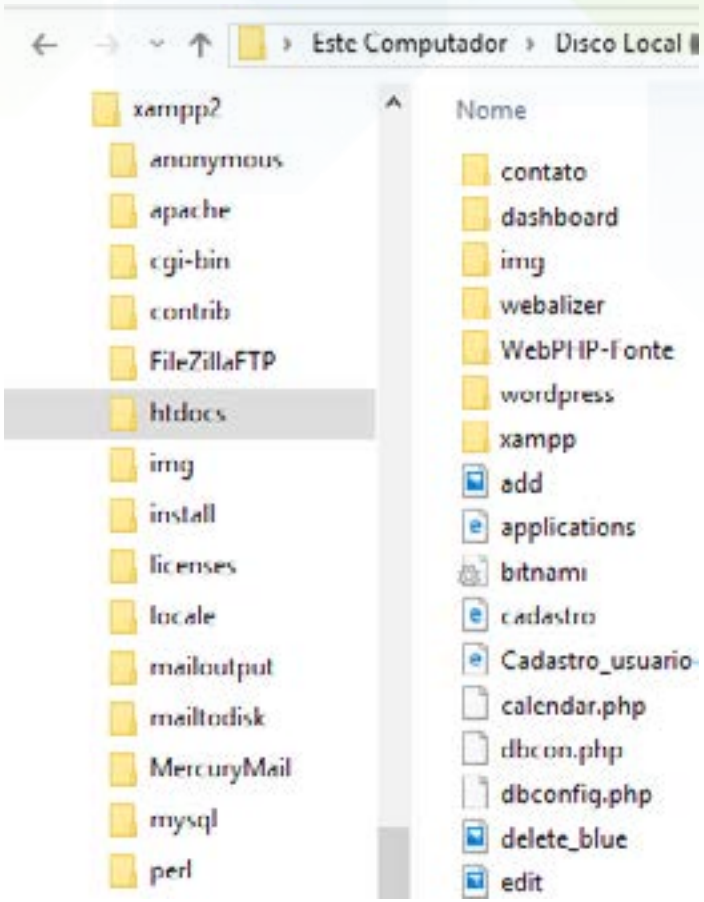
2.1. Instalando o WordPress

Depois de ser feito o *download* da aplicação, vamos para a parte da instalação. O WordPress, segundo Hedengren (2012), pode ser instalado de três formas: a instalação guiada, a instalação manual e a instalação em um clique. A instalação guiada é usada quando temos uma hospedagem e devemos fazer o *download* do WordPress, descompactar e fazer o *upload* dos arquivos para a pasta que deseja instalar apenas configurando os dados do servidor. Para a instalação manual, deve-se baixar todos os arquivos, como a versão mais recente do WordPress, o MySQL e um programa de FTP para *upload*. Depois de tudo instalado, deve-se fazer as configurações necessárias para, então, estar pronto para usar (veremos isso mais adiante). A última opção é a instalação com um clique, que deve ser feita quando a hospedagem fornece esse tipo de instalação (HEDENGREN, 2012).

Para uma instalação local, a fim de criarmos um ambiente de teste, usaremos a instalação manual que pode ser feita por meio dos passos a seguir.

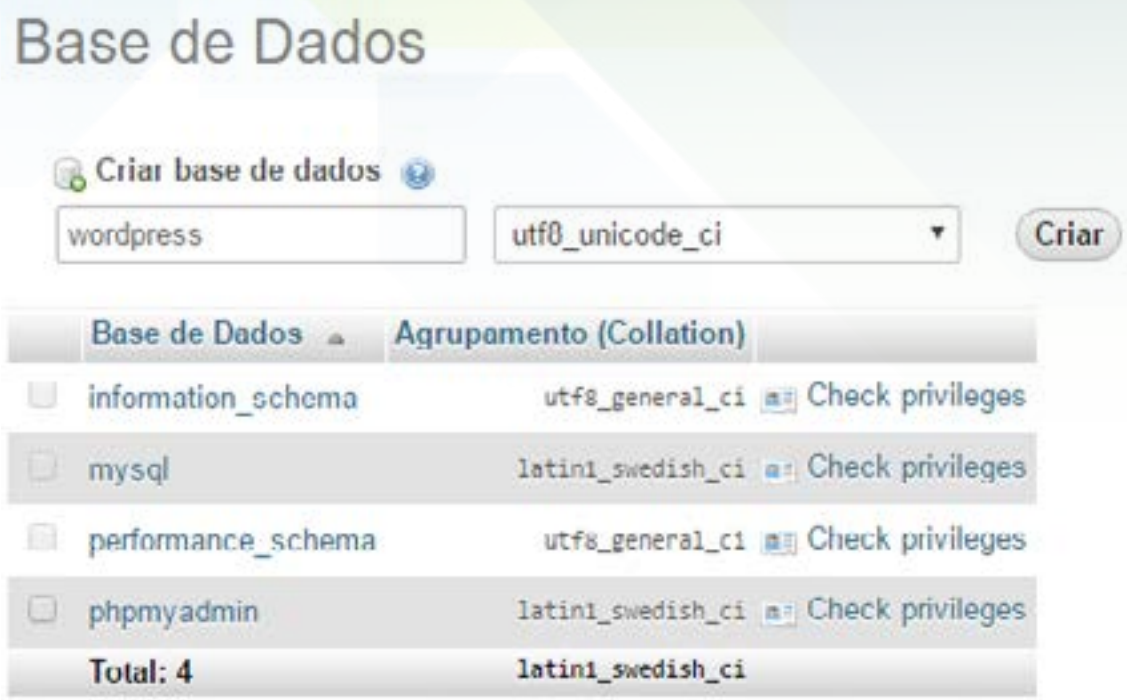
2.1.1. Instalando o WordPress - Configuração

Descompacte o arquivo .zip na pasta *htdocs* localizada no diretório do XAMPP. Assim, após descompactar, a pasta ficará dessa forma:

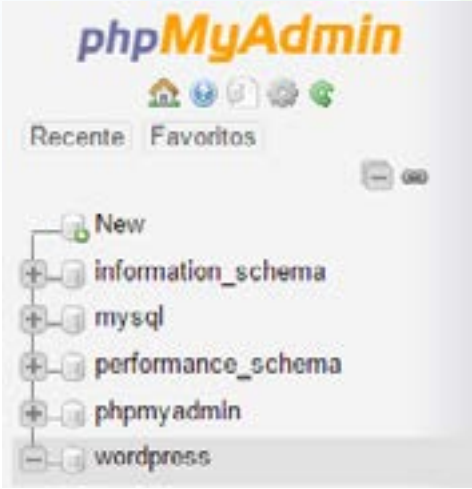


2.1.2. Acessando o phpMyAdmin e criando o banco de dados

Para que o WordPress funcione, devemos criar um banco de dados dentro do MySQL por meio do phpMyAdmin, já usado anteriormente. Assim, deve-se fazer a seguinte configuração:



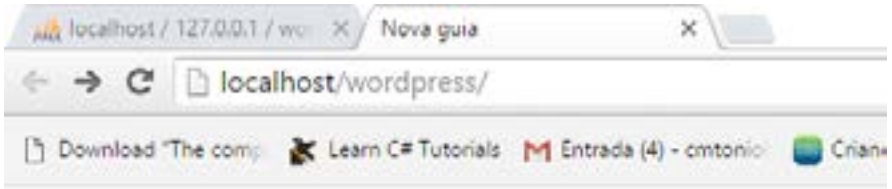
Note que o nome do banco de dados é “wordpress” e que foi selecionado o “Collation” (Verificação), utf8_unicode_ci. Após isso, clique em Criar. Se tudo der certo, a seguinte tela aparecerá:



Assim, o banco de dados “wordpress” será adicionado na lista de bancos de dados do MySQL.

2.1.3. Instalando o WordPress

Após o *download*, descompacte a pasta do WordPress dentro do XAMPP, abra o navegador e digite a seguinte URL: <http://localhost/wordpress/> ou <localhost/wordpress/>, com a qual você será redirecionado para a página de instalação:



Esse *link* deverá enviar para a página orientações de como inserir os dados para o instalador do WordPress e poder criar as tabelas para o banco de dados, como: nome do banco, usuário, senha, servidor e prefixo da tabela. O desenvolvedor deverá clicar no botão Vamos lá!, para prosseguir.



A próxima tela solicita as informações para os dados, como o nome do banco de dados, que, no caso, foi configurado como “wordpress”, nome do usuário, que é o mesmo usuário do MySQL, ou seja, “root”. A senha foi configurada em branco, pois foi definida dessa forma no banco de

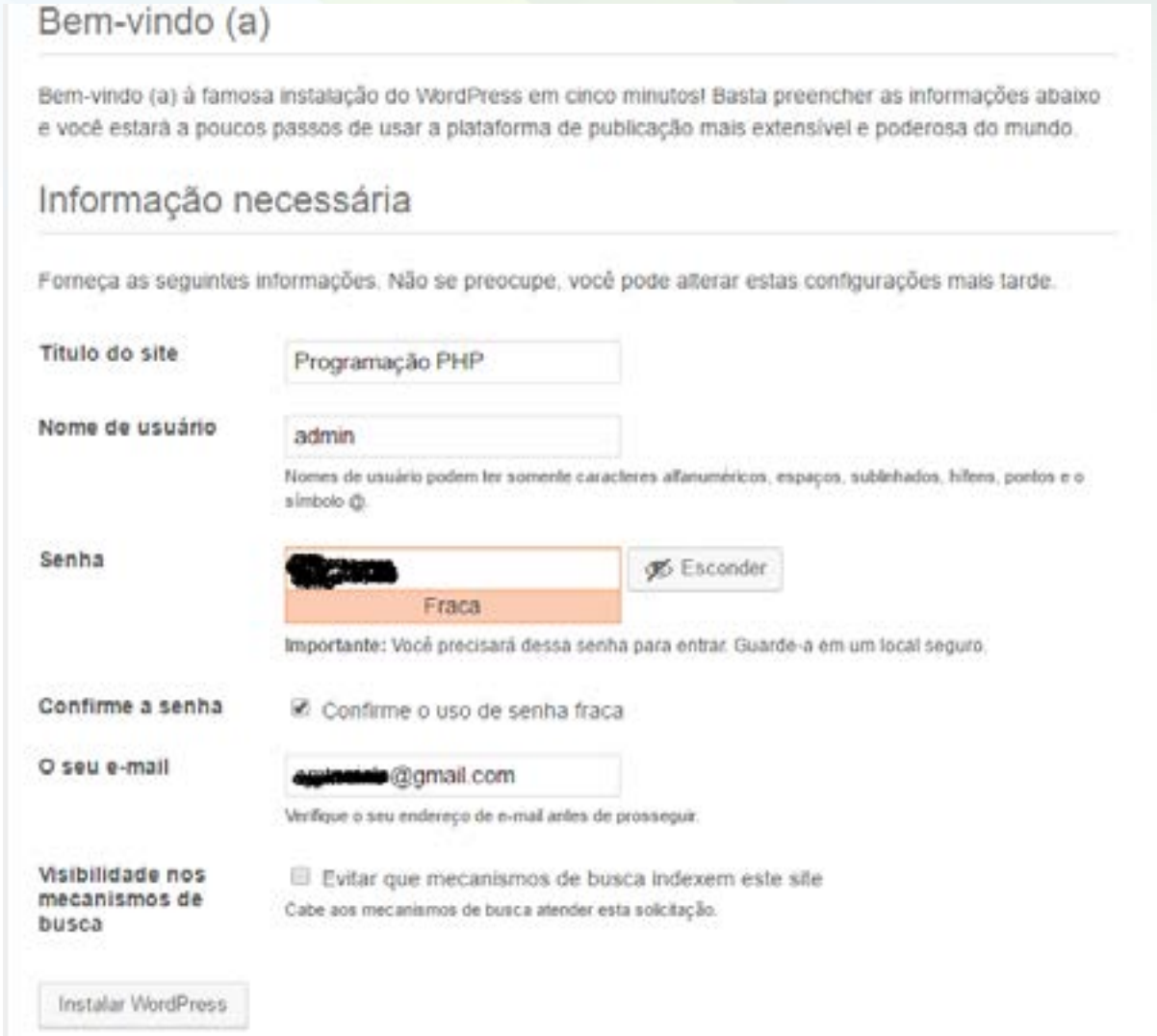
dados. Para o servidor do banco de dados, nomeou-se como localhost. Para ambiente de teste local e o prefixo para o wordpress, por padrão, deve-se deixar wp_.



Feito isso, a próxima tela confirmará a comunicação com o banco de dados, permitindo a instalação do WordPress no ambiente local. Clique no botão Instalar.



As configurações a seguir são referentes à configuração da plataforma WordPress para uso: coloque um Título para o site, nome do usuário que o administrará, uma senha e sua confirmação, e um e-mail de contato do administrador do site.


Bem-vindo (a)
Bem-vindo (a) à famosa instalação do WordPress em cinco minutos! Basta preencher as informações abaixo e você estará a poucos passos de usar a plataforma de publicação mais extensível e poderosa do mundo.
Informação necessária
Forneça as seguintes informações. Não se preocupe, você pode alterar estas configurações mais tarde.
Título do site
Programação PHP
Nome de usuário
admin
Nomes de usuário podem ter somente caracteres alfanuméricos, espaços, sublinhados, hífens, pontos e o símbolo @.
Senha
Esconder
Fraca
Importante: Você precisará dessa senha para entrar. Guarde-a em um local seguro.
Confirme a senha
Confirme o uso de senha fraca
O seu e-mail
@gmail.com
Verifique o seu endereço de e-mail antes de prosseguir.
Visibilidade nos mecanismos de busca
Evitar que mecanismos de busca indexem este site
Cabe aos mecanismos de busca atender esta solicitação.
Instalar WordPress

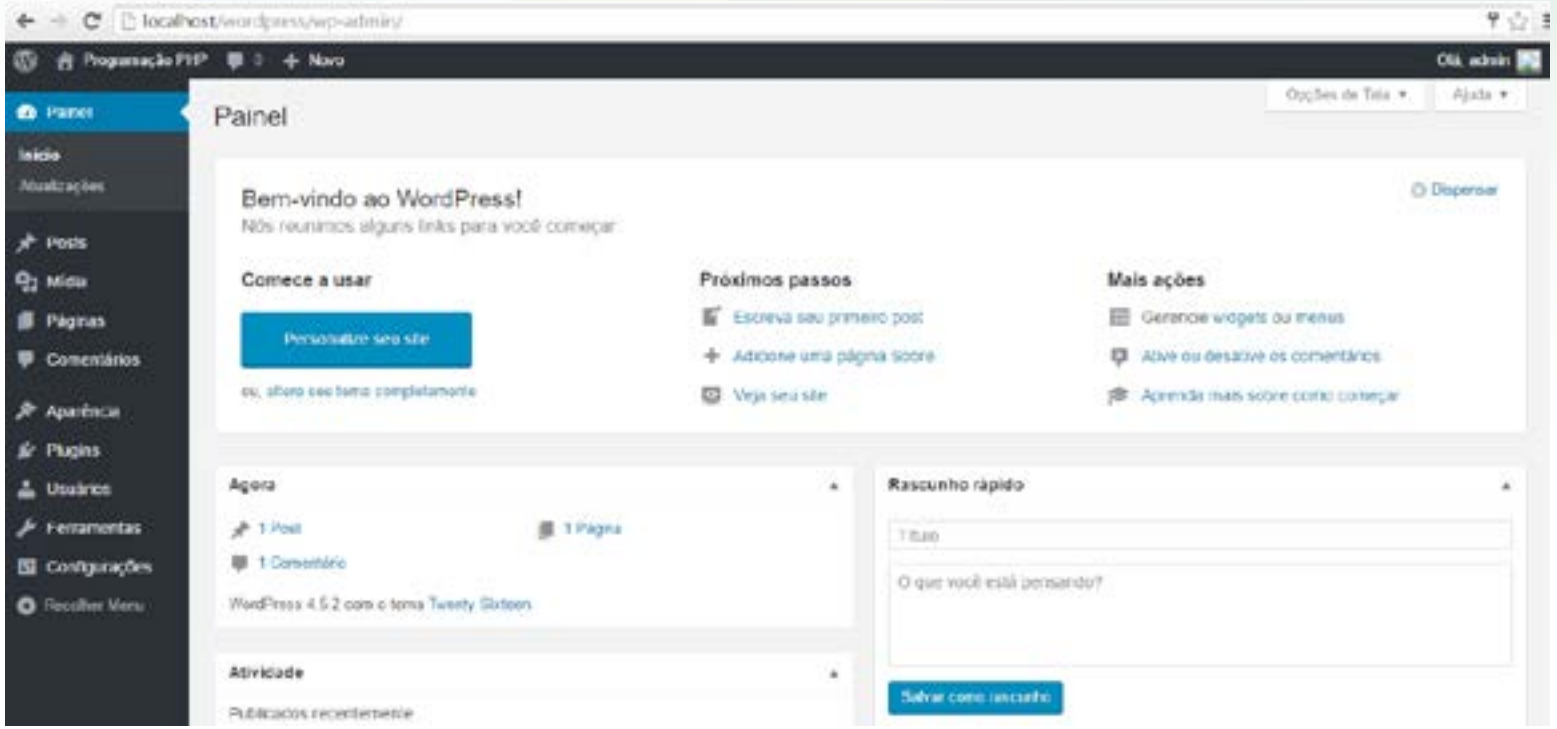
Caso a instalação seja feita sem nenhum problema, a tela a seguir será exibida para fazer o login no sistema e começar a usar o WordPress criado localmente. Ao clicar no botão Fazer Login, aparecerá a tela de Login e Senha no WordPress.



Na tela a seguir, o usuário ou desenvolvedor deve inserir seu e-mail ou nome de usuário e sua senha cadastrada no início da instalação. Feito isso, clique no botão Fazer Login.



Esta tela é a inicial do sistema que tem todas as opções de configuração de um WordPress, como a inserção de postagens, criação de temas, entre outras.


Painel
Bem-vindo ao WordPress!
Comece a usar
Próximos passos
Mais ações
Agora
Rascunho rápido
Atividade

Glossário

CMS: *Custom Management System* ou Sistema de Gerenciamento de Conteúdo é um tipo de aplicação que tem como finalidade a criação, a recuperação, a publicação e o gerenciamento de conteúdo de sites, blogs e outras aplicações para WEB.

Open-source: em termos gerais, significa código aberto, que permite a alteração de código-fonte para que seja usado em diferentes fins, para os quais seu uso é estritamente técnico.

Folhas de estilo: arquivos de configuração de temas em websites que têm a funcionalidade de formatação tanto de seu *layout* quanto de sua aparência, como estilos de cor e texto.

Questão para reflexão



Nesta aula tema, aprendemos sobre a aplicação WordPress, *open-source* e usada para gerenciamento de conteúdo. A proposta para reflexão é a seguinte: quais são os aplicativos open-source que concorrem com o WordPress no quesito conteúdo? Quais são os pontos fortes e fracos de cada um deles?

Considerações Finais

- Estudamos os conceitos de aplicativos open-source.
- No tema proposto, quais são os conceitos sobre CMS e gerenciador de conteúdo.
- Como adquirir o aplicativo WordPress e instalá-lo.
- Instalação do WordPress pelo navegador WEB.
- Criação de usuário administrador do site e sua respectiva senha.

Referências

CHAGAS, F. et al. Um estudo sobre os sistemas de gerenciamento de conteúdo de código aberto. Technical Report Universidade Federal de Goiás. Techinical Report – RT-INF_002-08 – Relatório Técnico – Dezembro 2008.

HEDENGREN, T. D. **Smashing WordPress**: além do blog. 2. ed. Porto Alegre: Bookman, 2012.

Open Source Initiative. The Open Source Definition. Disponível em: <https://opensource.org/osd>. Acesso em: 8 set. 2016.

Assista a suas aulas



Aula 7 - Tema: Sistemas Open-Source em PHP, Criação e Apresentação - Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/08723d10f171c42fbc39c938d8773181>.



Aula 7 - Tema: Sistemas Open-Source em PHP, Criação e Apresentação - Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/c33754301db9fe02d6de1b57de83d92d>.

Questão 1

1. Atualmente, qual é a vantagem de se usar um gerenciador de conteúdo?

a) Pela necessidade de aparecer na WEB simplesmente.

b) Porque as empresas necessitam de WebSites poderosos e custosos.

c) Pela necessidade de criar conteúdo na WEB, sem a ajuda de ninguém.

d) Pelo fato de que as empresas e as pessoas precisam mostrar, gerenciar e criar conteúdo na WEB de forma simples.

e) Pelo fato de que as empresas não necessitam colocar conteúdo na WEB, e sim para aparecer mais do que os concorrentes.

Questão 2

2. Qual é a função de um gerenciador de conteúdo?

a) Permitir que os usuários possam escrever o que quiser na WEB e pagar para inserir esse conteúdo em uma página.

b) Permitir que os usuários possam “colocar” conteúdo na WEB e ter todas as ferramentas para a gestão deste.

c) Permitir que as empresas possam criar blogs para gerenciar conteúdo sem a necessidade de um WebSite profissional.

d) Garantir a qualidade de conteúdo “colocado” na WEB.

e) Garantir que os usuários tenham qualidade no que escrevem para a WEB.

Questão 3

3. Em relação às formas de instalação do WordPress, qual é a alternativa correta?

a) Somente as instalações guiadas e em um clique.

b) Instalação manual, instalação em um clique e instalação por FTP.

c) Instalação guiada, instalação manual e instalação em um clique.

d) Instalação guiada, instalação normal e instalação em um clique.

e) Instalação com wizard, instalação manual e instalação em um clique.

Questão 4

4. Podemos dividir um gerenciador de conteúdo em três áreas, quais são elas?

a) Criação de layout, entrega de conteúdo e recuperação de conteúdo.

b) Criação de conteúdo estático, entrega de conteúdo e recuperação de conteúdo.

c) Criação de conteúdo, entrega de conteúdo estático e recuperação de conteúdo.

d) Criação de conteúdo, entrega de conteúdo e recuperação de parte do conteúdo.

e) Criação de conteúdo, entrega de conteúdo e recuperação de conteúdo.

Questão 5

5. Por que na instalação manual do WordPress precisamos dos programas separados?

a) Pela necessidade de se configurar todos os dados de instalação.

b) Pela possibilidade de criar uma instalação local antes de enviar o site para a WEB.

c) Porque podemos definir o que realmente deverá funcionar.

d) Sistema de informação local e possibilidade de enviar somente as páginas para a WEB.

e) Porque é mais útil do que instaladores padrão.

Gabarito

1. Resposta: D.

Atualmente, com todos os dispositivos que existem, empresas e usuários têm a necessidade de colocar conteúdo na WEB. No entanto, além de poder gerenciar este conteúdo e criar novos, isso deve ser feito de forma simples. Por isso, o uso de gerenciadores de conteúdo com suas aplicações e sistemas empresarias a cada dia tem mais adeptos.

2. Resposta: B.

Sua função é de criação, entrega e gerenciamento de conteúdo, podendo ser feito por qualquer usuário que tenha um pouco de experiência em navegar e criar sites WEB.

3. Resposta: C.

Estes tipos de instalação são referentes às possibilidades que existem no mercado para hospedagem de sites WordPress, podendo o usuário escolher qual melhor lhe convém para o momento e aplicação.

4. Resposta: E.

As divisões são feitas por causa da existência de gerenciadores de conteúdo, que executam cada uma das funções separadamente, ou seja, ou só criam conteúdo, ou só o publicam, e assim por diante. O uso depende do objetivo a ser alcançado.

5. Resposta: B.

Essa configuração serve para criarmos uma instalação local, ou seja, para que o desenvolvedor entenda como as configurações são feitas, pois, ao chegar no mercado de trabalho, muita coisa estará pronta para uso e não haverá a possibilidade de reinstalação.

Unidade 8

Práticas com Frameworks PHP Criação e Apresentação

Objetivos

1. Os objetivos deste tema são focados na aprendizagem de um *framework* para o desenvolvimento de aplicações em PHP, que tem como principal finalidade a agilidade no processo de desenvolvimento de aplicações. O nome deste *framework* é CodeIgniter que também é open-source e tem grande facilidade de instalação.
2. Assim, o aluno saberá como adquiri-lo, fazer sua instalação, testá-la e criar uma aplicação com a finalidade de entender esse processo.

Introdução

Atualmente, quando existe a possibilidade de uma empresa desenvolver uma aplicação WEB, uma das perguntas que deve ser respondida é em relação ao tempo de desenvolvimento. Sendo assim, surgiram as aplicações chamadas *frameworks*, que têm como finalidade agilizar esse processo de desenvolvimento, ajudando a criar novas aplicações. Além disso, por conter uma quantidade de bibliotecas relacionada a diversas características de um WebSite que não seria viável se o desenvolvimento partisse do zero, ou seja, fazendo que o time de desenvolvedores tivesse de criar todas essas ferramentas.

O CodeIgniter é um *framework* para o desenvolvimento de aplicações WEB usando a linguagem PHP. Ele tem muitas características que vão desde sua documentação, bem vasta e que cobre todos os aspectos do *framework*, até sua utilidade em ambiente de desenvolvimento compartilhado e com uma performance de processamento excepcional (GRIFFITHS, 2010).

Este *framework* utiliza-se do conceito de desenvolvimento baseado em *Model View Controller* (MVC), um padrão moderno para o desenvolvimento de aplicações tanto para a WEB quanto para Cliente/Servidor. Neste o Model é a camada que representa o domínio da aplicação, ou seja, que trabalha com a representação de qualquer tipo de dados. Já a View representa a interface com o usuário, isto é, as informações

apresentadas na tela do navegador para os usuários (GRIFFITHS, 2010).

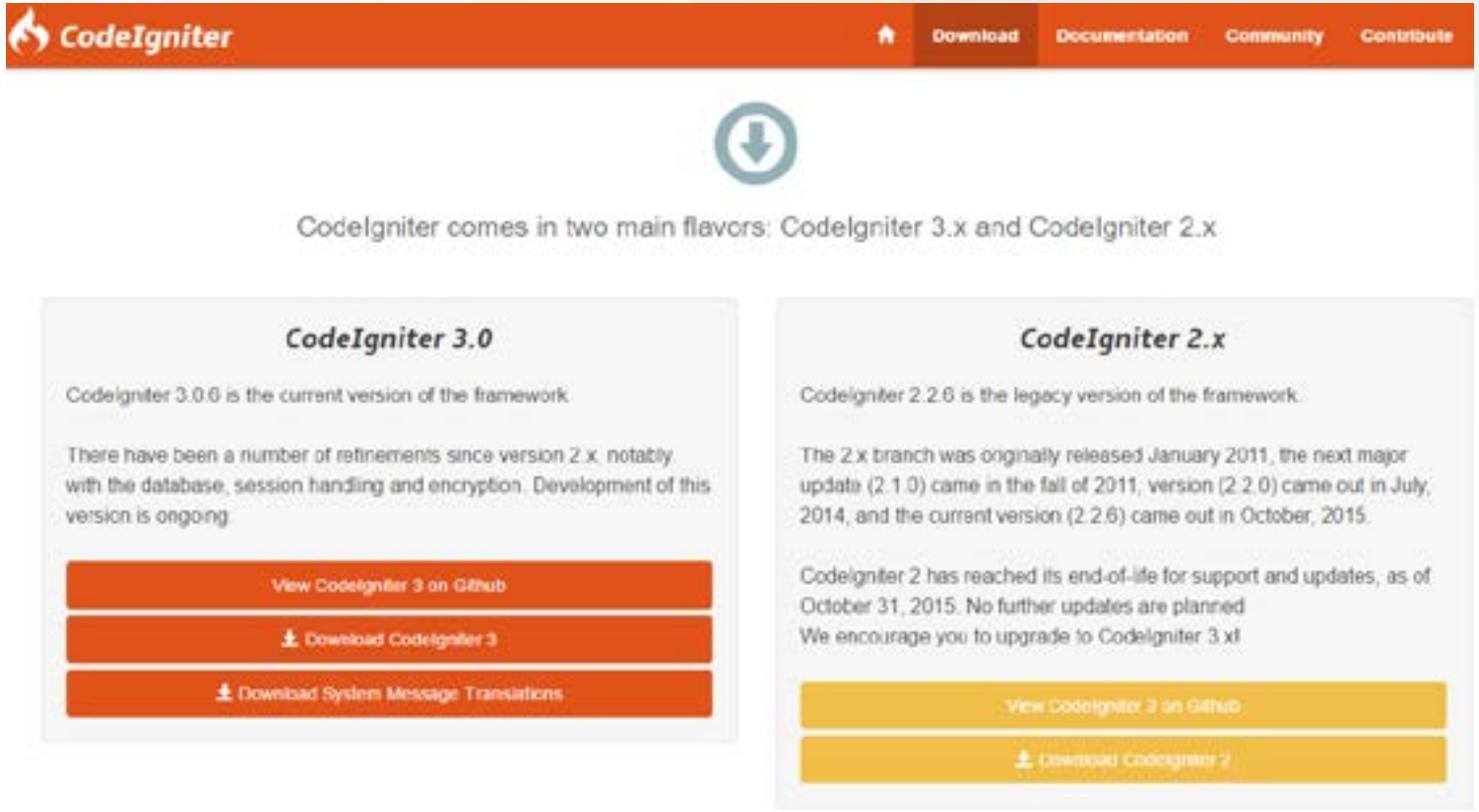
Controller é a camada que representa os eventos ou ações dos usuários, como validação de dados, além de filtrar as informações passadas por eles. De forma mais objetiva, é a camada da lógica de negócios da aplicação.

Este tipo de arquitetura de software tem o funcionamento da seguinte forma, quando o usuário faz uma solicitação por meio de um formulário, esta é enviada para o Controller que, por sua vez, envia para o Model. Este faz a análise dessa solicitação e devolve a resposta para aquele que devolve o resultado para a View. Esta renderiza essa informação e mostra o resultado na tela para o usuário.

Os próximos passos são a aquisição do programa, sua instalação e configuração.

2. Fazendo o *download* do CodeIgniter

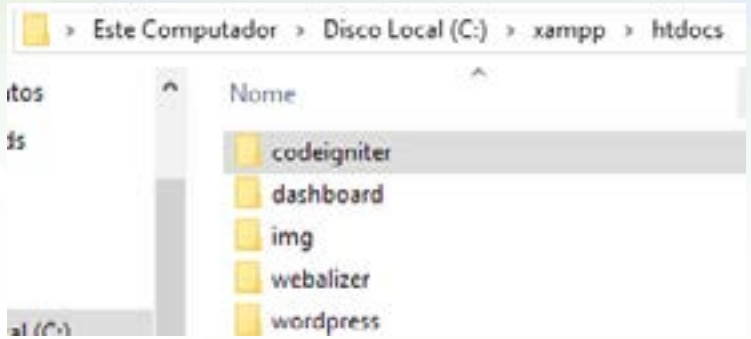
O CodeIgniter pode ser encontrado neste endereço da WEB: <https://www.codeigniter.com/download> (acesso em: 8 set. 2016), no qual você será redirecionado para a seguinte página, com possível atualização de versão.



Para este material foi usada a versão 3.0.6, mas nada impede o aluno de testar a versão anterior. Feito o *download*, descompacte o arquivo CodeIgniter-3.0.6. Dentro dele existem três pastas: *application*, *system* e *user_guide* que, respectivamente, tratam de configurações de aplicação, sistema e um guia de usuário; manual do CodeIgniter que pode ser usado

localmente. A pasta *system* contém todas as bibliotecas do CodeIgniter, *logs* de configuração e arquivos de cache (GRIFFITHS, 2010).

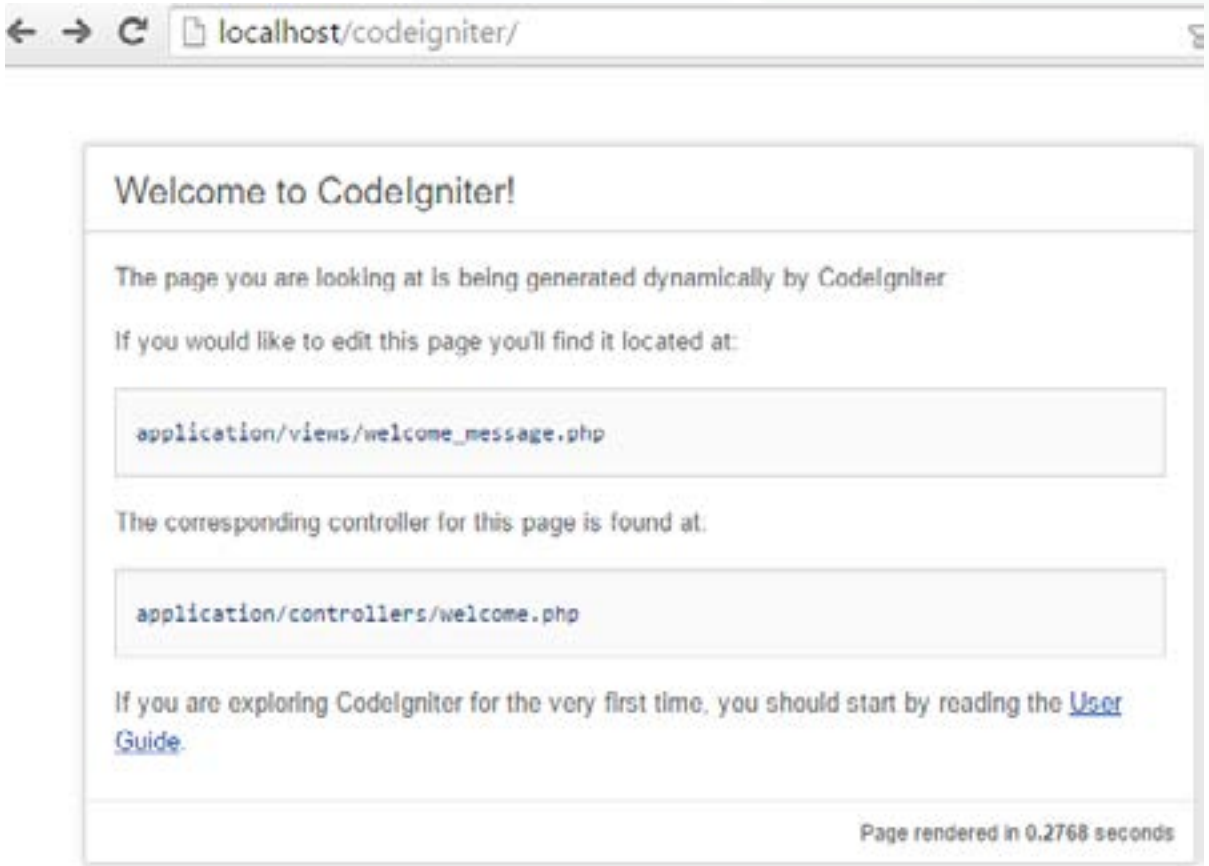
A pasta *application* contém o arquivo da página principal do CodeIgniter, apresentada quando a configuração for realizada de forma correta. Além de controllers e outros arquivos, esta é a pasta na qual serão criados as Model-View-Controller (MVC) e outros arquivos dos projetos de WebSites desenvolvidos por cada desenvolvedor. Depois de descompactada, copie esta pasta a *htdocs* do diretório principal do XAMPP, como ilustra a imagem a seguir:



Note o caminho que está destacado e a pasta CodeIgniter criada dentro da *htdocs*. Perceba também que o nome dela foi alterado para que seja mais fácil sua digitação na barra de endereços do navegador WEB. Na seção seguinte, será feita a configuração do CodeIgniter para que ele possa ser testado no navegador.

3. Configurando e rodando o CodeIgniter

Na pasta XAMPP\htdocs\codeigniter\application\config, encontre o arquivo config.php e o abra para edição. Procure a linha $config[‘base_url’] para que seja configurada a URL base do site. No caso da configuração da pasta criada no passo anterior, essa URL fica da seguinte forma:



Note que o protocolo http:// foi suprimido, isto porque o navegador WEB tem como padrão este protocolo, portanto, pode ser escrito ou suprimido do endereço. Como pode ser percebido, essa é a página que aparece quando a configuração ocorre de forma correta. Nela é descrito que foram encontrados a View, que é a própria página renderizada, e o código de Controller, que faz referência às ações do usuário, no caso, devolver a página de boas-vindas para a View referente à sua renderização, exibindo-a na tela do navegador.

Ao acessar a pasta application\config\, podemos notar que o controller padrão aponta para a página welcome.php. Isso será modificado sempre que necessário para acessar as páginas de outras aplicações.

3.1. Conceitos básicos do CodeIgniter

O *framework* CodeIgniter baseia-se no conceito de MVC, abordado anteriormente. Para entender seu funcionamento, temos que saber o que são URLs, Controllers, Rotas e Views. A partir desse momento, será abordado cada um destes.

URLs são escritas no CodeIgniter da seguinte forma: www.siteweb.com/classe/metodo/

Essa definição é dada por ser fácil para o padrão MVC e por ser mais encontrada

pelos mecanismos de busca na internet. Tecnicamente falando, o primeiro parâmetro faz referência ao Controller, que é uma classe, e o segundo ao Método desta, como no exemplo a seguir:

www.siteweb.com/estudantes/listar/

Podemos notar que a classe/controller Estudantes será instanciada, utilizando-se do método listar() desta classe.

Os controllers são os arquivos responsáveis por controlar quais e como as informações serão apresentadas pelo usuário, como mostra o exemplo a seguir, no qual dentro da pasta controllers, criaremos o arquivo chamado loja.php:

```
1  <?php  if ( ! defined('BASEPATH')) exit('No direct script access allowed');
2
3  class Loja extends CI_Controller {
4          public function index()
5          {
6                  echo 'Bem vindo a página de Vitrine da Loja';
7          }
8          public function livros()
9          {
10                 echo 'Ofertas de livros de Tecnologia';
11         }
12 }
```

Observe que temos dois métodos dentro dessa classe que, por padrão, executará o método index e exibirá a mensagem dentro dele com base na seguinte chamada no navegador:

localhost/loja/

Para acessar o segundo método, devemos usar a seguinte URL: localhost/loja/livros

Dessa forma, será exibida a segunda mensagem referente à oferta de livros de tecnologia. No entanto, a fim de alterarmos a página padrão do CodeIgniter para nossa aplicação, devemos editar o arquivo routes.php, que se encontra na seguinte pasta application\config\routes.php:

```
41  $route['default_controller'] = "Loja";
```

Antes estava o nome da classe Welcome, agora, será Loja. Digitando o endereço localhost/codeigniter/, seremos redirecionados para a página de Vitrine da loja, como pode ser visto na seguinte imagem:



Isso mostra como é extremamente fácil usar o CodeIgniter para a criação de WebSites. O próximo conceito é o das Views, ou seja, dos códigos que exibirão os dados e as informações na tela para os usuários. Para isso, criamos o arquivo

usersview.php que contém o código HTML a seguir, incluindo uma head com uma mensagem e, dentro das *tags* body, a mensagem que será exibida na tela:

```
<html>
    <head>
        <title>View de Usuários</title>
    </head>
    <body>
        <h1>View de Usuários</h1>
    </body>
</html>
```

O seguinte passo é a criação do arquivo de Controller para gerenciar esta View, que será nomeado como users.php e representado na figura a seguir:

```
<?php  if ( ! defined('BASEPATH')) exit('No direct script access allowed');

class Users extends CI_Controller {
        public function index()
        {
            /* Carrega a view */
            $this->load->view('usersview');
        }
}
```

Podemos notar neste código que a função index chama a função view (‘usersview’), que tem a view criada anteriormente como parâmetro dentro do método load referenciado pela classe

Users por meio do $this, ou seja, usando recursos de orientação a objetos. Com base nesses conceitos, pode-se criar quantas páginas (Views) forem necessárias e a mesma quantidade de Controllers para tratar os eventos de cada uma das classes.

Glossário

MVC: Model View Controller, arquitetura de desenvolvimento de sistemas que permite ao desenvolvedor separar e organizar tudo que é referente à exibição de informações na tela (Views), controlar o que pode ser acessado ou consultado no sistema (Controller) e fazer sua lógica em arquivos de códigos mais organizados e de fácil manutenção (Models).

Framework: programas que permitem a criação de novos programas e aplicações com maior rapidez e dentro de padrões usados, atualmente, pelo mercado de desenvolvimento de software que está relacionado à Programação Orientada a Objetos.

CodeIgniter: Um *framework* que permite o desenvolvimento ágil com base em bibliotecas e funções já prontas para uso, sem a necessidade de criar tudo do início, com a linguagem de programação PHP.

Questão para reflexão



Neste tema, foram mostrados como adquirir o CodeIgniter, como instalá-lo, configurá-lo e como criar exemplos simples e rápidos para mostrar a qualidade e agilidade no desenvolvimento de aplicações para WEB. Assim, como proposta de autoestudo temos a criação de uma aplicação mais elaborada, usando o conceito de Models juntamente com as Views e os Controllers, além de fazer que essas Models sejam conectadas a um banco de dados em MySQL. É possível o desenvolvimento dessa forma?

Considerações Finais

- Entender os conceitos relacionados a *frameworks* e o uso da arquitetura MVC.
- Adquirir, instalar e configurar o *framework* CodeIgniter para o desenvolvimento de WebSites de forma rápida.
- Testar a configuração padrão do CodeIgniter e criar uma aplicação usando Views.
- Criação de uma aplicação usando Views e Controllers para mostrar os métodos do CodeIgniter.

Referências

GRIFFITHS, Adam. **CodeIgniter 1.7 Professional Development**. Birmingham: Editora Packt Publishing. 2010.

CODEIGNITER. Disponível em: <https://www.codeigniter.com/download>. Acesso: 8 maio 2016.

Assista a suas aulas



Aula 8 - Tema: Práticas com Frameworks PHP Criação e Apresentação - Bloco I

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/b3456caac2dec650ec146544f2a67731>.



Aula 8 - Tema: Práticas com Frameworks PHP Criação e Apresentação - Bloco II

Disponível em: <http://fast.player.liquidplatform.com/pApiv2/embed/dbd3957c747affd3be431606233e0f1d/9fa323f2792b34c2c42d7e9503e4a234>.

Questão 1

1. O que é o modelo MVC?

a) Padrão antigo de desenvolvimento de aplicações tanto para a WEB quanto para o Cliente/Servidor.

b) Padrão moderno de desenvolvimento de aplicações somente para a WEB.

c) Padrão moderno de desenvolvimento de aplicações para o Cliente/Servidor.

d) Padrão moderno de projeto de aplicações tanto para a WEB quanto para o Cliente/Servidor.

e) Padrão moderno de desenvolvimento de aplicações tanto para a WEB quanto para o Cliente/Servidor.

Questão 2

2. Qual é a grande vantagem de um *framework*?

a) Usar somente as bibliotecas que o PHP não têm.

b) Usar bibliotecas, views, controllers e outras ferramentas para agilizar o desenvolvimento de aplicações.

c) Grande quantidade de arquivos existentes na sua instalação.

d) Com certeza, não começar as coisas do zero.

e) Falta de documentação requerida para aprender a programar.

Questão 3

3. Como é a comunicação dentro do modelo MVC?

a) Uma View solicita uma informação, outra recebe e analisa, enviando para uma Model que retorna a solicitação para o Controller, o qual, por sua vez, envia de volta para a View exibir a informação.

b) Uma View solicita uma informação, um Controller recebe e analisa, enviando para uma Model que retorna a solicitação para o Controller, o qual, por sua vez, envia de volta para a Model exibir a informação.

c) Uma View solicita uma informação, um Controller recebe e analisa, enviando para uma View que retorna a solicitação para o Controller, o qual, por sua vez, envia de volta para a Model exibir a informação.

d) Uma View solicita uma informação, um Controller recebe e analisa, enviando para uma Model que retorna a solicitação para o Controller, o qual, por sua vez, envia de volta para a View exibir a informação.

e) Uma View solicita uma informação, um Controller recebe e analisa, enviando para uma Controller que retorna a solicitação para o Model, o qual, por sua vez, envia de volta para a View exibir a informação.

Questão 4

4. Onde alteramos a View principal do CodeIgniter para a nossa View?

a) No arquivo config.php linha $route[‘default_controller’].

b) No arquivo routes.php linha $config[‘default_controller’].

c) No arquivo routes.php linha $route[‘default_controller’].

d) No arquivo routes.php linha $route[‘default_view’].

e) No arquivo config.php linha $route[‘default_config’].

Questão 5

5. Por que usar o CodeIgniter no desenvolvimento de aplicações para a WEB?

a) Pela facilidade na instalação, pouca documentação e por trabalhar em ambientes compartilhados.

b) Pela facilidade na instalação, pouca documentação e por não trabalhar em ambientes compartilhados.

c) Pela dificuldade na instalação, documentação abrangente e por trabalhar em ambientes compartilhados.

d) Pela facilidade na instalação, documentação abrangente e por trabalhar em ambientes compartilhados.

e) Pela dificuldade na instalação, documentação restrita e por trabalhar em ambientes compartilhados.

Gabarito

1. Resposta: E.

Atualmente, o desenvolvimento de sistemas está cada dia mais pautado em um modelo de desenvolvimento, sendo o MVC um dos que mais atraem desenvolvedores pelo simples fato de organizar o desenvolvimento dos códigos em arquivos separados, obedecendo a Orientação a Objetos.

2. Resposta: B.

Essa vantagem de não ter que criar tudo a partir do zero permite que os desenvolvedores possam criar aplicações com maior rapidez, com o uso dos recursos mencionados desde bibliotecas de conexão até banco de dados que facilitam e permitem a manutenção das aplicações.

3. Resposta: D.

Esse caminho é lógico, pois tudo parte de um formulário (View) que tem como função passar as informações para os Controllers e para as Model. Assim, evita-se que requisições sem informações possam ser processadas pelas Models (o que causaria erro).

4. Resposta: C.

Arquivo de rota que com a configuração padrão sempre apontaria para a página *welcome* do CodeIgniter e não para a do sistema que está sendo criado.

5. Resposta: D.

Dos *frameworks* mais usados para PHP no mercado, este é o que mostra maior facilidade em tudo, até mesmo na curva de aprendizagem, uma vez que é estruturado e sua documentação, além de abranger todos seus conceitos, é bem didática.

